

ARQUITECTURA Y DESARROLLO / APLICACIÓN MÓVIL

ARCHIVO MAESTRO — App móvil riachuelo-mobile

Sistema inteligente para el monitoreo y detección de plagas en cultivos de vid — Agrícola Riachuelo S.A.C. (Ica)

Guía única para construir la app por módulos con ayuda de una IA.

CÓMO USARLO

Este PDF tiene el mismo contenido que docs/MAESTRO_APP_MOVIL.md. El archivo Markdown es el que se entrega a la IA dentro del repositorio; este PDF es para el equipo y el docente.

Archivo	docs/MAESTRO_APP_MOVIL.md (este archivo)
Versión	1.0 — 01/10/2026
Estado	Vigente para la construcción. Cada cambio de contrato sube la versión (sección 25).
Alcance	Solo la app móvil (Android e iOS, 3 celulares, 3 funciones). Backend, web e IA se mencionan solo donde la app se conecta con ellos.
Base documental	Especificación técnico-funcional v2.0; Stack Tecnológico v2.1; documento académico «Curso Integrador II»; archivo «MAPEO CHANCHITO MOLINOS actualizado»; Especificación App Móvil v1.0; decisiones del equipo del 01/10/2026.
Precedencia	Para construir la app, este archivo prevalece sobre la Especificación App Móvil v1.0 y sobre las secciones móviles de la Especificación v2.0 y del Stack v2.1. No cambia la arquitectura general del sistema.

Contenido

0. Cómo usar este archivo	6
0.1 Qué es	6
0.2 Cómo lo usa el equipo	6
0.3 Cómo lo lee la IA (orden obligatorio)	6
0.4 Estados de validación usados en este archivo	6
0.5 Qué se validó en esta versión	6
1. Reglas permanentes para la IA (obligatorias)	7
2. Contexto del proyecto (resumen)	8
3. Alcance de la app	8
4. Decisiones vigentes	9
4.1 Cambios respecto de documentos anteriores	10
5. Stack, versiones y configuración del proyecto	10
5.1 Base (Verificado)	10
5.2 Módulos y librerías	10
5.3 Creación del proyecto (Tarea T-00)	11
5.4 app.json de referencia	12
5.5 eas.json de referencia (Android)	12
5.6 Variables de entorno	13
5.7 Tipos de versión de la app	13
5.8 Entorno de desarrollo	13
6. Arquitectura y estructura del código	13
6.1 Capas y reglas de dependencia	13
6.2 Estructura de carpetas	14
6.3 Interfaces de transporte (contrato)	15
6.4 Arranque de la app	15
7. Usuarios, cuentas, inicio de sesión y funciones del dispositivo	16
7.1 Roles de usuario y acceso a la app	16
7.2 Estados de cuenta	16
7.3 Registro (con internet)	16
7.4 Inicio de sesión con internet	16
7.5 Inicio de sesión sin internet	17
7.6 Contraseña temporal y cambio de contraseña (con internet)	17
7.7 Recuperación vía administrador	17
7.8 Cierre de sesión de usuario y revocación	18
7.9 Función del dispositivo	18
7.10 Qué hace cada función	18
7.11 Sesión de usuario al reabrir la app	18
7.12 «Sesión de monitoreo abierta» en cada función	19
8. Flujos de la app	19

8.1 Primer uso de un celular	19
8.2 Preparación con internet (antes de salir)	19
8.3 Emparejamiento y prueba corta (en campo, sin internet)	19
8.4 Pasada y bucle de captura	19
8.5 Repetición por calidad	20
8.6 Cambio de marcador o segmento	20
8.7 Cambio de lateral, repetición de lateral y cierre de hilera	20
8.8 Cierre de la sesión de monitoreo	20
8.9 Sincronización al volver	20
8.10 Recuperación tras cierre o reinicio	21
8.11 Mapa de navegación	21
8.12 Cierre del contexto en la cámara	21
8.13 Fotos que llegan después del cierre	21
9. Catálogo de pantallas	22
10. Requerimientos	24
10.1 Requerimientos funcionales	24
10.2 Requerimientos no funcionales	26
10.3 Reglas de negocio	27
10.4 Historias de usuario	28
11. Contratos de dominio (TypeScript)	28
11.1 Tipos — src/domain/types.ts	28
11.2 Transiciones permitidas — src/domain/stateMachines.ts	32
11.3 Estado de la secuencia — src/domain/sequence.ts	33
12. Base de datos local	34
12.1 Esquema v1 — src/storage/migrations/001_initial.ts	34
12.2 Migraciones — src/storage/db.ts	38
12.3 SecureStore	38
12.4 Archivos	39
12.5 Reglas de escritura	39
13. Estados: eventos y condiciones	39
14. Protocolo local v1	42
14.1 Canales y puertos	42
14.2 QR de emparejamiento	42
14.3 Sobre y reglas comunes	42
14.4 Catálogo de mensajes	43
14.5 Tipos del protocolo — src/protocol/messages.ts	44
14.6 Emparejamiento paso a paso	47
14.7 Orden de captura paso a paso	48
14.8 Transferencia de fotos	48
14.9 Heartbeat, desconexión, reconexión y RESYNC	49
14.10 Temporizadores (valores en src/config, sección 17)	50

14.11 Implementación del servidor de control	50
14.12 Prueba corta (contrato)	50
15. Contrato con el backend (/api/v1)	51
15.1 Reglas generales	51
15.2 Endpoints	51
15.3 DTO — src/api/dto.ts	52
15.4 Ejemplos	55
15.5 Orden de sincronización y errores	56
15.6 Backend simulado (Fase 1)	56
16. Calidad de imagen	57
16.1 Qué decide	57
16.2 Procedimiento	57
16.3 Perfil Q0 (valores iniciales, no validados)	57
16.4 Rendimiento	58
16.5 Calibración del perfil Q1 (Fase 6)	58
17. Configuración — src/config/defaults.ts (CFG-1)	58
18. Seguridad y privacidad	61
19. Errores y mensajes al usuario — src/ui/messages.ts	61
20. Registro de eventos, diagnóstico e indicadores	63
20.1 event_log	63
20.2 Exportar diagnóstico	63
20.3 Indicadores que deben poder calcularse con los datos de la app	63
21. Pruebas	63
21.1 Pruebas unitarias obligatorias	63
21.2 Casos de prueba funcionales (manuales)	64
21.3 Prueba técnica obligatoria de comunicación local	65
21.4 Fallos inducidos	65
21.5 Combinaciones de dispositivos	66
21.6 Evidencia de prueba	66
22. Plan de construcción por tareas	66
Fase 0 — Preparación	66
Fase 1 — Base en iPhone con Expo Go	66
Fase 2 — Red local con dos teléfonos (Android, Development Build)	69
Fase 3 — Tres teléfonos	70
Fase 4 — Sincronización	70
Fase 5 — Validación en iOS	70
Fase 6 — Campo y versión del piloto	70
23. Plantillas	71
23.1 Pedido a la IA	71
23.2 Reporte de avance (lo entrega la IA)	71
23.3 Registro de decisión (ADR) — docs/adr/ADR-NNN-titulo.md	71

24. Registro de avance	71
25. Decisiones registradas y preguntas abiertas	72
25.1 Versiones de este archivo	72
25.2 Decisiones técnicas (ADR)	72
25.3 Preguntas abiertas	72
26. Glosario	73
27. Fuentes y trazabilidad	73
Anexo A. Código de referencia probado: red local	74
A.1 Servidor WebSocket mínimo — src/local-network/wsCodec.ts	74
A.2 Receptor HTTP de fotos (opción A) — src/local-network/httpUpload.ts	78
Anexo B. Código de referencia: calidad de imagen	80
B.1 Métricas y decisión (probado) — src/device/quality.ts	80
B.2 Integración con Expo (boceto que compila contra las firmas del SDK 54) — src/device/qualityPipeline.ts	81
Anexo C. Código de referencia probado: autenticación, validaciones y reintentos	82
C.1 Login sin internet — src/auth/offlineAuth.ts	82
C.2 Validaciones de formularios — src/auth/validation.ts	84
C.3 Reintentos y clasificación de errores — src/sync/retry.ts	84
Anexo D. Datos del archivo de mapeo para el backend simulado	85
Anexo E. Pruebas de referencia	86
E.1 tests/domain.test.ts	86
E.2 tests/wsCodec.test.ts	87
E.3 tests/httpUpload.test.ts	88
E.4 tests/quality.test.ts	90
E.5 tests/offlineAuth.test.ts	91
E.6 tests/validation.test.ts	91
E.7 tests/retry.test.ts	92

0. Cómo usar este archivo

0.1 Qué es

Es el contexto completo que necesita una IA (o un desarrollador) para construir la app **por partes**: reglas, decisiones, stack con versiones, arquitectura, pantallas, requerimientos con criterios de aceptación, contratos exactos (tipos, base de datos, protocolo, API, configuración), código de referencia probado, pruebas y un plan de tareas ordenado. Si un dato no está aquí, la IA no debe inventarlo: debe preguntarlo (regla R-06).

0.2 Cómo lo usa el equipo

1. Guardar este archivo en el repositorio como docs/MAESTRO_APP_MOVIL.md.
2. Crear en la raíz del repositorio un archivo AGENTS.md (o CLAUDE.md si usan Claude Code) con una sola instrucción: «Lee docs/MAESTRO_APP_MOVIL.md antes de cada tarea y cumple la sección 1».
3. Pedir a la IA **una tarea por vez** de la sección 22 usando la plantilla de la sección 23.1.
4. Probar el resultado en el celular, revisar el código y hacer commit.
5. Actualizar la sección 24 (avance) y, si se decidió algo, la sección 25 (decisiones y preguntas).
6. Si la IA tiene poca memoria de contexto, darle siempre las secciones 1 a 7 más las secciones que cita la tarea.

0.3 Cómo lo lee la IA (orden obligatorio)

1. Sección 1 (reglas permanentes).
2. Secciones 2 a 7 (contexto, alcance, decisiones, stack, arquitectura, cuentas y funciones).
3. La tarea pedida en la sección 22 y todas las secciones que esa tarea cita.
4. La sección 24 para saber qué ya existe en el repositorio.

0.4 Estados de validación usados en este archivo

Estado	Significado
Confirmado	Decidido por el equipo o por los documentos vigentes del proyecto.
Verificado	Comprobado contra los paquetes y la documentación oficial del SDK 54 de Expo o con pruebas automáticas.
Propuesto	Diseño o valor inicial de este archivo. Se implementa tal cual hasta que el equipo lo cambie con evidencia.
PoC	Se decide con la prueba técnica en celulares reales (Fase 2).
Calibrar	Parámetro que se ajusta con mediciones; el código lo lee siempre de la configuración (sección 17).

0.5 Qué se validó en esta versión

- **Versiones del SDK 54:** tomadas de bundledNativeModules.json del paquete expo@54.0.37 (última versión 54 publicada, agosto de 2026) y de la plantilla blank-typescript@sdk-54.
- **APIs de Expo:** revisadas en las declaraciones TypeScript de cada paquete del SDK 54 (expo-sqlite, expo-file-system, expo-secure-store, expo-camera, expo-image-manipulator, expo-location, expo-crypto, expo-network, expo-router, expo-build-properties, react-native-tcp-socket).
- **Código de referencia:** tipos, máquinas de estado, configuración, protocolo, códec WebSocket, receptor HTTP de fotos, calidad de imagen, login sin internet, validaciones y reintentos **compilan con TypeScript 5.9 en modo estricto** y pasan **32 pruebas automáticas** (sección 21.1 y Anexo E). La integración de calidad con Expo (qualityPipeline.ts) compila contra las firmas verificadas y se prueba en el celular en la tarea T-11. El códec WebSocket se probó contra un cliente WebSocket real y el receptor HTTP con una foto de 3 MB.
- **Esquema SQLite:** ejecutado en SQLite 3.45 con claves foráneas activas; 33 comprobaciones de restricciones (estados, laterales, duplicados, un celular por rol, rol liberado, capturas de prueba, colas).
- **Revisión independiente en dos rondas:** la primera encontró 25 inconsistencias (contrato de la prueba corta, datos para reconectar, contexto de las repeticiones, tamaño del RESYNC, fotos que llegan después del cierre,

umbrales para reanudar, tiempos de espera, orden de pantallas y plan de tareas); la segunda verificó esas correcciones y encontró 24 detalles más (resincronización de pasadas ya enviadas, filas de capturas sin foto, orden al guardar el token, acceso a PANT-13, dependencias entre tareas, entre otros). Todo se corrigió en esta versión y los contratos se volvieron a compilar y probar.

- **Correcciones encontradas al validar** (respecto de la Especificación App Móvil v1.0):
 - expo/fetch con un File como cuerpo **carga la foto completa en memoria de JavaScript** antes de enviarla. Para subir fotos se usa uploadAsync de expo-file-system/legacy, que envía el archivo de forma nativa (sección 14.8 y 15.5).
 - expo-crypto del SDK 54 **no incluye PBKDF2**; el login sin internet usa @noble/hashes 1.8.0 (sección 7.5).
 - SecureStore admite valores de **hasta 2048 bytes** y claves con letras, números, ., - y _ (sección 12.3).
 - El Expo Go de la **App Store llega solo hasta el SDK 54**; en Android, el Expo Go del SDK 54 se instala desde expo.dev/go eligiendo el SDK.

1. Reglas permanentes para la IA (obligatorias)

- **R-01 SDK fijo.** Expo SDK 54 (expo ~54.0.x, React Native 0.81.5, React 19.1.0, TypeScript ~5.9.2). No actualizar el SDK. Instalar paquetes de Expo y nativos solo con `npx expo install <paquete>`.
- **R-02 TypeScript estricto.** "strict": true. Sin any salvo en el borde con una librería, y con comentario.
- **R-03 Multiplataforma.** Solo librerías que funcionen en Android e iOS. No agregar dependencias nativas nuevas sin una decisión registrada en la sección 25.
- **R-04 Expo Go en Fase 1.** Todo lo nativo (react-native-tcp-socket, servidor de archivos) se usa solo detrás de las interfaces de la sección 6.3. En Expo Go se usa el simulador. La elección se hace con `isRunningInExpoGo()` exportado por expo (verificado en SDK 54), con `require()` diferido para que el módulo nativo nunca se cargue en Expo Go.
- **R-05 Capas.** Respetar la estructura de la sección 6. Las pantallas no usan SQLite, archivos ni red directamente: llaman a servicios.
- **R-06 No inventar.** No crear requisitos, campos, estados, mensajes del protocolo ni endpoints que no estén aquí. Si falta algo, detenerse y listarlo en «Preguntas» del reporte. Si se decide algo menor para avanzar, marcarlo como «Supuesto».
- **R-07 Contratos exactos.** Los tipos (11), el SQL (12), el protocolo (14), la API (15) y la configuración (17) se implementan tal como están. Cambiarlos exige actualizar este archivo y su versión.
- **R-08 Parámetros en configuración.** Ningún umbral, intervalo, puerto o tiempo va como número suelto: todo sale de `src/config` (sección 17).
- **R-09 Pruebas.** Toda lógica de dominio y servicios tiene pruebas unitarias. Antes de entregar: `npm run typecheck`, `npm run lint` y `npm test` sin errores.
- **R-10 Secretos.** Nunca registrar contraseñas, tokens ni el verificador offline en logs. Nunca guardar contraseñas. Las variables `EXPO_PUBLIC_*` quedan dentro de la app: no son secretas.
- **R-11 Evidencia.** No borrar fotos ni registros salvo por las reglas de retención (RN-09).
- **R-12 Idioma.** Textos de interfaz en español (Perú), centralizados en `src/ui/strings.ts` y `src/ui/messages.ts`. Identificadores de código en inglés. Comentarios en español solo cuando aporten.
- **R-13 Formatos.** Fechas en ISO-8601 UTC (`new Date().toISOString()`). IDs con `Crypto.randomUUID()` de expo-crypto.
- **R-14 Transacciones.** Toda operación que escriba en más de una tabla usa `db.withExclusiveTransactionAsync(async (txn) => ...)` y ejecuta las consultas sobre `txn`.
- **R-15 Uso en campo.** Contraste alto, botones de al menos 48 dp, textos de al menos 16 pt, acciones críticas visibles sin desplazarse, estado de conexión siempre visible.
- **R-16 Reporte.** Al terminar cada tarea, entregar el reporte de la sección 23.2.

- **R-17 Fuera de alcance.** No implementar diagnóstico de plaga, tratamientos, dosis, calendarios ni niveles de urgencia.
- **R-18 Backend.** La app solo habla con la API /api/v1 de Spring Boot. Nunca con PostgreSQL, S3, RabbitMQ ni el servicio de IA.
- **R-19 Primer plano.** La captura y la coordinación ocurren con la app en primer plano y la pantalla encendida. No depender de tareas en segundo plano.
- **R-20 Pocas dependencias.** Antes de agregar una librería JavaScript nueva, justificarla en el reporte; preferir lo que ya está en la sección 5.

2. Contexto del proyecto (resumen)

- **Problema.** El monitoreo fitosanitario de la vid se hace con recorridos manuales (10 a 20 personas) y matrices de Excel por lote, hilera y planta, sin evidencia fotográfica ni ubicación vinculada a cada hallazgo.
- **Solución.** Un operador recorre cada hilera con **dos celulares de captura** (CAMERA_1 y CAMERA_2) montados en un soporte de hombros, ambos mirando el **mismo lateral**, y un **tercer celular** en la mano (CONTROLADOR). Una hilera se cubre con dos pasadas: **LATERAL_A** y luego **LATERAL_B**.
- **Red de campo.** Los tres celulares se comunican por **Wi-Fi local sin internet**: WebSocket para órdenes y estados; HTTP local para las fotos. Al volver, el controlador **sincroniza** con el backend.
- **Backend y web (fuera de esta app).** Spring Boot (JWT) guarda datos en PostgreSQL y fotos en Amazon S3, publica tareas en RabbitMQ, un servicio Python/FastAPI con YOLO sugiere indicios y Angular + Leaflet permite la revisión del especialista. La IA no diagnostica.
- **Piloto.** Lotes SWG 1, SWG 2 y SWG 5; 12 segmentos de 25 plantas (n = 12 pares pretest–postest). Postest previsto del 9 al 18 de noviembre de 2026 con tres celulares Android y una versión congelada. En la sustentación se expone la app en Android e iPhone; los resultados vienen del piloto.
- **Equipo móvil.** La app la construye el equipo del proyecto con apoyo de IA; el backend lo desarrolla otro integrante. Mientras el backend no esté listo, la app usa un **backend simulado** con el mismo contrato (sección 15.6).

3. Alcance de la app

Incluye:

- Una sola app con tres **funciones de dispositivo**: CONTROLADOR, CÁMARA 1 y CÁMARA 2, elegidas en cada celular después de iniciar sesión.
- Funcionamiento en **Android e iOS** para las tres funciones.
- **Registro** de usuarios con aprobación del administrador, **inicio de sesión obligatorio** en los tres celulares, inicio de sesión **sin internet** después de un primer ingreso con internet, **cambio y recuperación** de contraseña vía administrador.
- Descarga de catálogos (lotes, hileras, segmentos, marcadores) para trabajar sin internet.
- Sesión de monitoreo, pasadas por lateral, secuencias coordinadas, modos manual, automático y mixto.
- Captura, validación técnica de calidad, repetición por cámara, transferencia al controlador con confirmación.
- GPS del controlador por secuencia, marcadores y segmentos.
- Monitor de cámaras, incidencias, recuperación tras cierre o reinicio, registro de diagnóstico.
- Sincronización idempotente con Spring Boot al volver.
- Simulador de cámaras para desarrollar en Expo Go con un solo iPhone.

No incluye: diagnóstico de plaga en el celular, tratamientos, dosis, calendarios, niveles de urgencia, revisión del especialista, mapa (eso está en la web), notificaciones de WhatsApp, publicación en tiendas, captura en segundo plano o con la pantalla apagada, uso del GPS como posición exacta de una planta.

4. Decisiones vigentes

ID	Decisión	Estado	Fuente
D-01	Una sola app React Native + Expo + TypeScript con tres funciones de dispositivo.	Confirmado	Esp. v2.0 §9; Stack v2.1 §10
D-02	Las tres funciones funcionan en Android y en iOS.	Confirmado	Equipo, 01/10/2026
D-03	Expo SDK 54 fijo durante el curso (el Expo Go de la App Store solo llega al SDK 54).	Confirmado / Verificado	Equipo; documentación de Expo
D-04	Inicio de sesión obligatorio en los tres celulares.	Confirmado	Equipo, 01/10/2026 (cambia Esp. v2.0 §9 y Stack v2.1 §32.1)
D-05	Registro desde la app; la cuenta queda pendiente hasta que el ADMINISTRADOR la aprueba y asigna el rol en la web.	Confirmado	Equipo, 01/10/2026
D-06	Login sin internet con la misma contraseña si ese usuario ya entró con internet en ese celular; ventana de 7 días (calibrable).	Confirmado (ventana: Propuesto)	Equipo, 01/10/2026
D-07	Recuperación de contraseña vía administrador: contraseña temporal y cambio obligatorio al entrar.	Confirmado	Equipo, 01/10/2026
D-08	La función del dispositivo se elige después del login y queda guardada en ese celular.	Confirmado	Equipo, 01/10/2026
D-09	Control por WebSocket; fotos por HTTP local; las fotos nunca viajan por WebSocket.	Confirmado	Esp. v2.0 §15
D-10	Servidor del controlador sobre react-native-tcp-socket 6.4.x (MIT) con un servidor WebSocket propio (Anexo A).	PoC (candidata principal)	Esp. App Móvil v1.0 §3.3
D-11	Transferencia de fotos: opción A (la cámara sube con uploadAsync a un receptor HTTP propio del controlador) o opción B (servidor de archivos en la cámara). Se elige en la Fase 2.	PoC	Esp. App Móvil v1.0 §12.4 (corregido)
D-12	Emparejamiento por QR con IP, puertos, sesión y token aleatorio (vale hasta cerrar la sesión o regenerar el QR).	Confirmado / formato Propuesto	Esp. v2.0 §23.2
D-13	SQLite para datos y colas; archivos para fotos; SecureStore para tokens y verificador.	Confirmado	Esp. v2.0 §15.3
D-14	Calidad: primero exposición (copia reducida) y luego nitidez (varianza del Laplaciano en recortes a resolución original).	Propuesto	Sección 16
D-15	Fase 1 en iPhone con Expo Go; Fases 2 a 4 y 6 en Android con Development Build y APK; Fase 5 valida iOS con Xcode y Apple ID gratuito.	Confirmado	Equipo
D-16	Piloto con tres celulares Android y versión congelada.	Confirmado	Equipo
D-17	Navegación con Expo Router 6 y rutas protegidas (Stack.Protected).	Propuesto / Verificado	Sección 6
D-18	Estado de interfaz con Zustand 5; validación de mensajes y respuestas con zod 4.	Propuesto	Sección 5
D-19	expo-network para IP y estado de red (sustituye a NetInfo de la Esp. App Móvil v1.0).	Propuesto	Sección 5
D-20	El verificador del login sin internet usa PBKDF2-SHA256 de @noble/hashes 1.8.0.	Propuesto / Verificado	Sección 7.5
D-21	La configuración de captura y calidad viaja del controlador a las cámaras al emparejar (todas usan la misma versión).	Propuesto	Sección 14
D-22	Las subidas al backend usan uploadAsync (multipart nativo) de expo-file-system/legacy.	Propuesto / Verificado (API)	Sección 15
D-23	Retención: la cámara libera fotos con ACK de sesiones que el controlador informa sincronizadas; el controlador libera solo por acción manual y solo lo sincronizado.	Propuesto	RN-09

4.1 Cambios respecto de documentos anteriores

Documento	Decía	Ahora
Esp. v2.0 §9 y Stack v2.1 §32.1	Solo el operador autentica el controlador; las cámaras se asocian sin iniciar sesión.	Los tres celulares inician sesión (D-04).
Esp. v2.0 §18	Las cuentas se gestionan solo en la web.	Además existe el registro desde la app con aprobación (D-05).
Esp. App Móvil v1.0 §5.2	NetInfo para conectividad.	expo-network (D-19).
Esp. App Móvil v1.0 §12.4	La subida con expo/fetch no carga la foto en JavaScript.	Sí la carga; se usa uploadAsync nativo (D-11, D-22).
Esp. App Móvil v1.0 §13.1	Tablas sin tipos.	Esquema SQL completo (sección 12).

5. Stack, versiones y configuración del proyecto

5.1 Base (Verificado)

Elemento	Versión	Nota
Expo SDK	expo ~54.0.x (54.0.37 verificada)	Fijo (R-01).
React Native	0.81.5	Incluido en el SDK 54. Nueva Arquitectura activa por defecto (se puede desactivar en el SDK 54 si una librería lo exige).
React	19.1.0	Incluido en el SDK 54.
TypeScript	~5.9.2	De la plantilla blank-typescript@sdk-54.
Motor JS	Hermes	Por defecto en Expo. Expo instala TextDecoder, URL y structuredClone globales.
Node.js (desarrollo)	20.19.4 o superior	Exigido por React Native 0.81.
Xcode (solo Fase 5)	16.1 mínimo; 26 recomendado por Expo	Changelog del SDK 54.
Mínimos de sistema	Android 7 (API 24) · iOS 15.1	Mínimos del SDK 54.

5.2 Módulos y librerías

Necesidad	Paquete (versión SDK 54)	Uso	Expo Go	Estado
Navegación	expo-router ~6.0.24 (+ react-native-safe-area-context ~5.6.0, react-native-screens ~4.16.0, expo-linking ~8.0.12, expo-constants ~18.0.14, expo-status-bar ~3.0.9)	Rutas por archivo; Stack.Protected para rutas con sesión	Sí	Propuesto
Base de datos	expo-sqlite ~16.0.10	openDatabaseAsync, execAsync, runAsync, getFirstAsync, getAllAsync, withExclusiveTransactionAsync, SQLiteProvider	Sí	Confirmado
Archivos	expo-file-system ~19.0.24	API nueva: File, Directory, Paths (Paths.document, Paths.availableDiskSpace), file.md5, file.open() → FileHandle.writeBytes; uploadAsync desde expo-file-system/legacy	Sí	Confirmado
Credenciales	expo-secure-store ~15.0.8	Tokens y verificador (máx. 2048 bytes por valor)	Sí	Confirmado
Cámara y QR	expo-camera ~17.0.10	CameraView, takePictureAsync({ quality, exif, shutterSound }), onBarcodeScanned, useCameraPermissions	Sí	Confirmado

Necesidad	Paquete (versión SDK 54)	Uso	Expo Go	Estado
Copia reducida y recortes	expo-image-manipulator ~14.0.8	ImageManipulator.manipulate(uri) con .resize() o .crop() → renderAsync() → saveAsync({ format, compress })	Sí	Confirmado
Decodificar JPEG	jpeg-js 0.4.4 (BSD-3-Clause)	decode(bytes, { useTArray: true, formatAsRGBA: false })	Sí	Confirmado
Ubicación	expo-location ~19.0.8	requestForegroundPermissionsAsync, watchPositionAsync, Accuracy.Highest	Sí	Confirmado
IDs y hash	expo-crypto ~15.0.9	randomUUID(), getRandomBytes(), digestStringAsync(SHA1, ..., { encoding: BASE64 })	Sí	Confirmado
Login sin internet	@noble/hashes 1.8.0 (MIT)	pbkdf2Async + sha256 (import @noble/hashes/pbkdf2 y @noble/hashes/sha2)	Sí	Propuesto / Verificado
Datos del equipo	expo-device ~8.0.10, expo-application ~7.0.8	Modelo, sistema, versión y compilación de la app	Sí	Confirmado
Batería	expo-battery ~10.0.8	Nivel para monitor y avisos	Sí	Confirmado
Pantalla encendida	expo-keep-awake ~15.0.8	useKeepAwake() en pantallas de pasada	Sí	Confirmado
Compartir diagnóstico	expo-sharing ~14.0.8	shareAsync(uri) del archivo de diagnóstico	Sí	Propuesto
Red	expo-network ~8.0.8	getIpAddressAsync(), getNetworkStateAsync(), addNetworkStateListener()	Sí	Propuesto
Estado de interfaz	zustand 5.0.x (MIT)	Estado de sesión de usuario y estado en vivo del monitor	Sí	Propuesto
Validación	zod 4.x (MIT)	Esquemas de mensajes del protocolo y respuestas de la API	Sí	Propuesto
Servidor TCP	react-native-tcp-socket 6.4.3 (MIT; usa buffer 6.0.3)	Servidor WebSocket y receptor HTTP en el controlador	No (Development Build)	PoC
Opción B	@dr.pogodin/react-native-static-server 0.28.0 + @dr.pogodin/react-native-fs	Servidor de archivos en la cámara	No	PoC (alternativa)
Cliente de desarrollo	expo-dev-client ~6.0.21	Development Build (Fases 2 a 4)	—	Confirmado
Configuración nativa	expo-build-properties ~1.0.10	android.usesCleartextTraffic: true	—	Confirmado
Pruebas	jest-expo ~54.0.18, jest, @types/jest	Pruebas unitarias	—	Confirmado

NOTA

@noble/hashes 2.x es solo ESM y exige Node 20.19+; se fija la 1.8.0, que publica CommonJS y ESM con las rutas @noble/hashes/pbkdf2 y @noble/hashes/sha2.

5.3 Creación del proyecto (Tarea T-00)

```
# 1. Proyecto con la plantilla del SDK 54
npx create-expo-app@latest riachuelo-mobile --template blank-typescript@sdk-54
cd riachuelo-mobile
# 2. Navegación
npx expo install expo-router react-native-safe-area-context react-native-screens expo-linking expo-constants expo-status-bar
# 3. Módulos del dispositivo (todos funcionan en Expo Go SDK 54)
npx expo install expo-sqlite expo-file-system expo-secure-store expo-camera expo-image-manipulator expo-location expo-crypto expo-device expo-application expo-battery expo-keep-awake expo-network expo-sharing expo-build-properties
# 4. Librerías JavaScript puras (versiones fijadas)
npm install jpeg-js@0.4.4 @noble/hashes@1.8.0 zustand@5 zod@4
# 5. Pruebas, lint y formato
npx expo install jest-expo jest @types/jest -- --save-dev
npx expo lint
npx expo install prettier eslint-config-prettier eslint-plugin-prettier -- --save-dev
# 6. Solo desde la Fase 2 (Development Build; no funciona en Expo Go)
npx expo install expo-dev-client
npm install react-native-tcp-socket@6.4.3
```

Ajustes después de instalar:

- En package.json: "main": "expo-router/entry".
- Scripts: "typecheck": "tsc --noEmit", "lint": "expo lint", "test": "jest", "test:watch": "jest --watchAll".
- Configuración de Jest en package.json: "jest": { "preset": "jest-expo" }.
- En tsconfig.json: extender expo/tsconfig.base, "strict": true y el alias "@/*": [".src/*"].

5.4 app.json de referencia

```
{
  "expo": {
    "name": "Riachuelo Monitoreo",
    "slug": "riachuelo-mobile",
    "scheme": "riachuelo",
    "version": "0.1.0",
    "orientation": "portrait",
    "userInterfaceStyle": "light",
    "experiments": { "typedRoutes": true },
    "ios": {
      "bundleIdentifier": "pe.riachuelo.monitoreo",
      "buildNumber": "1",
      "infoPlist": {
        "NSLocalNetworkUsageDescription": "Se usa para coordinar las cámaras y el controlador en el Wi-Fi del campo.",
        "NSAppTransportSecurity": { "NSAllowsLocalNetworking": true }
      }
    },
    "android": {
      "package": "pe.riachuelo.monitoreo",
      "versionCode": 1
    },
    "plugins": [
      "expo-router",
      "expo-sqlite",
      "expo-secure-store",
      ["expo-camera", {
        "cameraPermission": "Se usa para fotografiar la vid y leer el QR de emparejamiento.",
        "microphonePermission": false,
        "recordAudioAndroid": false
      }],
      ["expo-location", {
        "locationWhenInUsePermission": "Se usa para registrar la ubicación de cada secuencia de fotos."
      }],
      ["expo-build-properties", { "android": { "usesCleartextTraffic": true } }]
    ]
  }
}
```

- usesCleartextTraffic permite HTTP sin cifrar en Android (red local y backend de desarrollo). Sin esto, la red local suele funcionar en desarrollo y fallar en el APK.
- NSLocalNetworkUsageDescription es obligatorio para que iOS muestre el permiso de red local. El efecto de NSAllowsLocalNetworking sobre conexiones a direcciones IP se comprueba en la Fase 5 (pregunta abierta Q-07).
- Los identificadores pe.riachuelo.monitoreo son ejemplos: el equipo define los definitivos antes de la Fase 2 y no se cambian después.

5.5 eas.json de referencia (Android)

```
{
  "cli": { "appVersionSource": "local" },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal",
      "android": { "buildType": "apk" }
    },
    "preview": {
      "distribution": "internal",
      "android": { "buildType": "apk" }
    }
  }
}
```

```
}
}
```

5.6 Variables de entorno

Variable	Ejemplo	Uso
EXPO_PUBLIC_API_URL	http://192.168.1.50:8080	URL base del backend (sin /api/v1).
EXPO_PUBLIC_USE MOCK_API	1	1 usa el backend simulado (sección 15.6).
EXPO_PUBLIC_APP_ENV	dev · piloto	Muestra el entorno en Ajustes; con piloto exige la misma versión de app en los tres celulares (RN-17).

Se leen solo en `src/config/env.ts`. Van en `.env` (fuera de Git) y `.env.example` (en Git).

5.7 Tipos de versión de la app

Tipo	Para qué	¿Necesita laptop?	Plataforma	Fases
Expo Go (SDK 54)	Pantallas, base de datos, cámara, calidad, GPS y simulador	Sí (Metro en la misma red)	iPhone (App Store) y Android (Expo Go del SDK 54 desde <code>expo.dev/go</code>)	F1
Development Build	Probar red local con código nativo	Sí (Metro)	Android (iOS solo con Mac)	F2 a F4
APK preview	Uso real sin laptop ni internet	No	Android	F6, piloto, sustentación
Release iOS con Xcode (Apple ID gratuito)	Validar las tres funciones en iPhone y sustentación	No	iPhone (vence a los 7 días; máximo 3 dispositivos)	F5, sustentación

5.8 Entorno de desarrollo

- Windows con Node 20.19.4+, Git, VS Code; Android Studio + JDK 17 para compilar Android localmente (alternativa a EAS Build, cuyo plan gratuito incluye hasta 15 compilaciones de Android).
- Un iPhone con Expo Go (SDK 54) para la Fase 1; dos o tres Android con depuración USB desde la Fase 2.
- Acceso a una Mac con Xcode para la Fase 5 y para instalar la versión de la sustentación (máximo 7 días antes).
- Router portátil a batería sin internet para reproducir el campo en el laboratorio.

6. Arquitectura y estructura del código

6.1 Capas y reglas de dependencia

Capa	Carpeta	Responsabilidad	Puede usar
Rutas y pantallas	<code>app/</code>	Mostrar estado, recoger acciones, navegar	Servicios, stores, <code>src/ui</code>
Servicios de función	<code>src/controller/</code> , <code>src/camera/</code>	Orquestar el recorrido (controlador) y la captura (cámara)	Dominio, almacenamiento, red local, dispositivo, sincronización
Dominio	<code>src/domain/</code> , <code>src/protocol/</code>	Tipos, reglas, estados, IDs y protocolo (TypeScript puro, sin React ni Expo)	Nada externo
Autenticación	<code>src/auth/</code>	Login con y sin internet, tokens, registro, contraseña	API, almacenamiento seguro, dominio
Dispositivo	<code>src/device/</code>	Cámara, calidad, GPS, batería, espacio, permisos, identidad del equipo	Módulos de Expo, dominio
Red local	<code>src/local-network/</code>	Servidor/cliente de control, transferencia de fotos, simulador	Interfaces de la sección 6.3, protocolo
Almacenamiento	<code>src/storage/</code>	SQLite (esquema, migraciones, repositorios), archivos, SecureStore	<code>expo-sqlite</code> , <code>expo-file-system</code> , <code>expo-secure-store</code>

Capa	Carpeta	Responsabilidad	Puede usar
Sincronización y API	src/sync/, src/api/	Cliente HTTP, backend simulado, cola hacia Spring Boot	Almacenamiento, auth, dominio
Diagnóstico	src/diagnostics/	event_log, incidencias, exportación	Almacenamiento
Interfaz compartida	src/ui/	Textos, mensajes, tema, componentes	React Native

Reglas: el dominio no importa nada de React, Expo ni de otras capas; las pantallas nunca abren la base de datos ni sockets; las librerías nativas solo se importan dentro de src/local-network/tcp y src/local-network/client, y se cargan con require() diferido (R-04).

6.2 Estructura de carpetas

```

riachuelo-mobile/
├── app/
│   ├── _layout.tsx           # Rutas (Expo Router)
│   ├── index.tsx            # SQLiteProvider + migraciones, tema, Stack.Protected
│   ├── (auth)/
│   │   ├── login.tsx        # PANT-02
│   │   ├── register.tsx     # PANT-03
│   │   ├── pending.tsx      # PANT-04
│   │   └── forgot-password.tsx # PANT-05
│   ├── change-password.tsx   # PANT-06
│   ├── (setup)/
│   │   ├── permissions.tsx   # PANT-07
│   │   └── role.tsx          # PANT-08
│   ├── settings.tsx         # PANT-09
│   ├── controller/
│   │   ├── index.tsx         # PANT-10 Panel del controlador
│   │   ├── catalogs.tsx     # PANT-11 Catálogos
│   │   ├── new-session.tsx   # PANT-12 Nueva sesión
│   │   ├── pairing.tsx       # PANT-13 Vincular cámaras (QR)
│   │   ├── short-test.tsx    # PANT-14 Prueba corta
│   │   ├── new-pass.tsx      # PANT-15 Nueva pasada
│   │   ├── pass.tsx          # PANT-16 Pasada activa (monitor)
│   │   ├── marker.tsx        # PANT-17 Cambiar marcador (modal)
│   │   ├── close-pass.tsx    # PANT-18 Cerrar pasada
│   │   ├── session-summary.tsx # PANT-19 Cerrar sesión de monitoreo
│   │   ├── sync.tsx          # PANT-20 Sincronización
│   │   └── incidents.tsx     # PANT-21 Incidencias
│   └── camera/
│       ├── index.tsx         # PANT-30 Escanear QR
│       ├── live.tsx          # PANT-31 Cámara en sesión
│       └── test.tsx          # PANT-32 Modo prueba de cámara
├── src/
│   ├── config/               defaults.ts · env.ts · index.ts
│   ├── domain/               types.ts · stateMachines.ts · sequence.ts · rules.ts · ids.ts · time.ts
│   ├── protocol/             messages.ts · schemas.ts (zod) · envelope.ts
│   ├── api/                  dto.ts · httpClient.ts · authApi.ts · bootstrapApi.ts · syncApi.ts · mock/
│   ├── auth/                 authService.ts · offlineAuth.ts · tokenStore.ts · validation.ts · authStore.ts
│   ├── device/               deviceIdentity.ts · permissions.ts · cameraService.ts · quality.ts · qualityPipeline.ts
│   │                       gpsService.ts · batteryService.ts · storageInfo.ts · keepAwake.ts
│   ├── local-network/        transport.ts · wsCodec.ts · httpUpload.ts · factory.ts
│   │   ├── tcp/              tcpControlServer.ts · tcpFileReceiver.ts # solo Development Build
│   │   ├── client/           wsControlClient.ts · uploadFileSender.ts
│   │   └── simulator/         simulatedNetwork.ts · virtualCamera.ts # Expo Go
│   ├── controller/           sessionService.ts · passService.ts · sequenceEngine.ts · pairingService.ts
│   │                       cameraMonitor.ts · consolidationService.ts · shortTest.ts
│   ├── camera/               cameraAgent.ts · captureService.ts · transferQueue.ts · retentionService.ts
│   ├── storage/              db.ts · migrations/001_initial.ts · repositories/ · files.ts · secureStore.ts
│   ├── sync/                 syncQueue.ts · syncService.ts · retry.ts
│   ├── diagnostics/          eventLog.ts · incidents.ts · exportDiagnostics.ts
│   └── ui/                   strings.ts · messages.ts · theme.ts · components/
├── _tests_/                  # Pruebas unitarias (espejo de src/)
├── assets/sample/            # Fotos de muestra para el simulador (útil, oscura, borrosa)
├── docs/                     MAESTRO_APP_MOVIL.md · adr/ · evidencias/
└── AGENTS.md · app.json · eas.json · package.json · tsconfig.json · eslint.config.js · .env.example

```

Notas:

- Las migraciones SQL se guardan como texto exportado desde un .ts (export const MIGRATION_001 = `...`), porque Metro no importa archivos .sql.

- Las fotos de muestra del simulador se cargan con `Asset.fromModule(require('...'))` de `expo-asset` (incluido con `expo`) y se copian al directorio de la cámara virtual.

6.3 Interfaces de transporte (contrato)

La app depende de estas interfaces. Hay dos implementaciones: la **real** (Development Build: `react-native-tcp-socket` en el controlador y el `WebSocket` nativo de React Native en las cámaras) y el **simulador** en memoria (`Expo Go`). `factory.ts` elige con `isRunningInExpoGo()`.

```
// src/local-network/transport.ts – Interfaces de transporte. La app depende de estas interfaces, nunca de la librería
// Implementaciones: (1) real con react-native-tcp-socket + WebSocket nativo (Development Build);
// (2) simulador en memoria para Expo Go (Fase 1). Se elige con isRunningInExpoGo() de 'expo'.
import type { CameraRole, UUID } from '../domain/types';
import type { Envelope, LocalCaptureMeta, LocalCaptureResponse, MessageType } from '../protocol/messages';
export type Unsubscribe = () => void;
export interface PeerInfo { deviceId: UUID; role: CameraRole | null; remoteAddress: string; }
/** Controlador: servidor de control (WebSocket). */
export interface ControlServer {
  start(port: number): Promise<{ host: string; port: number }>;
  stop(): Promise<void>;
  send<T extends MessageType>(deviceId: UUID, message: Envelope<T>): void;
  broadcast<T extends MessageType>(message: Envelope<T>): void;
  disconnect(deviceId: UUID, reason: string): void;
  onMessage(handler: (from: PeerInfo, message: Envelope) => void): Unsubscribe;
  onPeerClosed(handler: (peer: PeerInfo, reason: string) => void): Unsubscribe;
}
/** Cámara: cliente de control (WebSocket nativo de React Native). */
export interface ControlClient {
  connect(url: string): Promise<void>; // ws://<host>:<controlPort>/control
  send<T extends MessageType>(message: Envelope<T>): void;
  close(): void;
  readonly connected: boolean;
  onMessage(handler: (message: Envelope) => void): Unsubscribe;
  onStatus(handler: (status: 'CONECTANDO' | 'CONECTADO' | 'DESCONECTADO', detail?: string) => void): Unsubscribe;
}
/** Controlador: receptor de fotos (opción A: HTTP POST /local/captures). */
export interface FileReceiver {
  start(port: number): Promise<void>;
  stop(): Promise<void>;
  /** El manejador verifica md5/tamaño, guarda archivo + registro y decide la respuesta (idempotente). */
  onCapture(handler: (meta: LocalCaptureMeta, tempFileUri: string) => Promise<LocalCaptureResponse>): Unsubscribe;
}
/** Cámara: envío de una foto al controlador. */
export interface FileSender {
  send(baseUrl: string, meta: LocalCaptureMeta, fileUri: string, timeoutMs: number): Promise<LocalCaptureResponse>;
}
export interface LocalNetworkFactory {
  createControlServer(): ControlServer;
  createControlClient(): ControlClient;
  createFileReceiver(): FileReceiver;
  createFileSender(): FileSender;
  /** IP del celular en la red Wi-Fi (expo-network getIpAddressAsync). */
  getLocalIp(): Promise<string | null>;
}
```

Simulador (Expo Go, Fase 1). Implementa `ControlServer` y `FileReceiver` en memoria con dos cámaras virtuales que reciben los mismos sobres del protocolo, copian una foto de muestra al directorio de la cámara virtual, ejecutan la **calidad real** (sección 16), responden `CAPTURE_OK` o `QUALITY_ERROR` y "transfieren" copiando el archivo. Permite configurar demoras, una tasa de error de calidad, caídas de enlace y fallos de transferencia para probar recuperación. El simulador nunca se incluye en las pantallas de la cámara real.

6.4 Arranque de la app

1. `SQLiteProvider` abre `riachuelo.db` y ejecuta las migraciones pendientes en `onInit` (sección 12.2).
2. Se carga o crea el `deviceId` (`SecureStore` `device.id` + `app_meta.device_id`).
3. Se carga el estado de autenticación (7.11).
4. Se redirige: sin sesión → `PANT-02`; cambio de contraseña obligatorio → `PANT-06`; sin función → `PANT-08`; faltan permisos de la función → `PANT-07`; controlador → `PANT-10`; cámara → `PANT-31` si tiene un `camera_context` abierto, si no `PANT-30`.

- Según la función, se ejecuta la **recuperación** (RF-44, 8.10): sesiones abiertas, colas pendientes y reconciliación antes de aceptar acciones nuevas.

7. Usuarios, cuentas, inicio de sesión y funciones del dispositivo

7.1 Roles de usuario y acceso a la app

Rol (backend)	¿Usa la app?	Uso
OPERADOR_CAMPO	Sí	Opera los tres celulares en campo.
ADMINISTRADOR	Sí (soporte y pruebas)	Aprueba cuentas, asigna roles, restablece contraseñas y revoca dispositivos desde la web.
ESPECIALISTA_FITOSANITARIO	No	Revisión en la web.
SUPERVISOR	No	Consultas en la web.

La app permite entrar solo a cuentas ACTIVO con rol OPERADOR_CAMPO o ADMINISTRADOR (MOBILE_ALLOWED_ROLES). El backend aplica la misma regla (RN-02). Un mismo usuario puede tener sesión abierta en los tres celulares a la vez; el backend debe permitir una sesión por dispositivo.

7.2 Estados de cuenta

PENDIENTE_APROBACION (tras registrarse) → ACTIVO (el administrador aprueba y asigna rol) o RECHAZADO. Una cuenta ACTIVO puede pasar a BLOQUEADO y volver. Además, la cuenta puede tener mustChangePassword = true cuando el administrador asigna una contraseña temporal.

7.3 Registro (con internet)

- PANT-03 pide: nombre completo, correo, celular (opcional), código de trabajador (opcional), contraseña, confirmación y aceptación del aviso de privacidad.
- Validación local (validation.ts, Anexo C): nombre de 5+ caracteres, correo válido, celular de 9 a 15 dígitos si se escribe, contraseña de 8+ caracteres con letras y números, confirmación igual, aviso aceptado.
- POST /api/v1/auth/register → 201 { userId, status: "PENDIENTE_APROBACION" } → PANT-04 «Solicitud enviada».
- Errores: EMAIL_ALREADY_REGISTERED, VALIDATION_ERROR (con fieldErrors que se muestran en cada campo), sin internet (SIN_INTERNET).
- Texto del aviso de privacidad (Propuesto, a validar por el equipo): «Tus datos (nombre, correo y celular) se usan solo para identificarte en el sistema de monitoreo de Agrícola Riachuelo. Las cámaras fotografían la vegetación; evita fotografiar personas.»

7.4 Inicio de sesión con internet

- PANT-02: correo y contraseña. Un solo botón «Ingresar».
- POST /api/v1/auth/login con los datos del dispositivo (deviceId, plataforma, modelo, sistema, versión de la app).
- Respuesta 200: guardar accessToken, refreshToken y sus vencimientos en SecureStore; guardar el usuario en users_cache; lastOnlineAuthAt = ahora.
- Si mustChangePassword = true → PANT-06 (no se crea verificador sin internet).
- Si no: crear o reemplazar el **verificador sin internet** con la contraseña escrita (PBKDF2, sección 7.5) mostrando «Preparando acceso sin internet...».
- Ir a PANT-08 si el celular aún no tiene función y luego a PANT-07 (permisos de esa función); si ya la tiene, a su pantalla principal (pasando por PANT-07 solo si falta algún permiso).
- Errores mostrados con API_ERROR_MESSAGES (sección 19): INVALID_CREDENTIALS, ACCOUNT_PENDING, ACCOUNT_REJECTED, ACCOUNT_BLOCKED, ROLE_NOT_ALLOWED, DEVICE_REVOKED, TOO_MANY_ATTEMPTS.

8. Si no hay respuesta del servidor (sin red o tiempo agotado), se intenta automáticamente el inicio de sesión sin internet (7.5).
9. Renovación: antes de cada llamada, si el `accessToken` vence en menos de `auth.refreshMarginSeconds`, se llama `POST /api/v1/auth/refresh`. Si la renovación responde `REFRESH_INVALID`, `ACCOUNT_BLOCKED`, `DEVICE_REVOKED` o `ROLE_NOT_ALLOWED`, se cierra la sesión de usuario (7.8) sin borrar datos de campo.

7.5 Inicio de sesión sin internet

- **Condiciones (RN-03):** existe un verificador en ese celular; el correo es el del último usuario que entró con internet; la cuenta estaba ACTIVO con rol permitido y sin cambio de contraseña pendiente; no hay bloqueo temporal; no pasaron más de `auth.offlineLoginMaxDays` (7) días desde `lastOnlineAuthAt`.
- **Verificador:** PBKDF2-HMAC-SHA256(contraseña, sal aleatoria de 16 bytes, `auth.pbkdf2Iterations` iteraciones, 32 bytes). Se guarda en `SecureStore` como JSON (`auth.offlineVerifier`, menos de 300 bytes) con la versión, el usuario, el correo normalizado, la sal, las iteraciones y el hash. La contraseña nunca se guarda. Código probado en el Anexo C.
- **Rendimiento:** 40 000 iteraciones tardan unos 90 ms en Node; en Hermes será bastante más lento. Se mide en la Fase 1 en el iPhone y en el Android más lento del piloto y se ajusta para quedar entre 0,5 y 1,5 s (Calibrar). Como el verificador guarda sus iteraciones, cambiar el valor no invalida verificadores existentes.
- **Resultado correcto:** estado AUTENTICADO con `mode = 'OFFLINE'`; banner «Sin internet — acceso válido hasta <fecha>». Se reinician los intentos fallidos.
- **Resultado incorrecto:** mismo mensaje que con internet («Correo o contraseña incorrectos»); tras `auth.offlineMaxFailedAttempts` (5) fallos, bloqueo de `auth.offlineLockoutMinutes` (15) minutos (RN-20).
- **Sin internet no se puede:** registrarse, cambiar o recuperar la contraseña, descargar catálogos ni sincronizar.
- **Al volver internet:** una sesión OFFLINE no tiene tokens (se borraron al cerrar sesión), así que no puede renovarse sola. Cuando expo-network informa conexión y `GET /health` responde, la app muestra el aviso no bloqueante `REAUTENTICACION_REQUERIDA` con un botón que abre una ventana de contraseña para el mismo usuario. Esa validación es un login con internet normal (7.4): guarda tokens, reemplaza el verificador, actualiza `lastOnlineAuthAt` y pasa el modo a ONLINE sin salir de la pantalla. Con `INVALID_CREDENTIALS` se muestra el error y la sesión OFFLINE sigue; con `ACCOUNT_BLOCKED`, `ROLE_NOT_ALLOWED` o `DEVICE_REVOKED` se aplica la revocación (7.8). El aviso no aparece durante una pasada ACTIVE.
- **Requisito para el backend:** el `refreshToken` debe durar al menos `offlineLoginMaxDays` (sugerido: 14 días con rotación).

7.6 Contraseña temporal y cambio de contraseña (con internet)

- Si el login devuelve `mustChangePassword = true`, la app solo permite PANT-06 o cerrar sesión (RN-04).
- PANT-06 pide la contraseña actual, la nueva y su confirmación → `POST /api/v1/auth/change-password` → 204. Luego se crea el verificador sin internet con la contraseña nueva y se continúa.
- También se puede cambiar desde Ajustes (PANT-09) cuando hay internet.

7.7 Recuperación vía administrador

1. PANT-05 pide el correo → `POST /api/v1/auth/password-reset-requests` → siempre 202 (no revela si el correo existe).
2. Mensaje: «Si el correo está registrado, el administrador recibirá tu solicitud y te dará una contraseña temporal».
3. El administrador asigna una contraseña temporal en la web; el backend marca `mustChangePassword = true` y revoca los `refreshToken` de ese usuario.
4. El usuario entra con internet usando la temporal y la app lo lleva a PANT-06.

7.8 Cierre de sesión de usuario y revocación

- **Cerrar sesión** (PANT-09): no se permite con una sesión de monitoreo abierta (7.12; RN-19). Con internet se llama `POST /api/v1/auth/logout`. Siempre se borran los tokens del celular. El verificador sin internet se conserva (solo sirve a ese mismo usuario).
- **Revocación** (respuesta `REFRESH_INVALID`, `ACCOUNT_BLOCKED`, `DEVICE_REVOKED` o `ROLE_NOT_ALLOWED`): se borran tokens y verificador, y se pide iniciar sesión con internet. Los datos de campo y las colas pendientes se conservan. La revocación no depende de RN-19: se aplica aunque haya una sesión de monitoreo abierta, que queda guardada tal como estaba y se recupera (8.10) cuando alguien vuelve a iniciar sesión en ese celular.

7.9 Función del dispositivo

- PANT-08 aparece después del primer login: **CONTROLADOR, CÁMARA 1 o CÁMARA 2**, con una descripción breve de cada una. Al confirmar se pasa a PANT-07 para pedir los permisos de esa función.
- Se guarda en `app_meta.device_role` y se muestra siempre en la cabecera junto al usuario y el modo (con o sin internet).
- Se cambia desde Ajustes solo si no hay una sesión de monitoreo abierta (7.12) ni transferencias o sincronizaciones pendientes en ese celular (RN-15); después del cambio se vuelve a PANT-07.
- La función es del celular, no del usuario: el mismo usuario puede tener un celular como controlador y otros dos como cámaras.

7.10 Qué hace cada función

Capacidad	Controlador	Cámara 1 / Cámara 2
Iniciar sesión (con o sin internet)	Sí	Sí
Descargar catálogos	Sí	No (recibe el contexto del controlador)
Crear sesión, pasadas y secuencias	Sí	No (RN-08)
Mostrar QR / escanear QR	Muestra	Escanea
GPS por secuencia	Sí	No
Tomar fotos y evaluar calidad	No	Sí
Recibir y consolidar fotos	Sí	No
Sincronizar con el backend	Sí	No
Monitor de cámaras e incidencias	Sí	Estado propio
Modo prueba de cámara	No	Sí
Simulador de cámaras (Expo Go)	Sí	No

7.11 Sesión de usuario al reabrir la app

El estado se guarda en `SecureStore` (`auth.userId`, `auth.mode` y los tokens, sección 12.3) para no pedir login en cada reinicio (RF-05):

Situación al abrir	Resultado
<code>auth.mode = ONLINE</code> y <code>auth.refreshTokenExpiresAt</code> en el futuro	AUTENTICADO en modo ONLINE (o CAMBIO_CONTRASEÑA_REQUERIDO si <code>users_cache.must_change_password = 1</code>). La renovación ocurre en la siguiente llamada con internet (7.4).
<code>auth.mode = OFFLINE</code> , o ONLINE con el refresh vencido (se borran los tokens), y se cumplen las condiciones de RN-03 salvo la contraseña	AUTENTICADO en modo OFFLINE, con el banner de 7.5.
Lo anterior sin cumplir RN-03 (ventana vencida, cuenta no activa, rol no permitido), o sin <code>auth.userId</code>	<code>SIN_SESION</code> → PANT-02. Los datos de campo se conservan.

Si la ventana sin internet vence con la app abierta, la sesión actual continúa hasta que se cierre la sesión de usuario o se reinicie la app.

7.12 «Sesión de monitoreo abierta» en cada función

RN-15 y RN-19 dependen de si el celular tiene una sesión de monitoreo abierta:

- **Controlador:** una sesión en PREPARING, READY, ACTIVE, PAUSED o CLOSING. Un borrador DRAFT no cuenta: todavía no tiene cámaras ni datos y se puede descartar (se elimina) desde PANT-10 o PANT-12.
- **Cámara:** un camera_context con closed_at vacío. Se cierra según 8.12.

Las fotos pendientes de transferir o de sincronizar no son una sesión abierta, pero también bloquean el cambio de función (RN-15).

8. Flujos de la app

8.1 Primer uso de un celular

1. Instalar la app y abrirla → PANT-02.
2. Si no tiene cuenta: «Crear cuenta» → PANT-03 → PANT-04 (espera la aprobación del administrador).
3. Con la cuenta aprobada: iniciar sesión con internet → PANT-08 (función) → PANT-07 (permisos de esa función).
4. Si es controlador: descargar catálogos (PANT-11).

8.2 Preparación con internet (antes de salir)

1. Iniciar sesión con internet en los **tres** celulares (deja listo el acceso sin internet por 7 días).
2. En el controlador, actualizar catálogos (PANT-11) y comprobar la fecha.
3. Revisar en cada celular: misma versión de la app, batería al 100 %, espacio libre (PANT-10 y PANT-31 lo muestran).

8.3 Emparejamiento y prueba corta (en campo, sin internet)

1. Encender el router portátil; poner las cámaras en modo avión con el Wi-Fi activo.
2. Controlador: PANT-12 → crear la sesión (modo e intervalo; queda DRAFT) → PANT-13 genera el token, inicia el servidor de control y el receptor de fotos y muestra el QR → sesión PREPARING.
3. Cada cámara: PANT-30 → escanear el QR → PAIR_REQUEST con el rol de su función → PAIRED → PANT-31 (14.6).
4. Con ambas cámaras CONECTADA, el operador pasa a PANT-14 y ejecuta la prueba corta: un CAPTURE_COMMAND con purpose = PRUEBA_CORTA a cada cámara, que captura, evalúa, transfiere y recibe el ACK (14.12).
5. Prueba aprobada en ambas → se guarda short_test_passed_at y la sesión pasa a READY → PANT-15 para crear la pasada.

8.4 Pasada y bucle de captura

1. PANT-15: hilera, lateral (A primero), dirección, segmento y marcador de inicio → START_PASS (ACK de ambas cámaras) + SESSION_CONTEXT → pasada ACTIVE, sesión ACTIVE.
2. Disparo según el modo: manual (botón grande «Capturar»), automático (temporizador del controlador) o mixto.
3. Por cada disparo el controlador crea la secuencia (número correlativo, un captureId por cámara, contexto y GPS) y envía CAPTURE_COMMAND a ambas cámaras (sección 14.7).
4. Cada cámara captura, guarda, evalúa calidad y responde CAPTURE_OK o QUALITY_ERROR; luego transfiere la foto (sección 14.8).

5. El controlador actualiza el resultado de cada cámara y el estado de la secuencia (deriveSequenceStatus).
6. No se emite una nueva orden (secuencia, repetición o prueba corta) mientras la anterior no tenga respuesta de las cámaras a las que se envió o no venza su espera (RN-11). Las transferencias pueden seguir en paralelo.

8.5 Repetición por calidad

1. La cámara responde QUALITY_ERROR (motivo y métricas) o no responde a tiempo (SIN_RESPUESTA).
2. PANT-16 muestra «Repetir CÁMARA X (secuencia N)» para la secuencia más reciente de la pasada con un resultado RECHAZADA_CALIDAD, ERROR_CAMARA o SIN_RESPUESTA en esa cámara. El botón está activo solo con la pasada ACTIVE, esa cámara CONECTADA y ninguna orden en curso (RN-11). En modo automático, la repetición se envía en lugar de la siguiente secuencia y el temporizador se reinicia al recibir su respuesta.
3. Al repetir: nuevo captureId para esa cámara con replacesCaptureId (applyRetake, sección 11.3). Antes de enviar la orden, el controlador guarda en retake_requests el contexto vigente en ese momento (segmento, marcador y GPS). La secuencia conserva su contexto original y la foto repetida se sincroniza con retakeContext (RN-07). La foto válida de la otra cámara se conserva.
4. Si no repite: la secuencia queda INCOMPLETE y visible en el resumen.

8.6 Cambio de marcador o segmento

PANT-17 → el controlador inserta una fila en marker_changes (marcador, segmento, hora y GPS), actualiza currentMarkerId/currentSegmentId de la pasada y reenvía SESSION_CONTEXT. Las secuencias ya creadas conservan su contexto. Los cambios viajan con la pasada al sincronizar (markerChanges).

8.7 Cambio de lateral, repetición de lateral y cierre de hilera

1. «Terminar pasada» → PANT-18 muestra secuencias completas e incompletas, transferencias pendientes e incidencias → el operador marca COMPLETED o INCOMPLETE (incidencia obligatoria) → END_PASS.
2. Crear la pasada LATERAL_B de la misma hilera en la misma sesión (RN-12) y repetir 8.4.
3. **Repetir lateral:** si un lateral debe hacerse de nuevo (pasada INCOMPLETE, cámara reemplazada, error del operador), PANT-15 o PANT-18 ofrecen «Repetir lateral»: nueva pasada del mismo lateral con passOrder = máximo de esa hilera en la sesión + 1 y una incidencia OPERADOR obligatoria con el motivo. Puede empezar en el marcador donde quedó la anterior.
4. Con pasadas cerradas en ambos laterales, la hilera queda cubierta (RN-13); se puede continuar con otra hilera en la misma sesión. Nunca hay dos pasadas abiertas a la vez.

8.8 Cierre de la sesión de monitoreo

1. PANT-19 → sesión CLOSING: no se emiten más órdenes; si hay una pasada abierta, se cierra INCOMPLETE con incidencia.
2. Si faltan fotos por llegar (capturas con fila en captures que aún no están RECIBIDA_CONTROLADOR; ver 14.7, paso 7), PANT-19 muestra cuántas por cámara y ofrece «Esperar» o «Cerrar igual» (mensaje CIERRE_CON_PENDIENTES). «Cerrar igual» registra una incidencia TRANSFERENCIA con el conteo.
3. Sesión CLOSED → se crean los elementos de sync_queue (15.5) → SESSION_CLOSED a las cámaras conectadas → los servidores se detienen al recibir sus ACK o agotar los reenvíos (14.3) → se borran los archivos de la prueba corta (RN-21).
4. Las fotos que falten llegan después según 8.13.

8.9 Sincronización al volver

PANT-20 (controlador, con internet y sesión de usuario ONLINE): comprueba GET /health, renueva el token si hace falta y envía en orden sesión, pasadas, secuencias, capturas, incidencias y cierre (sección 15.5). Una sesión queda SYNCED cuando todos sus elementos están confirmados; si después llega una foto tardía de esa sesión, vuelve a CLOSED hasta sincronizarla (8.13). En el siguiente emparejamiento, el controlador informa a las

cámaras qué sesiones ya están sincronizadas para que liberen espacio (RN-09).

8.10 Recuperación tras cierre o reinicio

- **Controlador:** al abrir, si hay una sesión abierta (7.12), la restaura desde SQLite y pasa todas sus cámaras a DESCONECTADA hasta su PAIR_REQUEST. Si tenía una pasada ACTIVE, la pasada y la sesión quedan PAUSED, y los resultados PENDIENTE de la última secuencia pasan a SIN_RESPUESTA. Reinicia el servidor de control y el receptor con el mismo token (pairing.token) y los mismos puertos. Si la IP del celular cambió, PANT-13 (accesible desde PANT-16 con la pasada en pausa) muestra el QR con la IP nueva y las cámaras deben reescanearlo. Las cámaras que vuelven envían PAIR_REQUEST; tras PAIRED, el controlador hace el RESYNC (14.9) y recién entonces se puede reanudar.
- **Cámara:** al abrir, si tiene un camera_context abierto, reconecta con controller_host, control_port y pairing.token, con las esperas de protocol.reconnectDelaysMs. Mientras no conecta, PANT-31 muestra «Sin conexión con el controlador» y ofrece «Reescanear QR». Las capturas que quedaron en CAPTURED (la app se cerró durante la calidad) se evalúan de nuevo y se encolan. Las transferencias pendientes se reanudan solas al conectar.

8.11 Mapa de navegación

```

app/index
├── (auth)/login ─┬── (auth)/register → (auth)/pending
                  ├── (auth)/forgot-password
                  └── change-password (si mustChangePassword)
├── (setup)/role → (setup)/permissions
├── controller/index ─┬── catalogs
                     ├── new-session → pairing → short-test → new-pass → pass ─┬── marker
                                         └── close-pass
                     ├── close-pass → new-pass | pairing | session-summary
                     ├── pairing (también desde new-pass, close-pass y pass en pausa)
                     ├── sync
                     └── incidents
├── camera/index (QR) ≠ camera/live («Reescanear QR») ; camera/test
└── settings (desde la cabecera en cualquier pantalla con sesión)

```

Guardas con Stack.Protected: (auth)/* solo sin sesión de usuario; change-password con sesión (es la única ruta disponible en CAMBIO_CONTRASENA_REQUERIDO); (setup)/*, settings, controller/* y camera/* solo en estado AUTENTICADO; controller/* exige además la función CONTROLADOR y camera/* una función de cámara.

8.12 Cierre del contexto en la cámara

El camera_context de una cámara se cierra (closed_at) cuando:

1. recibe SESSION_CLOSED;
2. recibe PAIR_REJECTED con motivo SESSION_CLOSED;
3. su sesión aparece en PAIRED.syncedSessionIds;
4. el operador escanea el QR de **otra** sesión (PANT-30 pide confirmación); o
5. el operador elige «Abandonar sesión» en PANT-31, solo sin conexión con el controlador (cliente CONECTANDO o DESCONECTADO) y con confirmación (queda en event_log).

Al cerrarse el contexto, la cámara borra pairing.token (si todavía es el de ese contexto), sus fotos de prueba corta y sus entradas en transfer_queue. Sus fotos de evidencia pendientes de transferir se conservan y siguen 8.13.

8.13 Fotos que llegan después del cierre

- La cámara guarda cada foto con la sesión a la que pertenece y solo la envía al controlador de esa sesión (camera_context.controller_device_id igual al controllerDeviceId de PAIRED). Si está vinculada a otro controlador, la foto espera y PANT-31 muestra FOTOS_DE_OTRO_CONTROLADOR (RN-22).

- El receptor del controlador acepta fotos de sesiones CLOSED o SYNCED si el deviceId estuvo registrado en esa sesión, aunque su rol se haya liberado.
- Al recibir una foto tardía de evidencia: se actualiza el resultado de su cámara en la secuencia (si el captureId sigue vigente), se recalculan los contadores de la pasada, se agrega la captura a sync_queue y la pasada y sus secuencias vuelven a remote_sync_status = PENDIENTE_NUBE con su lote pendiente (15.5). Si la sesión estaba SYNCED, vuelve a CLOSED hasta sincronizarlos; su propio remote_sync_status no cambia.
- Si llega una foto de prueba corta de una sesión cerrada, se responde RECEIVED y se descarta.

9. Catálogo de pantallas

Formato: **ID** — **Nombre (ruta)** · Para quién · Contenido · Acciones · Validaciones y estados · Requerimientos.

PANT-01 — Arranque (app/index.tsx) · Todos · Logo y «Preparando...» mientras migra la base de datos y carga la sesión · Ninguna · Si la migración falla, mensaje y botón «Exportar diagnóstico» · RF-11, RF-44.

PANT-02 — Iniciar sesión ((auth)/login) · Todos · Correo, contraseña (con mostrar/ocultar), indicador «Con internet / Sin internet», versión de la app · «Ingresar», «Crear cuenta», «¿Olvidaste tu contraseña?» · Campos obligatorios; mensajes de 7.4 y 7.5; bloqueo temporal con hora de fin · RF-02 a RF-05, RF-10.

PANT-03 — Crear cuenta ((auth)/register) · Personas nuevas · Nombre completo, correo, celular (opcional), código de trabajador (opcional), contraseña, confirmación, casilla del aviso de privacidad · «Enviar solicitud» · Validación de 7.3; deshabilitado sin internet · RF-01.

PANT-04 — Solicitud enviada ((auth)/pending) · Personas nuevas · «Tu cuenta quedó pendiente de aprobación» y qué hacer · «Volver a iniciar sesión» · — · RF-01.

PANT-05 — Recuperar contraseña ((auth)/forgot-password) · Todos · Correo · «Enviar solicitud» · Mensaje neutro; requiere internet · RF-07.

PANT-06 — Cambiar contraseña (change-password) · Usuario con sesión · Contraseña actual, nueva, confirmación; aviso si es obligatorio · «Guardar», «Cerrar sesión» · Política de contraseña; requiere internet · RF-06, RF-08.

PANT-07 — Permisos ((setup)/permissions) · Todos · Se muestra después de PANT-08 y otra vez tras cambiar la función. Lista según la función, con el estado de cada permiso: cámara (solo funciones de cámara: fotos y lectura del QR), ubicación (controlador), red local (iPhone, todas las funciones; iOS no informa su estado, así que se explica cómo revisarlo en Ajustes) · «Conceder», «Abrir Ajustes», «Continuar» · No continúa sin los permisos obligatorios de la función · RF-14.

PANT-08 — Función del dispositivo ((setup)/role) · Todos · Tres tarjetas grandes: Controlador, Cámara 1, Cámara 2, con descripción; datos del equipo · «Usar como ...» · Confirmación; RN-15 al cambiar; luego PANT-07 · RF-12, RF-13.

PANT-09 — Ajustes (settings) · Todos · Usuario, modo de acceso y fecha límite sin internet, función, versión de app/protocolo/perfil de calidad/configuración, entorno, espacio libre, batería · «Cambiar contraseña», «Cambiar función», «Reintentar transferencias con error» (cámaras), «Exportar diagnóstico», «Liberar espacio», «Cerrar sesión» · RN-15, RN-19; liberar espacio solo según RN-09 · RF-08, RF-09, RF-13, RF-35, RF-45.

PANT-10 — Panel del controlador (controller/index) · Controlador · Estado de preparación (catálogos y fecha, batería, espacio, GPS), sesión abierta si existe, sesiones pendientes de sincronizar · «Nueva sesión», «Continuar sesión» (abre PANT-13 si la sesión está DRAFT o PREPARING, PANT-15 si está READY, PANT-16 si hay una pasada abierta y PANT-19 si está CLOSING), «Descartar borrador», «Catálogos», «Sincronizar», «Incidencias» · Avisos de catálogos antiguos, batería y espacio · RF-15, RF-42, RF-46.

PANT-11 — Catálogos (controller/catalogs) · Controlador · Versión y fecha de catálogos, conteos de lotes, hileras, segmentos y marcadores · «Actualizar» · Requiere internet; reemplazo atómico de catálogos · RF-15.

PANT-12 — Nueva sesión (controller/new-session) · Controlador · Modo (manual, automático, mixto) e intervalo (si aplica, dentro de los límites de configuración) · «Crear sesión» · Requiere catálogos; batería y espacio mínimos (RN-10) · RF-17.

PANT-13 — Vincular cámaras (controller/pairing) · Controlador · QR grande (sección 14.2), IP y puertos (con ingreso manual de la IP si no se detecta, Q-04), tarjeta por cámara (estado, usuario, modelo, versión, batería, espacio) · «Regenerar QR», «Liberar CÁMARA X», «Continuar» (a PANT-14 si falta la prueba corta; si no, vuelve a la pantalla de origen) · Se abre desde PANT-12, PANT-10, PANT-15, PANT-18 y PANT-16 (pasada en pausa). Continúa solo con ambas cámaras CONECTADA. «Liberar CÁMARA X» solo con esa cámara en PERDIDA o DESCONECTADA y sin pasada abierta (si no, LIBERAR_CAMARA_NO_PERMITIDO); pide confirmación, deja el rol libre para otro celular (RN-16) y exige repetir la prueba corta (sesión READY → PREPARING) · RF-18.

PANT-14 — Prueba corta (controller/short-test) · Controlador · Resultado por cámara: respuesta (CAPTURE_OK o QUALITY_ERROR), calidad con métricas, transferencia, ACK, tiempos y desfase entre cámaras · «Ejecutar prueba», «Repetir», «Continuar» · Aprobación según 14.12; una foto de mala calidad aprueba la comunicación pero muestra PRUEBA_CORTA_CALIDAD; continúa solo con ambas aprobadas · RF-19.

PANT-15 — Nueva pasada (controller/new-pass) · Controlador · Lote, hilera, lateral (A o B), dirección, segmento y marcador de inicio · «Iniciar pasada», «Repetir lateral» (incidencia obligatoria), «Vincular cámaras» (PANT-13) · RN-10 y RN-12 · RF-16, RF-20.

PANT-16 — Pasada activa (controller/pass) · Controlador · Cabecera: hilera, lateral, segmento, marcador, modo; GPS y precisión; contador de secuencias (completas, parciales, incompletas); dos tarjetas de cámara (enlace, batería, espacio, pendientes, último resultado, desfase); avisos · Botón grande «Capturar» (manual/mixto), «Pausar/Reanudar», «Repetir CÁMARA X (secuencia N)» (8.5), «Cambiar marcador», «Vincular cámaras» (solo en pausa, para mostrar el QR), «Terminar pasada» · useKeepAwake; pausa automática por enlace perdido, batería o espacio; nunca dos órdenes superpuestas · RF-21 a RF-25, RF-38, RF-42.

PANT-17 — Cambiar marcador (controller/marker, modal) · Controlador · Lista de marcadores y segmentos de la hilera · «Confirmar» · — · RF-24.

PANT-18 — Cerrar pasada (controller/close-pass) · Controlador · Resumen de la pasada; selector COMPLETED/INCOMPLETE; incidencia obligatoria si es INCOMPLETE · «Cerrar pasada», luego «Crear LATERAL_B», «Repetir lateral», «Otra hilera», «Vincular cámaras» o «Cerrar sesión» · RN-12, RN-13 · RF-26, RF-47.

PANT-19 — Cerrar sesión de monitoreo (controller/session-summary) · Controlador · Resumen por pasada, fotos que faltan llegar por cámara, incidencias · «Cerrar sesión», y con fotos pendientes «Esperar» o «Cerrar igual» · Confirmación; «Cerrar igual» registra la incidencia (8.8) · RF-27.

PANT-20 — Sincronización (controller/sync) · Controlador · Por sesión: total, enviados, pendientes, errores; progreso; último intento · «Sincronizar ahora», «Reintentar errores» · Requiere internet y sesión de usuario ONLINE · RF-39 a RF-41.

PANT-21 — Incidencias (controller/incidents) · Controlador · Lista por sesión con tipo, severidad, hora · «Agregar incidencia» · — · RF-43.

PANT-30 — Escanear QR (camera/index) · Cámaras · Vista de cámara con lector de QR, función de la cámara, estado de red · Escanear; «Reintentar» · QR válido (type = RIACHUELO_PAIR), mensajes PAIR_REJECT_MESSAGES; si ya tiene un contexto abierto de otra sesión, pide confirmación antes de cerrarlo (8.12) · RF-28.

PANT-31 — Cámara en sesión (camera/live) · Cámaras · CameraView siempre montada a pantalla completa; franja de estado grande con colores y texto: sesión, lateral, conexión, pendientes, última captura (calidad y hora), batería, espacio · Ninguna durante la pasada (todo lo ordena el controlador); sin pasada activa, «Reintentar transferencias con error»; sin conexión con el controlador, «Reescanear QR» y «Abandonar sesión» (8.12); «Salir» solo sin sesión abierta · useKeepAwake; sin enlace: «Sin conexión con el controlador — en pausa»; aviso de fotos de otra sesión (8.13) · RF-29 a RF-31, RF-35, RF-36.

PANT-32 — Modo prueba de cámara (camera/test) · Cámaras · Vista de cámara, botón de captura, resultado de calidad con métricas y tiempo · «Tomar foto de prueba» · No crea datos de sesión; las fotos de prueba se guardan aparte y se pueden borrar · RF-32, RF-33.

10. Requerimientos

10.1 Requerimientos funcionales

Función: **T** = todos, **C** = controlador, **K** = cámaras. Prioridad: **P0** crítica para la prueba técnica y el piloto, **P1** alta, **P2** media. Fase: primera fase en que debe funcionar.

ID	Requerimiento	Func.	Prio.	Fase	Criterio de aceptación	Fuente
RF-01	Registrar una cuenta desde la app (nombre, correo, celular y código opcionales, contraseña, aviso de privacidad); queda PENDIENTE_APROBACION.	T	P0	F1	Con datos válidos e internet aparece PANT-04 y la API responde 201; los datos inválidos se marcan por campo; sin internet el botón está deshabilitado.	D-05
RF-02	Iniciar sesión con correo y contraseña contra el backend, enviando los datos del dispositivo, y guardar tokens en SecureStore.	T	P0	F1	Con credenciales correctas se llega a PANT-08 o a la pantalla de la función; los tokens están en SecureStore y nunca en logs.	D-04; Esp. v2.0 §10.1
RF-03	Mostrar el motivo cuando no se puede entrar.	T	P0	F1	Cada código de 7.4 y 7.5 muestra el texto de la sección 19.	D-04
RF-04	Iniciar sesión sin internet según RN-03, mostrando el modo y la fecha límite.	T	P0	F1	En modo avión, el último usuario que entró con internet hace ≤ 7 días entra con su contraseña; otro usuario o una contraseña incorrecta no entra.	D-06
RF-05	Mantener la sesión entre reinicios, renovarla con refresh y cerrarla si el backend la revoca.	T	P0	F1	Reiniciar no pide login; con token vencido y refresh válido la llamada se repite; con REFRESH_INVALID se vuelve a PANT-02 sin perder datos de campo.	Stack v2.1 §28
RF-06	Exigir el cambio de contraseña cuando mustChangePassword = true.	T	P0	F1	Con la contraseña temporal solo se accede a PANT-06 o a cerrar sesión; tras cambiarla se crea el verificador.	D-07
RF-07	Solicitar el restablecimiento de la contraseña.	T	P1	F1	Con cualquier correo se muestra el mensaje neutro; una llamada por envío.	D-07
RF-08	Cambiar la contraseña desde Ajustes.	T	P1	F1	Con la contraseña actual correcta e internet, se cambia y se reemplaza el verificador.	D-07
RF-09	Cerrar la sesión de usuario respetando RN-19.	T	P0	F1	Con sesión de monitoreo abierta se bloquea con mensaje; sin ella se borran los tokens y se vuelve a PANT-02.	D-04
RF-10	Bloqueo temporal tras intentos fallidos sin internet (RN-20).	T	P1	F1	Tras 5 fallos, 15 minutos de bloqueo con la hora de fin visible.	Propuesto
RF-11	Generar y conservar un deviceId por instalación y registrar plataforma, modelo, sistema y versión de la app.	T	P0	F1	El deviceId no cambia tras reinicios y viaja en el login y en el protocolo.	Esp. v2.0 §21
RF-12	Elegir la función del dispositivo después del login y mostrarla siempre.	T	P0	F1	La función persiste tras reinicios y aparece en la cabecera.	D-08
RF-13	Cambiar la función desde Ajustes respetando RN-15.	T	P1	F1	Con sesión abierta o pendientes se bloquea con mensaje.	D-08
RF-14	Verificar y pedir los permisos de la función con pantalla guía.	T	P0	F1	Sin permiso de cámara (cámaras) o de ubicación (controlador) no se continúa; en iPhone se explica cómo activar Red local.	Esp. App Móvil v1.0
RF-15	Descargar catálogos y parámetros con internet, guardarlos en SQLite y avisar si son antiguos.	C	P0	F1	PANT-11 muestra conteos y fecha; el reemplazo es atómico; con más de 72 h aparece el aviso.	Esp. v2.0 §10.1
RF-16	Seleccionar lote, hilera, segmento y marcador desde los catálogos locales sin internet.	C	P0	F1	En modo avión se eligen todos los valores.	Esp. v2.0 §13
RF-17	Crear una sesión de monitoreo con modo e intervalo.	C	P0	F1	La sesión queda DRAFT con versión de app, configuración y perfil de calidad.	Esp. v2.0 §10.1
RF-18	Mostrar el QR de emparejamiento y el estado de cada cámara; regenerar el QR.	C	P0	F1 (simulador) / F2	Con el QR vigente las cámaras se vinculan; tras regenerarlo, el anterior se rechaza con INVALID_TOKEN.	Esp. v2.0 §23.2
RF-19	Ejecutar y registrar la prueba corta en ambas cámaras.	C	P0	F1 (simulador) / F2	La sesión pasa a READY solo con ambas pruebas aprobadas según 14.12; el resultado queda en event_log (SHORT_TEST) y en short_test_passed_at.	Esp. v2.0 §10.1

ID	Requerimiento	Func.	Prio.	Fase	Criterio de aceptación	Fuente
RF-20	Crear pasadas por lateral con hilera, dirección y marcador de inicio, cumpliendo RN-10 y RN-12.	C	P0	F1	No se inicia sin ambas cámaras conectadas y prueba aprobada; LATERAL_B exige LATERAL_A cerrado en la misma hilera y sesión; «Repetir lateral» exige incidencia.	Esp. v2.0 §10.3
RF-21	Generar secuencias con número correlativo, un captureId por cámara, contexto vigente y GPS.	C	P0	F1	Cada secuencia tiene un sequenceId y dos captureId distintos; los números no se repiten en la pasada.	Esp. v2.0 §10.2
RF-22	Modos manual, automático y mixto respetando RN-11.	C	P0	F1	En automático se emite una secuencia por intervalo solo si la anterior terminó; manual y mixto responden al botón.	Stack v2.1 §11
RF-23	Pausar y reanudar, manualmente y de forma automática por enlace perdido, batería, espacio o app fuera de primer plano.	C	P0	F1 (simulador) / F2	Si una cámara no responde por más de 6 s, la pasada queda PAUSED y no se emiten órdenes.	Esp. v2.0 §15
RF-24	Cambiar marcador o segmento durante la pasada.	C	P1	F1	Las secuencias posteriores llevan el nuevo marcador; las anteriores no cambian.	Esp. v2.0 §13.1
RF-25	Mostrar errores de calidad o de cámara por cámara y repetir solo esa cámara (RN-07).	C	P0	F1	La repetición crea un captureId nuevo con replacesCaptureId y una fila en retake_requests con el contexto vigente; la foto de la otra cámara no se duplica.	Esp. v2.0 §12
RF-26	Cerrar la pasada con resumen y estado COMPLETED o INCOMPLETE.	C	P0	F1	INCOMPLETE exige una incidencia; los contadores coinciden con las secuencias.	Esp. v2.0 §10.3
RF-27	Cerrar la sesión de monitoreo sin órdenes nuevas; las fotos que falten se esperan o se registran con incidencia.	C	P0	F1	La sesión llega a CLOSED; las cámaras reciben SESSION_CLOSED; se encola la sincronización; una foto que llega después se acepta y se sincroniza (8.13).	Esp. v2.0 §10
RF-28	Escanear el QR y emparejarse con el rol de la función.	K	P0	F2	Con QR válido la cámara queda CONECTADA y muestra la sesión; con QR inválido o rol ocupado muestra el motivo.	Esp. v2.0 §22.2
RF-29	Mantener la vista de cámara activa y la pantalla encendida durante la sesión, con estado legible.	K	P0	F1	La pantalla no se apaga en 30 minutos de pasada; el estado se lee a 1 m.	Esp. v2.0 §8.3
RF-30	Ejecutar CAPTURE_COMMAND: capturar, guardar archivo y registro, evaluar calidad y responder.	K	P0	F1 (simulador) / F2	La respuesta sale después de guardar e incluye md5, tamaño, dimensiones, calidad y versión de perfil.	Esp. v2.0 §10.2
RF-31	Rechazar órdenes vencidas, duplicadas o de otra sesión o pasada.	K	P0	F2	Una orden vencida responde ERROR COMMAND_EXPIRED; un messageId repetido devuelve la respuesta guardada.	Esp. v2.0 §23.2
RF-32	Modo prueba de cámara en un solo celular.	K	P1	F1	Toma una foto y muestra calidad, métricas y tiempo sin crear datos de sesión.	Equipo
RF-33	Evaluar exposición y nitidez con el perfil vigente y guardar métricas, resultado, motivos y versión.	K	P0	F1	Las fotos de muestra oscura, borrosa y útil dan REPETIR_EXPOSICION, REPETIR_NITIDEZ y UTILIZABLE; el tiempo queda registrado.	Esp. v2.0 §11
RF-34	Servidor de control en el controlador y cliente en las cámaras con heartbeat, detección de desconexión y reconexión automática.	T	P0	F2	Con el router apagado 10 s y encendido otra vez, la cámara vuelve a CONECTADA sin intervención.	Esp. v2.0 §15.1
RF-35	Transferir cada foto con metadatos y md5 al controlador, con confirmación, reintentos e idempotencia.	T	P0	F2	Tras cortar la red en plena transferencia, la foto llega una sola vez y con el mismo captureId.	Esp. v2.0 §12.3
RF-36	Conciliar estados tras una reconexión antes de nuevas órdenes (RESYNC paginado, 14.9).	T	P0	F2	Tras reconectar, el controlador conoce los pendientes de la cámara y no repite secuencias.	Stack v2.1 §13
RF-37	Transferir primero las fotos útiles y después las rechazadas (configurable).	K	P2	F2	Con ambas en cola, la útil sale primero.	Propuesto
RF-38	Leer el GPS de forma continua durante la pasada y asociar a cada secuencia la última lectura válida con precisión y antigüedad (RN-18).	C	P0	F1	Sin GPS la secuencia se guarda sin coordenadas; con GPS, la antigüedad registrada es ≤ 10 s.	Esp. v2.0 §14
RF-39	Sincronizar con Spring Boot en orden, con idempotencia y reintentos.	C	P0	F4	Repetir la sincronización no duplica nada; los errores se clasifican según 15.5.	Esp. v2.0 §15.4
RF-40	Pantalla de sincronización con pendientes, progreso, errores y reintento.	C	P0	F4	Los contadores coinciden con la cola; un error definitivo muestra el motivo.	Esp. App Móvil v1.0

ID	Requerimiento	Func.	Prio.	Fase	Criterio de aceptación	Fuente
RF-41	Marcar SINCRONIZADO solo con confirmación del backend y liberar espacio solo según RN-09.	C	P0	F4	Ninguna foto se borra sin SINCRONIZADO.	Esp. v2.0 \$15.3
RF-42	Monitor de cámaras (enlace, batería, espacio, pendientes, desfase), GPS y avisos.	C	P0	F1 (simulador) / F3	Los valores se actualizan con cada heartbeat (≤ 2 s).	Stack v2.1 \$8
RF-43	Registrar incidencias automáticas y manuales.	C	P1	F1	Desconexiones, errores de calidad, batería y espacio generan incidencias; el operador puede agregar una.	Esp. v2.0 \$24
RF-44	Recuperar sesión, pasada, secuencias y colas tras cierre o reinicio, y conciliar antes de continuar.	T	P0	F1 / F2	Al cerrar la app a mitad de pasada y abrirla, la pasada queda PAUSED con todos sus datos y se reanuda.	Esp. v2.0 \$24
RF-45	Registro de eventos y exportación del diagnóstico.	T	P2	F1	El JSON exportado contiene versiones, configuración y eventos, sin contraseñas ni tokens.	Stack v2.1 \$35
RF-46	Avisar y pausar por batería o espacio bajo.	T	P1	F1	Bajo el umbral de aviso aparece el aviso; bajo el umbral crítico la pasada se pausa.	Esp. v2.0 \$24
RF-47	Resumen por pasada y por segmento.	C	P2	F1	Muestra completas, incompletas, repeticiones y duración.	Curso \$2.7
RF-48	Simulador de cámaras en Expo Go con el mismo protocolo e interfaces.	C	P0	F1	En un iPhone con Expo Go se completa una pasada con dos cámaras virtuales, con errores de calidad y caídas simuladas.	Esp. v2.0 \$25.2

10.2 Requerimientos no funcionales

ID	Atributo	Requerimiento	Cómo se verifica
RNF-01	Compatibilidad	Las tres funciones en Android 7+ e iOS 15.1+ con la misma base de código.	Pruebas de las Fases 2 a 6 (Android) y 5 (iOS).
RNF-02	Sin internet	Solo requieren internet: registro, primer login, cambio y recuperación de contraseña, catálogos y sincronización.	Pruebas en modo avión.
RNF-03	Confiabilidad	Cero pérdida de evidencia ante cierre, reinicio, corte de red o reintentos.	Fallos inducidos (21.4).
RNF-04	Idempotencia	Mensajes, transferencias y sincronización no crean duplicados.	Reintentos forzados.
RNF-05	Rendimiento de calidad	≤ 2 s por foto en los celulares del piloto (RNF02 del curso).	quality_results.duration_ms.
RNF-06	Latencia de orden	CAPTURE_COMMAND $\rightarrow$ CAPTURE_OK $\leq 1,5$ s (meta inicial, calibrar).	event_log por secuencia.
RNF-07	Desfase entre cámaras	≤ 500 ms por secuencia (meta inicial, calibrar), con capturedAt corregido por clockOffsetMs.	Cálculo por secuencia.
RNF-08	Usabilidad	Con las cámaras listas, iniciar una pasada toma ≤ 3 toques (RNF03 del curso).	Prueba con operadores.
RNF-09	Legibilidad en campo	Contraste alto, botones ≥ 48 dp, texto ≥ 16 pt, estados con icono y texto (no solo color).	Revisión de interfaz.
RNF-10	Seguridad	Tokens en SecureStore; contraseñas nunca guardadas; verificador PBKDF2; HTTPS remoto; QR con token aleatorio que vale hasta cerrar la sesión o regenerarlo; sin credenciales de infraestructura.	Revisión de código y pruebas.
RNF-11	Privacidad	Fotos en almacenamiento privado; aviso de privacidad; diagnóstico sin datos sensibles.	Revisión.
RNF-12	Autonomía	Una jornada de recorrido con power bank.	Medición en F6.
RNF-13	Almacenamiento	Aviso y pausa por espacio. Estimación: espacio por hilera ≈ 2 cámaras $\times$ secuencias por pasada $\times 2$ pasadas $\times$ tamaño medio de foto.	Prueba de espacio bajo.
RNF-14	Respuesta de interfaz	La pantalla de pasada responde al toque en < 200 ms aun con transferencias en curso (meta).	Medición manual.
RNF-15	Mantenibilidad	Dominio en TypeScript puro con pruebas; cobertura del dominio ≥ 80 % (meta); typecheck y lint sin errores.	CI local.
RNF-16	Trazabilidad	Cada registro guarda versión de app, protocolo, configuración y perfil de calidad.	Revisión de datos exportados.

ID	Atributo	Requerimiento	Cómo se verifica
RNF-17	Primer plano	Captura y coordinación solo en primer plano con pantalla encendida.	Prueba minimizando la app.
RNF-18	Instalación	Android por APK sin tienda; iOS con Xcode y Apple ID gratuito.	Fases 5 y 6.

10.3 Reglas de negocio

ID	Regla
RN-01	Ninguna función (controlador o cámara) se usa sin una sesión de usuario válida, con o sin internet.
RN-02	Solo pueden usar la app cuentas ACTIVO con rol OPERADOR_CAMPO o ADMINISTRADOR.
RN-03	El login sin internet solo es posible para el último usuario que entró con internet en ese celular, si su verificador existe, su cuenta estaba activa con rol permitido y sin cambio de contraseña pendiente, no hay bloqueo temporal y no pasaron más de offlineLoginMaxDays desde la última validación con el backend.
RN-04	Con contraseña temporal (mustChangePassword) solo se permite cambiarla (con internet) o cerrar sesión; no se crea verificador sin internet con una contraseña temporal.
RN-05	Una captura de evidencia no se registra sin sessionId, passId, lateralCode, sequenceId, captureId, deviceId, cameraRole, userId y capturedAt. lotId, rowId, segmentId y markerId (cuando están definidos) y el GPS (solo si hay lectura válida) van en su secuencia en el controlador y, en una repetición, también en retake_requests. En la cámara, una captura de evidencia exige un SESSION_CONTEXT vigente con esa pasada. Las capturas de prueba corta siguen RN-21.
RN-06	Una secuencia es COMPLETE solo si ambas cámaras tienen una foto aceptada (CAPTURE_OK: UTILIZABLE, o PENDIENTE_REVISION_TECNICA cuando la calidad no pudo decidirse a tiempo) y recibida por el controlador (deriveSequenceStatus).
RN-07	Se repite solo la cámara afectada: nueva toma con nuevo captureId y replacesCaptureId; la foto válida de la otra cámara se conserva. La secuencia conserva su contexto; el contexto vigente al repetir (segmento, marcador y GPS) se guarda en retake_requests y viaja con la foto repetida (retakeContext): si el operador ya avanzó, la nueva toma registra el contexto actual, nunca el anterior.
RN-08	Las cámaras no crean secuencias ni contexto; sin enlace de control no capturan.
RN-09	La cámara no borra una foto sin ACK del controlador; con política TRAS_SINCRONIZACION borra sus copias con ACK solo de sesiones que el controlador informa como SINCRONIZADAS. El controlador no borra una foto sin SINCRONIZADO y solo por acción manual (CONSERVAR). Nada se borra de una sesión abierta. Las fotos de prueba corta siguen RN-21.
RN-10	Iniciar una pasada exige ambas cámaras CONECTADA, prueba corta aprobada en la sesión, lote, hilera, lateral y marcador definidos, y en los tres celulares batería $\geq$ device.minBatteryToStartPct y espacio $\geq$ device.minFreeSpaceToStartBytes. Reanudar una pasada pausada exige ambas cámaras CONECTADA, ningún RESYNC en curso (toda cámara que volvió a emparejarse durante la pausa terminó su RESYNC) y, en los tres celulares, batería y espacio sobre los umbrales de pausa (device.pauseBatteryPct, device.pauseFreeSpaceBytes); no se repite la prueba corta.
RN-11	No se emite una nueva orden de captura (secuencia, repetición o prueba corta) mientras la anterior no tenga respuesta (CAPTURE_OK, QUALITY_ERROR o ERROR) de las cámaras a las que se envió o no venza captureResponseTimeoutMs.
RN-12	En una sesión, la pasada LATERAL_B de una hilera exige una pasada LATERAL_A cerrada de esa hilera en la misma sesión. Un lateral se puede repetir con «Repetir lateral»: nueva pasada con passOrder = máximo de la hilera en la sesión + 1 e incidencia obligatoria. Solo puede haber una pasada abierta (READY, ACTIVE o PAUSED) por sesión.
RN-13	Una hilera queda cubierta cuando tiene pasadas cerradas en LATERAL_A y LATERAL_B (COMPLETED, o INCOMPLETE con incidencia).
RN-14	UTILIZABLE significa técnicamente útil, no «sin plaga». Una foto rechazada no se usa como negativo en la IA sin una decisión explícita.
RN-15	La función del dispositivo solo cambia sin sesión de monitoreo abierta (7.12) y sin transferencias ni sincronizaciones pendientes en ese celular.
RN-16	En una sesión, cada rol de cámara lo ocupa un solo dispositivo activo y un dispositivo ocupa un solo rol; un dispositivo que se reconecta conserva su rol. El operador puede liberar el rol de una cámara desconectada (PANT-13) para reemplazarla por otro celular; el celular liberado solo vuelve si su rol sigue libre.
RN-17	Una cámara con otra versión de protocolo siempre se rechaza (PROTOCOL_MISMATCH). Con ENV.appEnv = 'piloto' (pairing.requireSameAppVersion = true), además debe tener la misma versión de app que el controlador (APP_VERSION_MISMATCH); en dev se permite otra versión de app.

ID	Regla
RN-18	No se inventan coordenadas: se usa la última lectura del GPS si su antigüedad es $\leq$ <code>gps.maxAgeMs</code> ; si no, la secuencia queda sin coordenadas y con la condición registrada.
RN-19	No se puede cerrar la sesión de usuario con una sesión de monitoreo abierta (7.12). Cerrar sesión nunca borra datos de campo. Esta regla no se aplica a la revocación (7.8).
RN-20	Tras <code>offlineMaxFailedAttempts</code> intentos fallidos sin internet, el login sin internet se bloquea <code>offlineLockoutMinutes</code> minutos.
RN-21	Las capturas de prueba corta (<code>isTest</code>) no son evidencia: no pertenecen a una pasada ni a una secuencia, no cambian contadores, nunca se sincronizan y sus archivos se borran al cerrar la sesión (controlador) o el contexto (cámara) (14.12).
RN-22	Una foto pertenece siempre a su sesión: la cámara la envía solo al controlador de esa sesión, y el controlador la acepta aunque la sesión esté cerrada o sincronizada si el dispositivo estuvo registrado en ella (8.13).

10.4 Historias de usuario

ID	Historia	Requerimientos
HU-01	Como operador nuevo, quiero registrarme desde la app para solicitar acceso.	RF-01
HU-02	Como operador, quiero iniciar sesión en cada celular para que todo quede registrado a mi nombre.	RF-02, RF-03, RF-05
HU-03	Como operador en campo sin internet, quiero volver a entrar con mi contraseña.	RF-04, RF-10
HU-04	Como operador que olvidó su contraseña, quiero pedir ayuda al administrador y cambiar la contraseña temporal.	RF-06, RF-07, RF-08
HU-05	Como operador, quiero elegir la función de cada celular.	RF-12, RF-13, RF-14
HU-06	Como operador, quiero vincular las cámaras escaneando un QR.	RF-18, RF-28
HU-07	Como operador, quiero crear la sesión y la pasada eligiendo lote, hilera, lateral y marcador.	RF-15, RF-16, RF-17, RF-20
HU-08	Como operador, quiero disparar las capturas desde el controlador en modo manual o automático.	RF-21, RF-22, RF-23
HU-09	Como operador, quiero saber qué cámara tomó una foto deficiente y repetir solo esa.	RF-25, RF-33
HU-10	Como operador, quiero que no se pierda ninguna foto aunque se corte la red o se cierre la app.	RF-35, RF-36, RF-44
HU-11	Como operador, quiero ver el estado de cada cámara.	RF-42, RF-46
HU-12	Como operador, quiero cerrar la pasada A y continuar con la B sin mezclar datos.	RF-20, RF-26
HU-13	Como operador, quiero sincronizar al volver y ver qué falta.	RF-39, RF-40, RF-41
HU-14	Como investigador, quiero que cada foto lleve versiones y contexto para auditar las pruebas.	RF-45, RF-47, RNF-16

11. Contratos de dominio (TypeScript)

Estos archivos se copian tal cual al repositorio. Compilan en modo estricto y están cubiertos por pruebas (sección 21.1).

11.1 Tipos — `src/domain/types.ts`

```
// src/domain/types.ts – Contrato de tipos del dominio (versión 1). No modificar sin actualizar el archivo maestro.
/** Fecha y hora ISO-8601 en UTC con milisegundos, p. ej. "2026-11-09T14:32:05.120Z" (Date.prototype.toISOString). */
export type ISODateString = string;
/** UUID v4 en minúsculas generado con Crypto.randomUUID(). */
export type UUID = string;
// ----- usuarios y cuentas
export type UserRole = 'ADMINISTRADOR' | 'OPERADOR_CAMPO' | 'ESPECIALISTA_FITOSANITARIO' | 'SUPERVISOR';
/** Roles de usuario que pueden usar la app móvil (los demás solo usan la web). */
export const MOBILE_ALLOWED_ROLES: readonly UserRole[] = ['OPERADOR_CAMPO', 'ADMINISTRADOR'];
export type AccountStatus = 'PENDIENTE_APROBACION' | 'ACTIVO' | 'RECHAZADO' | 'BLOQUEADO';
export interface UserProfile {
  id: UUID;
```

```

    fullName: string;
    email: string;
    roles: UserRole[];
    status: AccountStatus;
    mustChangePassword: boolean;
}
export type AuthMode = 'ONLINE' | 'OFFLINE';
export type AuthStatus = 'SIN_SESION' | 'CAMBIO_CONTRASENA_REQUERIDO' | 'AUTENTICADO';
export interface AuthState {
    status: AuthStatus;
    mode: AuthMode | null;
    user: UserProfile | null;
    /** Último login o renovación validados por el backend. */
    lastOnlineAuthAt: ISODateString | null;
    /** Límite para volver a entrar sin internet (lastOnlineAuthAt + offlineLoginMaxDays). */
    offlineValidUntil: ISODateString | null;
}
// ----- dispositivo
export type DeviceRole = 'CONTROLADOR' | 'CAMERA_1' | 'CAMERA_2';
export type CameraRole = Exclude<DeviceRole, 'CONTROLADOR'>;
export const CAMERA_ROLES: readonly CameraRole[] = ['CAMERA_1', 'CAMERA_2'];
export type Platform = 'android' | 'ios';
export interface DeviceIdentity {
    /** Generado una sola vez por instalación y guardado en SecureStore + app_meta. */
    deviceId: UUID;
    platform: Platform;
    model: string;
    osVersion: string;
    appVersion: string;
    buildNumber: string;
}
// ----- catálogos de campo (bootstrap)
export interface Lot { id: string; code: string; name: string; active: boolean; }
export interface FieldRow { id: string; lotId: string; number: number; plantCount: number; active: boolean; }
export interface Segment { id: string; rowId: string; code: string; startPlant: number; endPlant: number; isPilot: boolean; }
export type MarkerPosition = 'INICIO' | 'FIN' | 'INTERMEDIO';
export interface Marker {
    id: string;
    rowId: string;
    segmentId: string | null;
    code: string;
    description: string | null;
    position: MarkerPosition;
    lat: number | null;
    lon: number | null;
}
// ----- recorrido
export type LateralCode = 'LATERAL_A' | 'LATERAL_B';
export type CaptureMode = 'MANUAL' | 'AUTOMATICO' | 'MIXTO';
/** Sentido de avance respecto de la numeración de plantas de la hilera. */
export type Direction = 'ASCENDENTE' | 'DESCENDENTE';
export type SessionStatus = 'DRAFT' | 'PREPARING' | 'READY' | 'ACTIVE' | 'PAUSED' | 'CLOSING' | 'CLOSED' | 'SYNCED';
export type PassStatus = 'READY' | 'ACTIVE' | 'PAUSED' | 'COMPLETED' | 'INCOMPLETE';
export type SequenceStatus = 'CREATED' | 'COMMAND_SENT' | 'PARTIAL' | 'COMPLETE' | 'INCOMPLETE' | 'CANCELLED';
export type QualityStatus =
    | 'CAPTURED'
    | 'UTILIZABLE'
    | 'REPETIR_NITIDEZ'
    | 'REPETIR_EXPOSICION'
    | 'ERROR_CAMARA'
    | 'PENDIENTE_REVISION_TECNICA';
export type LocalTransferStatus = 'PENDIENTE_LOCAL' | 'TRANSFIRIENDO_LOCAL' | 'RECIBIDA_CONTROLADOR' | 'ERROR_LOCAL';
export type RemoteSyncStatus = 'PENDIENTE_NUBE' | 'SUBIENDO' | 'SINCRONIZADO' | 'ERROR_SINCRONIZACION';
export type CameraLinkStatus = 'DESCONECTADA' | 'EMPAREJANDO' | 'CONECTADA' | 'INESTABLE' | 'PERDIDA';
export interface GpsFix {
    lat: number;
    lon: number;
    accuracyM: number | null;
    /** Hora de la lectura del GPS (no la hora de la secuencia). */
    timestamp: ISODateString;
}
export interface MonitoringSession {
    sessionId: UUID;
    operatorUserId: UUID;
    controllerDeviceId: UUID;
    status: SessionStatus;
    mode: CaptureMode;
    intervalMs: number;
    startedAt: ISODateString | null;
    endedAt: ISODateString | null;
    appVersion: string;
}

```

```

    configVersion: string;
    qualityProfileVersion: string;
    /** Hash SHA-256 del token del QR vigente (el token no se guarda). */
    pairingTokenHash: string | null;
    shortTestPassedAt: ISODateString | null;
    createdAt: ISODateString;
    updatedAt: ISODateString;
    remoteSyncStatus: RemoteSyncStatus;
}
export interface SessionDevice {
    sessionId: UUID;
    role: CameraRole;
    deviceId: UUID;
    userId: UUID;
    platform: Platform;
    model: string;
    osVersion: string;
    appVersion: string;
    pairedAt: ISODateString;
    lastSeenAt: ISODateString | null;
    linkStatus: CameraLinkStatus;
    /** true cuando el operador liberó el rol para reemplazar el celular (PANT-13). */
    released: boolean;
    batteryLevel: number | null;
    freeSpaceBytes: number | null;
    pendingTransfers: number;
    /** Diferencia estimada reloj cámara - reloj controlador (ms). */
    clockOffsetMs: number | null;
}
export interface MonitoringPass {
    passId: UUID;
    sessionId: UUID;
    lotId: string;
    rowId: string;
    lateralCode: LateralCode;
    /** Orden de creación de las pasadas de esa hilera en la sesión: 1, 2, 3... (una repetición toma el siguiente). */
    passOrder: number;
    direction: Direction;
    startMarkerId: string | null;
    endMarkerId: string | null;
    currentSegmentId: string | null;
    currentMarkerId: string | null;
    status: PassStatus;
    startedAt: ISODateString | null;
    endedAt: ISODateString | null;
    sequencesTotal: number;
    sequencesComplete: number;
    sequencesIncomplete: number;
    remoteSyncStatus: RemoteSyncStatus;
}
/** Resultado de cada cámara dentro de una secuencia. */
export type SlotOutcome =
    | 'PENDIENTE'
    | 'OK_PENDIENTE_ARCHIVO'
    | 'OK_RECIBIDA'
    | 'RECHAZADA_CALIDAD'
    | 'ERROR_CAMARA'
    | 'SIN_RESPUESTA';
export interface SequenceSlot {
    role: CameraRole;
    /** captureId vigente para este rol (cambia si se repite la toma). */
    captureId: UUID;
    outcome: SlotOutcome;
}
export interface CaptureSequence {
    sequenceId: UUID;
    passId: UUID;
    sessionId: UUID;
    sequenceNumber: number;
    mode: CaptureMode;
    status: SequenceStatus;
    lotId: string;
    rowId: string;
    segmentId: string | null;
    markerId: string | null;
    gps: GpsFix | null;
    gpsAgeMs: number | null;
    issuedAt: ISODateString;
    expiresAt: ISODateString;
    completedAt: ISODateString | null;
    slots: Record<CameraRole, SequenceSlot>;
    remoteSyncStatus: RemoteSyncStatus;
}

```

```

}
export type QualityReason =
  | 'EXPOSICION_OSCURA'
  | 'EXPOSICION_SATURADA'
  | 'NITIDEZ_BAJA'
  | 'ARCHIVO_INVALIDO'
  | 'FALLO_CAMARA'
  | 'TIEMPO_AGOTADO';
export interface QualityMetrics {
  luminanceMean: number;
  darkRatio: number;
  brightRatio: number;
  /** Máximo de la varianza del Laplaciano entre las regiones analizadas. */
  laplacianVariance: number;
  analyzedRegions: number;
  durationMs: number;
}
export interface QualityResult {
  status: Exclude<QualityStatus, 'CAPTURED'>;
  reasons: QualityReason[];
  metrics: QualityMetrics | null;
  profileVersion: string;
}
/** Una captura de evidencia (isTest = false) cumple RN-05. Las de prueba corta (isTest = true) no tienen pasada ni la
    lateral. */
export interface Capture {
  captureId: UUID;
  sequenceId: UUID;
  sessionId: UUID;
  passId: UUID | null;
  lateralCode: LateralCode | null;
  isTest: boolean;
  deviceId: UUID;
  cameraRole: CameraRole;
  /** Usuario con sesión iniciada en la cámara que tomó la foto. */
  userId: UUID;
  capturedAt: ISODateString;
  filePath: string | null;
  sizeBytes: number | null;
  width: number | null;
  height: number | null;
  md5: string | null;
  qualityStatus: QualityStatus;
  qualityProfileVersion: string;
  replacesCaptureId: UUID | null;
  localTransferStatus: LocalTransferStatus;
  /** Solo en el controlador; null en las cámaras. */
  remoteSyncStatus: RemoteSyncStatus | null;
  transferAttempts: number;
  syncAttempts: number;
  lastError: string | null;
  /** Fecha en que se liberó el archivo por retención (la fila se conserva). */
  fileDeletedAt: ISODateString | null;
}
/**
 * Repetición pedida por el operador (controlador). Guarda el contexto vigente en el momento de repetir:
 * la secuencia conserva su contexto original y la foto repetida se sincroniza con este (RN-07).
 */
export interface RetakeRequest {
  /** captureId nuevo emitido para la repetición. */
  captureId: UUID;
  sequenceId: UUID;
  passId: UUID;
  cameraRole: CameraRole;
  replacesCaptureId: UUID;
  requestedAt: ISODateString;
  segmentId: string | null;
  markerId: string | null;
  gps: GpsFix | null;
  gpsAgeMs: number | null;
}
/** Cambio de marcador o segmento durante una pasada (controlador). */
export interface MarkerChange {
  markerChangeId: UUID;
  passId: UUID;
  markerId: string | null;
  segmentId: string | null;
  changedAt: ISODateString;
  gps: GpsFix | null;
}
/** Sesión a la que está unida una cámara (tabla camera_context). */
export interface CameraContext {

```

```

    sessionId: UUID;
    controllerDeviceId: UUID;
    role: CameraRole;
    controllerHost: string;
    controlPort: number;
    filePort: number;
    pairedAt: ISODateString;
    /** JSON de PAIRED.config (se valida con zod al leerlo). */
    configJson: string;
    /** JSON del último SESSION_CONTEXT. */
    contextJson: string | null;
    closedAt: ISODateString | null;
    updatedAt: ISODateString;
}
export type IncidentType =
  | 'CALIDAD'
  | 'DESCONEXION'
  | 'BATERIA'
  | 'TEMPERATURA'
  | 'ESPACIO'
  | 'GPS'
  | 'TRANSFERENCIA'
  | 'SINCRONIZACION'
  | 'SOPORTE'
  | 'OPERADOR'
  | 'OTRO';
export interface Incident {
  incidentId: UUID;
  sessionId: UUID;
  passId: UUID | null;
  sequenceId: UUID | null;
  captureId: UUID | null;
  deviceId: UUID | null;
  type: IncidentType;
  severity: 'INFO' | 'AVISO' | 'ERROR';
  detail: string;
  occurredAt: ISODateString;
  createdBy: 'SISTEMA' | 'OPERADOR';
}

```

11.2 Transiciones permitidas — src/domain/stateMachines.ts

```

// src/domain/stateMachines.ts — Transiciones permitidas (versión 1).
import type {
  AuthStatus,
  CameraLinkStatus,
  LocalTransferStatus,
  PassStatus,
  QualityStatus,
  RemoteSyncStatus,
  SequenceStatus,
  SessionStatus,
  SlotOutcome,
} from './types';
export type TransitionMap<S extends string> = Readonly<Record<S, readonly S[]>>;
export const SESSION_TRANSITIONS: TransitionMap<SessionStatus> = {
  DRAFT: ['PREPARING'], // un borrador que se descarta se elimina (no tiene datos dependientes)
  PREPARING: ['READY', 'CLOSING'],
  READY: ['ACTIVE', 'PREPARING', 'CLOSING'],
  ACTIVE: ['PAUSED', 'READY', 'CLOSING'],
  PAUSED: ['ACTIVE', 'READY', 'CLOSING'],
  CLOSING: ['CLOSED'],
  CLOSED: ['SYNCED'],
  // Si llegan datos tardíos de una sesión ya sincronizada (p. ej., una foto pendiente), vuelve a CLOSED hasta sincronizarlos.
  SYNCED: ['CLOSED'],
};
export const PASS_TRANSITIONS: TransitionMap<PassStatus> = {
  READY: ['ACTIVE', 'INCOMPLETE'],
  ACTIVE: ['PAUSED', 'COMPLETED', 'INCOMPLETE'],
  PAUSED: ['ACTIVE', 'COMPLETED', 'INCOMPLETE'],
  COMPLETED: [],
  INCOMPLETE: [],
};
/** El estado de la secuencia se deriva de los resultados por cámara (deriveSequenceStatus). */
export const SEQUENCE_TRANSITIONS: TransitionMap<SequenceStatus> = {
  CREATED: ['COMMAND_SENT', 'CANCELLED'],
  COMMAND_SENT: ['PARTIAL', 'COMPLETE', 'INCOMPLETE'],
  PARTIAL: ['COMMAND_SENT', 'COMPLETE', 'INCOMPLETE'],
  // Una repetición o un resultado tardío pueden reabrir o completar una secuencia incompleta (RN-07).
}

```

```

    INCOMPLETE: ['COMMAND_SENT', 'PARTIAL', 'COMPLETE'],
    COMPLETE: [],
    CANCELLED: [],
  };
  /** Resultado de cada cámara dentro de una secuencia. */
  export const SLOT_TRANSITIONS: TransitionMap<SlotOutcome> = {
    PENDIENTE: ['OK_PENDIENTE_ARCHIVO', 'OK_RECIBIDA', 'RECHAZADA_CALIDAD', 'ERROR_CAMARA', 'SIN_RESPUESTA'],
    OK_PENDIENTE_ARCHIVO: ['OK_RECIBIDA'],
    OK_RECIBIDA: [],
    RECHAZADA_CALIDAD: ['PENDIENTE'], // repetición
    ERROR_CAMARA: ['PENDIENTE'], // repetición
    // Resultado tardío (llegó después de vencer la espera) o repetición.
    SIN_RESPUESTA: ['OK_PENDIENTE_ARCHIVO', 'OK_RECIBIDA', 'RECHAZADA_CALIDAD', 'ERROR_CAMARA', 'PENDIENTE'],
  };
  export const QUALITY_TRANSITIONS: TransitionMap<QualityStatus> = {
    CAPTURED: ['UTILIZABLE', 'REPETIR_NITIDEZ', 'REPETIR_EXPOSICION', 'ERROR_CAMARA', 'PENDIENTE_REVISION_TECNICA'],
    UTILIZABLE: [],
    REPETIR_NITIDEZ: [],
    REPETIR_EXPOSICION: [],
    ERROR_CAMARA: [],
    PENDIENTE_REVISION_TECNICA: [],
  };
  export const LOCAL_TRANSFER_TRANSITIONS: TransitionMap<LocalTransferStatus> = {
    PENDIENTE_LOCAL: ['TRANSFIRIENDO_LOCAL'],
    TRANSFIRIENDO_LOCAL: ['RECIBIDA_CONTROLADOR', 'PENDIENTE_LOCAL', 'ERROR_LOCAL'],
    RECIBIDA_CONTROLADOR: [],
    ERROR_LOCAL: ['PENDIENTE_LOCAL'],
  };
  export const REMOTE_SYNC_TRANSITIONS: TransitionMap<RemoteSyncStatus> = {
    PENDIENTE_NUBE: ['SUBIENDO'],
    SUBIENDO: ['SINCRONIZADO', 'PENDIENTE_NUBE', 'ERROR_SINCRONIZACION'],
    // Solo una pasada y sus secuencias reabiertas por una foto tardía (8.13) vuelven a PENDIENTE_NUBE.
    SINCRONIZADO: ['PENDIENTE_NUBE'],
    ERROR_SINCRONIZACION: ['PENDIENTE_NUBE'],
  };
  export const CAMERA_LINK_TRANSITIONS: TransitionMap<CameraLinkStatus> = {
    DESCONECTADA: ['EMPAREJANDO'],
    EMPAREJANDO: ['CONECTADA', 'DESCONECTADA'],
    CONECTADA: ['INESTABLE', 'PERDIDA', 'DESCONECTADA'],
    INESTABLE: ['CONECTADA', 'PERDIDA', 'DESCONECTADA'],
    PERDIDA: ['EMPAREJANDO', 'DESCONECTADA'],
  };
  export const AUTH_TRANSITIONS: TransitionMap<AuthStatus> = {
    SIN_SESION: ['AUTENTICADO', 'CAMBIO_CONTRASENA_REQUERIDO'],
    CAMBIO_CONTRASENA_REQUERIDO: ['AUTENTICADO', 'SIN_SESION'],
    AUTENTICADO: ['SIN_SESION'],
  };
  export function canTransition<S extends string>(map: TransitionMap<S>, from: S, to: S): boolean {
    return map[from].includes(to);
  }
  export class InvalidTransitionError extends Error {
    constructor(public readonly entity: string, public readonly from: string, public readonly to: string) {
      super('Transición inválida en ${entity}: ${from} → ${to}');
      this.name = 'InvalidTransitionError';
    }
  }
  export function assertTransition<S extends string>(entity: string, map: TransitionMap<S>, from: S, to: S): void {
    if (!canTransition(map, from, to)) throw new InvalidTransitionError(entity, from, to);
  }
}

```

11.3 Estado de la secuencia — src/domain/sequence.ts

```

// src/domain/sequence.ts – Estado de una secuencia derivado del resultado de cada cámara.
import type { CameraRole, SequenceSlot, SequenceStatus, SlotOutcome } from './types';
const FINAL_OUTCOMES: readonly SlotOutcome[] = ['OK_RECIBIDA', 'RECHAZADA_CALIDAD', 'ERROR_CAMARA', 'SIN_RESPUESTA'];
export function isFinalOutcome(outcome: SlotOutcome): boolean {
  return FINAL_OUTCOMES.includes(outcome);
}
/**
 * Regla (RN-06): COMPLETE solo si ambas cámaras tienen una foto aceptada (CAPTURE_OK) y recibida por el controlador.
 * - Si todavía falta algún resultado final: PARTIAL (si alguna cámara ya respondió) o COMMAND_SENT.
 * - Si todos los resultados son finales y no ambos son OK_RECIBIDA: INCOMPLETE.
 */
export function deriveSequenceStatus(slots: Record<CameraRole, SequenceSlot>): SequenceStatus {
  const outcomes = [slots.CAMERA_1.outcome, slots.CAMERA_2.outcome];
  if (outcomes.every((o) => o === 'OK_RECIBIDA')) return 'COMPLETE';
  if (outcomes.every(isFinalOutcome)) return 'INCOMPLETE';
  if (outcomes.every((o) => o === 'PENDIENTE')) return 'COMMAND_SENT';
  return 'PARTIAL';
}

```

```

/** Una repetición reemplaza el captureId del rol y reabre su resultado. */
export function applyRetake(
  slots: Record<CameraRole, SequenceSlot>,
  role: CameraRole,
  newCaptureId: string,
): Record<CameraRole, SequenceSlot> {
  return { ...slots, [role]: { role, captureId: newCaptureId, outcome: 'PENDIENTE' } };
}

```

12. Base de datos local

12.1 Esquema v1 — src/storage/migrations/001_initial.ts

El SQL se exporta como texto (export const MIGRATION_001 = `...`). Probado en SQLite 3.45 con claves foráneas activas. Las columnas usan snake_case; los repositorios convierten a los tipos camelCase del dominio y nunca exponen filas crudas. Las cámaras usan app_meta, users_cache, camera_context, captures, quality_results, transfer_queue, processed_messages y event_log; el controlador usa todas menos camera_context y transfer_queue. En las cámaras no existe capture_sequences, por eso captures.sequence_id no tiene clave foránea (en una captura de prueba corta guarda el identificador de la prueba). Un celular ocupa un solo rol por sesión (clave primaria session_id, device_id) y cada rol tiene un solo celular activo (índice único parcial con released=0).

```

-- src/storage/migrations/001_initial.sql - Esquema SQLite v1 (PRAGMA user_version = 1).
-- Fechas: TEXT ISO-8601 UTC. IDs: TEXT UUID. Booleanos: INTEGER 0/1. JSON: TEXT.
-- Se ejecuta dentro de una transacción exclusiva; al terminar: PRAGMA user_version = 1.
CREATE TABLE IF NOT EXISTS app_meta (
  key TEXT PRIMARY KEY NOT NULL,
  value TEXT
);
-- Claves usadas: device_id, device_role, catalog_version, bootstrap_at, config_version,
-- quality_profile_version, offline_failed_attempts, offline_locked_until,
-- short_test_pending (controlador: JSON {sessionId, testId, captureIds, issuedAt} de la prueba corta en curso).
CREATE TABLE IF NOT EXISTS users_cache (
  user_id TEXT PRIMARY KEY NOT NULL,
  email TEXT NOT NULL,
  full_name TEXT NOT NULL,
  roles_json TEXT NOT NULL,
  account_status TEXT NOT NULL CHECK (account_status IN ('PENDIENTE_APROBACION', 'ACTIVO', 'RECHAZADO', 'BLOQUEADO'
    )),
  must_change_password INTEGER NOT NULL DEFAULT 0 CHECK (must_change_password IN (0,1)),
  last_online_auth_at TEXT,
  updated_at TEXT NOT NULL
);
----- catálogos (bootstrap)
CREATE TABLE IF NOT EXISTS cat_lots (
  id TEXT PRIMARY KEY NOT NULL,
  code TEXT NOT NULL UNIQUE,
  name TEXT NOT NULL,
  active INTEGER NOT NULL DEFAULT 1 CHECK (active IN (0,1))
);
CREATE TABLE IF NOT EXISTS cat_rows (
  id TEXT PRIMARY KEY NOT NULL,
  lot_id TEXT NOT NULL REFERENCES cat_lots(id),
  number INTEGER NOT NULL CHECK (number > 0),
  plant_count INTEGER NOT NULL CHECK (plant_count >= 0),
  active INTEGER NOT NULL DEFAULT 1 CHECK (active IN (0,1)),
  UNIQUE (lot_id, number)
);
CREATE TABLE IF NOT EXISTS cat_segments (
  id TEXT PRIMARY KEY NOT NULL,
  row_id TEXT NOT NULL REFERENCES cat_rows(id),
  code TEXT NOT NULL UNIQUE,
  start_plant INTEGER NOT NULL CHECK (start_plant > 0),
  end_plant INTEGER NOT NULL,
  is_pilot INTEGER NOT NULL DEFAULT 0 CHECK (is_pilot IN (0,1)),
  CHECK (end_plant >= start_plant)
);
CREATE TABLE IF NOT EXISTS cat_markers (
  id TEXT PRIMARY KEY NOT NULL,
  row_id TEXT NOT NULL REFERENCES cat_rows(id),
  segment_id TEXT REFERENCES cat_segments(id),
  code TEXT NOT NULL UNIQUE,
  description TEXT,

```

```

position    TEXT NOT NULL CHECK (position IN ('INICIO','FIN','INTERMEDIO')),
lat         REAL,
lon         REAL
);
----- recorrido (controlador)
CREATE TABLE IF NOT EXISTS monitoring_sessions (
  session_id      TEXT PRIMARY KEY NOT NULL,
  operator_user_id TEXT NOT NULL,
  controller_device_id TEXT NOT NULL,
  status          TEXT NOT NULL CHECK (status IN ('DRAFT','PREPARING','READY','ACTIVE','PAUSED','CLOSING','CLO
    ↳ SED','SYNCED')),
  mode            TEXT NOT NULL CHECK (mode IN ('MANUAL','AUTOMATICO','MIXTO')),
  interval_ms     INTEGER NOT NULL CHECK (interval_ms > 0),
  pairing_token_hash TEXT,
  short_test_passed_at TEXT,
  started_at      TEXT,
  ended_at        TEXT,
  app_version     TEXT NOT NULL,
  config_version  TEXT NOT NULL,
  quality_profile_version TEXT NOT NULL,
  remote_sync_status TEXT NOT NULL DEFAULT 'PENDIENTE_NUBE'
    CHECK (remote_sync_status IN ('PENDIENTE_NUBE','SUBIENDO','SINCRONIZADO','ERROR_SINCRONIZACI
    ↳ ON')),
  created_at      TEXT NOT NULL,
  updated_at      TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS session_devices (
  session_id      TEXT NOT NULL REFERENCES monitoring_sessions(session_id),
  role            TEXT NOT NULL CHECK (role IN ('CAMERA_1','CAMERA_2')),
  device_id       TEXT NOT NULL,
  user_id         TEXT NOT NULL,
  platform        TEXT NOT NULL CHECK (platform IN ('android','ios')),
  model           TEXT NOT NULL,
  os_version      TEXT NOT NULL,
  app_version     TEXT NOT NULL,
  paired_at       TEXT NOT NULL,
  last_seen_at    TEXT,
  link_status     TEXT NOT NULL CHECK (link_status IN ('DESCONECTADA','EMPAREJANDO','CONECTADA','INESTABLE','PERDID
    ↳ A')),
  battery_level   REAL,
  free_space_bytes INTEGER,
  pending_transfers INTEGER NOT NULL DEFAULT 0,
  clock_offset_ms INTEGER,
  released        INTEGER NOT NULL DEFAULT 0 CHECK (released IN (0,1)), -- 1 = rol liberado para reemplazar el ce
    ↳ lular
  PRIMARY KEY (session_id, device_id) -- un celular, un rol por sesión (RN-16
    ↳ )
);
-- Un solo celular activo (no liberado) por rol y sesión.
CREATE UNIQUE INDEX IF NOT EXISTS ux_session_devices_active_role ON session_devices(session_id, role) WHERE released =
  ↳ 0;
CREATE TABLE IF NOT EXISTS monitoring_passes (
  pass_id         TEXT PRIMARY KEY NOT NULL,
  session_id      TEXT NOT NULL REFERENCES monitoring_sessions(session_id),
  lot_id          TEXT NOT NULL,
  row_id          TEXT NOT NULL,
  lateral_code     TEXT NOT NULL CHECK (lateral_code IN ('LATERAL_A','LATERAL_B')),
  pass_order      INTEGER NOT NULL CHECK (pass_order > 0),
  direction       TEXT NOT NULL CHECK (direction IN ('ASCENDENTE','DESCENDENTE')),
  start_marker_id TEXT,
  end_marker_id   TEXT,
  current_segment_id TEXT,
  current_marker_id TEXT,
  status          TEXT NOT NULL CHECK (status IN ('READY','ACTIVE','PAUSED','COMPLETED','INCOMPLETE')),
  started_at      TEXT,
  ended_at        TEXT,
  sequences_total  INTEGER NOT NULL DEFAULT 0,
  sequences_complete INTEGER NOT NULL DEFAULT 0,
  sequences_incomplete INTEGER NOT NULL DEFAULT 0,
  remote_sync_status TEXT NOT NULL DEFAULT 'PENDIENTE_NUBE'
    CHECK (remote_sync_status IN ('PENDIENTE_NUBE','SUBIENDO','SINCRONIZADO','ERROR_SINCRONIZACION
    ↳ ')),
  created_at      TEXT NOT NULL,
  updated_at      TEXT NOT NULL,
  UNIQUE (session_id, row_id, pass_order)
);
CREATE TABLE IF NOT EXISTS capture_sequences (
  sequence_id     TEXT PRIMARY KEY NOT NULL,
  pass_id         TEXT NOT NULL REFERENCES monitoring_passes(pass_id),
  session_id      TEXT NOT NULL REFERENCES monitoring_sessions(session_id),
  sequence_number  INTEGER NOT NULL CHECK (sequence_number > 0),
  mode            TEXT NOT NULL CHECK (mode IN ('MANUAL','AUTOMATICO','MIXTO')),

```

```

status          TEXT NOT NULL CHECK (status IN ('CREATED','COMMAND_SENT','PARTIAL','COMPLETE','INCOMPLETE','CANC
↳ ELLED')),
lot_id          TEXT NOT NULL,
row_id          TEXT NOT NULL,
segment_id      TEXT,
marker_id       TEXT,
lat             REAL,
lon             REAL,
gps_accuracy_m  REAL,
gps_timestamp   TEXT,
gps_age_ms      INTEGER,
issued_at      TEXT NOT NULL,
expires_at      TEXT NOT NULL,
completed_at    TEXT,
slot1_capture_id TEXT NOT NULL,
slot1_outcome    TEXT NOT NULL DEFAULT 'PENDIENTE',
slot2_capture_id TEXT NOT NULL,
slot2_outcome    TEXT NOT NULL DEFAULT 'PENDIENTE',
remote_sync_status TEXT NOT NULL DEFAULT 'PENDIENTE_NUBE'
CHECK (remote_sync_status IN ('PENDIENTE_NUBE','SUBIENDO','SINCRONIZADO','ERROR_SINCRONIZACION')
↳ ),
CHECK (slot1_outcome IN ('PENDIENTE','OK_PENDIENTE_ARCHIVO','OK_RECIBIDA','RECHAZADA_CALIDAD','ERROR_CAMARA','SIN_RE
↳ SPUESTA')),
CHECK (slot2_outcome IN ('PENDIENTE','OK_PENDIENTE_ARCHIVO','OK_RECIBIDA','RECHAZADA_CALIDAD','ERROR_CAMARA','SIN_RE
↳ SPUESTA')),
UNIQUE (pass_id, sequence_number)
);
CREATE TABLE IF NOT EXISTS marker_changes (
marker_change_id TEXT PRIMARY KEY NOT NULL,
pass_id          TEXT NOT NULL REFERENCES monitoring_passes(pass_id),
marker_id        TEXT,
segment_id       TEXT,
changed_at       TEXT NOT NULL,
lat              REAL,
lon              REAL,
gps_accuracy_m   REAL,
gps_timestamp    TEXT
);
-- Repeticiones pedidas por el operador: contexto vigente al repetir (RN-07).
CREATE TABLE IF NOT EXISTS retake_requests (
capture_id       TEXT PRIMARY KEY NOT NULL,          -- captureId nuevo de la repetición
sequence_id      TEXT NOT NULL REFERENCES capture_sequences(sequence_id),
pass_id          TEXT NOT NULL REFERENCES monitoring_passes(pass_id),
camera_role      TEXT NOT NULL CHECK (camera_role IN ('CAMERA_1','CAMERA_2')),
replaces_capture_id TEXT NOT NULL,
requested_at     TEXT NOT NULL,
segment_id       TEXT,
marker_id        TEXT,
lat              REAL,
lon              REAL,
gps_accuracy_m   REAL,
gps_timestamp    TEXT,
gps_age_ms       INTEGER
);
-- ----- contexto recibido (solo cámaras)
CREATE TABLE IF NOT EXISTS camera_context (
session_id       TEXT PRIMARY KEY NOT NULL,
controller_device_id TEXT NOT NULL,
role             TEXT NOT NULL CHECK (role IN ('CAMERA_1','CAMERA_2')),
controller_host  TEXT NOT NULL,  -- datos del QR para reconectar (el token va en SecureStore: pairing.token)
control_port     INTEGER NOT NULL,
file_port        INTEGER NOT NULL,
paired_at        TEXT NOT NULL,
config_json      TEXT NOT NULL,  -- PAIRED.config
context_json     TEXT,          -- último SESSION_CONTEXT
closed_at        TEXT,
updated_at       TEXT NOT NULL
);
-- ----- capturas (cámaras y controlador)
CREATE TABLE IF NOT EXISTS captures (
capture_id       TEXT PRIMARY KEY NOT NULL,
sequence_id      TEXT NOT NULL,
session_id       TEXT NOT NULL,
pass_id          TEXT,          -- NULL solo en capturas de prueba corta
lateral_code     TEXT CHECK (lateral_code IS NULL OR lateral_code IN ('LATERAL_A','LATERAL_B')),
is_test          INTEGER NOT NULL DEFAULT 0 CHECK (is_test IN (0,1)),
device_id        TEXT NOT NULL,
camera_role      TEXT NOT NULL CHECK (camera_role IN ('CAMERA_1','CAMERA_2')),
user_id          TEXT NOT NULL,
captured_at      TEXT NOT NULL,
file_path        TEXT,
size_bytes       INTEGER,

```

```

width                INTEGER,
height              INTEGER,
md5                 TEXT,
quality_status      TEXT NOT NULL CHECK (quality_status IN ('CAPTURED','UTILIZABLE','REPETIR_NITIDEZ','REPETIR_E
    ↳ XPOSICION','ERROR_CAMARA','PENDIENTE_REVISION_TECNICA')),
quality_profile_version TEXT NOT NULL,
replaces_capture_id TEXT,
local_transfer_status TEXT NOT NULL CHECK (local_transfer_status IN ('PENDIENTE_LOCAL','TRANSFIRIENDO_LOCAL','RECI
    ↳ BIDA_CONTROLADOR','ERROR_LOCAL')),
remote_sync_status  TEXT CHECK (remote_sync_status IS NULL OR remote_sync_status IN ('PENDIENTE_NUBE','SUBIENDO'
    ↳ 'SINCRONIZADO','ERROR_SINCRONIZACION')),
transfer_attempts   INTEGER NOT NULL DEFAULT 0,
sync_attempts       INTEGER NOT NULL DEFAULT 0,
last_error          TEXT,
file_deleted_at     TEXT,
created_at          TEXT NOT NULL,
updated_at          TEXT NOT NULL,
CHECK (is_test = 1 OR (pass_id IS NOT NULL AND lateral_code IS NOT NULL))
);
CREATE TABLE IF NOT EXISTS quality_results (
  capture_id         TEXT PRIMARY KEY NOT NULL REFERENCES captures(capture_id),
  profile_version    TEXT NOT NULL,
  status             TEXT NOT NULL CHECK (status IN ('UTILIZABLE','REPETIR_NITIDEZ','REPETIR_EXPOSICION','ERROR_CAMARA
    ↳ 'PENDIENTE_REVISION_TECNICA')),
  reasons_json       TEXT NOT NULL DEFAULT '[]',
  luminance_mean     REAL,
  dark_ratio         REAL,
  bright_ratio       REAL,
  laplacian_variance REAL,
  analyzed_regions   INTEGER,
  duration_ms        INTEGER,
  created_at         TEXT NOT NULL
);
-- Cola de la cámara hacia el controlador.
CREATE TABLE IF NOT EXISTS transfer_queue (
  capture_id         TEXT PRIMARY KEY NOT NULL REFERENCES captures(capture_id),
  priority           INTEGER NOT NULL DEFAULT 0,          -- -1 = prueba corta (primero), 0 = útil, 1 = rechazada
  status             TEXT NOT NULL CHECK (status IN ('PENDIENTE','EN_CURSO','HECHO','ERROR')),
  attempts           INTEGER NOT NULL DEFAULT 0,
  checksum_failures INTEGER NOT NULL DEFAULT 0,          -- MD5_MISMATCH / SIZE_MISMATCH acumulados (classifyLocalTransf
    ↳ er)
  next_attempt_at    TEXT NOT NULL,
  last_error         TEXT,
  updated_at         TEXT NOT NULL
);
-- Cola del controlador hacia Spring Boot.
CREATE TABLE IF NOT EXISTS sync_queue (
  id                 INTEGER PRIMARY KEY AUTOINCREMENT,
  entity_type        TEXT NOT NULL CHECK (entity_type IN ('SESSION','PASS','SEQUENCE_BATCH','CAPTURE','INCIDENT_BATCH','S
    ↳ ESSION_CLOSE')),
  entity_id          TEXT NOT NULL,                      -- SESSION/SESSION_CLOSE: sessionId · PASS y SEQUENCE_BATCH: p
    ↳ assId · CAPTURE: captureId · INCIDENT_BATCH: sessionId
  session_id         TEXT NOT NULL,
  order_key          INTEGER NOT NULL,                   -- 10 sesión, 20 pasada, 30 secuencias, 40 capturas, 50 incide
    ↳ ncias, 60 cierre
  status             TEXT NOT NULL CHECK (status IN ('PENDIENTE','EN_CURSO','HECHO','ERROR_DEFINITIVO')),
  attempts           INTEGER NOT NULL DEFAULT 0,
  next_attempt_at    TEXT NOT NULL,
  last_error_code    TEXT,
  last_error         TEXT,
  created_at         TEXT NOT NULL,
  updated_at         TEXT NOT NULL,
  UNIQUE (entity_type, entity_id)
);
CREATE TABLE IF NOT EXISTS processed_messages (
  message_id         TEXT PRIMARY KEY NOT NULL,
  type               TEXT NOT NULL,
  received_at        TEXT NOT NULL,
  response_json      TEXT
);
CREATE TABLE IF NOT EXISTS incidents (
  incident_id        TEXT PRIMARY KEY NOT NULL,
  session_id         TEXT NOT NULL,
  pass_id            TEXT,
  sequence_id        TEXT,
  capture_id         TEXT,
  device_id          TEXT,
  type               TEXT NOT NULL CHECK (type IN ('CALIDAD','DESCONEXION','BATERIA','TEMPERATURA','ESPACIO','GPS','TR
    ↳ ANSFERENCIA','SINCRONIZACION','SOPORTE','OPERADOR','OTRO')),
  severity           TEXT NOT NULL CHECK (severity IN ('INFO','AVISO','ERROR')),
  detail             TEXT NOT NULL,
  occurred_at        TEXT NOT NULL,

```

```

    created_by      TEXT NOT NULL CHECK (created_by IN ('SISTEMA','OPERADOR')),
    remote_sync_status TEXT NOT NULL DEFAULT 'PENDIENTE_NUBE'
                      CHECK (remote_sync_status IN ('PENDIENTE_NUBE','SUBIENDO','SINCRONIZADO','ERROR_SINCRONIZACION'))
);
CREATE TABLE IF NOT EXISTS event_log (
  id      INTEGER PRIMARY KEY AUTOINCREMENT,
  ts      TEXT NOT NULL,
  level   TEXT NOT NULL CHECK (level IN ('DEBUG','INFO','WARN','ERROR')),
  category TEXT NOT NULL,
  event   TEXT NOT NULL,
  session_id TEXT,
  device_id TEXT,
  data_json TEXT
);
CREATE INDEX IF NOT EXISTS idx_rows_lot      ON cat_rows(row_id);
CREATE INDEX IF NOT EXISTS idx_segments_row  ON cat_segments(row_id);
CREATE INDEX IF NOT EXISTS idx_markers_row   ON cat_markers(row_id);
CREATE INDEX IF NOT EXISTS idx_passes_session ON monitoring_passes(session_id);
CREATE INDEX IF NOT EXISTS idx_sequences_pass ON capture_sequences(pass_id);
CREATE INDEX IF NOT EXISTS idx_marker_changes_pass ON marker_changes(pass_id);
CREATE INDEX IF NOT EXISTS idx_retakes_sequence ON retake_requests(sequence_id);
CREATE INDEX IF NOT EXISTS idx_captures_sequence ON captures(sequence_id);
CREATE INDEX IF NOT EXISTS idx_captures_session ON captures(session_id);
CREATE INDEX IF NOT EXISTS idx_captures_transfer ON captures(local_transfer_status);
CREATE INDEX IF NOT EXISTS idx_captures_sync ON captures(remote_sync_status);
CREATE INDEX IF NOT EXISTS idx_transfer_next ON transfer_queue(status, priority, next_attempt_at);
CREATE INDEX IF NOT EXISTS idx_sync_next ON sync_queue(status, order_key, next_attempt_at);
CREATE INDEX IF NOT EXISTS idx_incidents_session ON incidents(session_id);
CREATE INDEX IF NOT EXISTS idx_event_log_ts ON event_log(ts);
CREATE INDEX IF NOT EXISTS idx_processed_received ON processed_messages(received_at);

```

12.2 Migraciones — src/storage/db.ts

```

import type { SQLiteDatabase } from 'expo-sqlite';
import { MIGRATION_001 } from './migrations/001_initial';
/** Índice + 1 = versión del esquema. Nunca modificar una migración publicada: agregar 002, 003... */
const MIGRATIONS: readonly string[] = [MIGRATION_001];
export async function migrateDbIfNeeded(db: SQLiteDatabase): Promise<void> {
  await db.execAsync('PRAGMA journal_mode = WAL; PRAGMA foreign_keys = ON;');
  const row = await db.getFirstAsync<{ user_version: number }>('PRAGMA user_version');
  const current = row?.user_version ?? 0;
  for (let v = current; v < MIGRATIONS.length; v++) {
    await db.withExclusiveTransactionAsync(async (txn) => {
      await txn.execAsync(MIGRATIONS[v]);
      await txn.execAsync(`PRAGMA user_version = ${v + 1}`);
    });
  }
}
// app/_layout.tsx: <SQLiteProvider databaseName="riachuelo.db" onInit={migrateDbIfNeeded}> ... </SQLiteProvider>

```

12.3 SecureStore

Claves válidas: letras, números, ., - y _. Cada valor ≤ 2048 bytes (Verificado en expo-secure-store 15).

Clave	Contenido
device.id	deviceId (también en app_meta.device_id).
auth.userId	Usuario con sesión iniciada.
auth.accessToken / auth.accessTokenExpiresAt	JWT de acceso y su vencimiento.
auth.refreshToken / auth.refreshTokenExpiresAt	Token de renovación y su vencimiento.
auth.offlineVerifier	JSON del verificador (sección 7.5).
auth.mode	ONLINE u OFFLINE (sección 7.11).
pairing.token	Token del QR de la sesión abierta: en el controlador, el que muestra; en la cámara, el de la sesión a la que está unida. Se borra al cerrar la sesión (controlador) o el contexto (cámara).

Si el backend emite tokens de más de 2 KB, la app falla al guardarlos: el backend debe mantener el JWT por debajo de ese tamaño (pregunta Q-06).

12.4 Archivos

Ruta (Paths.document = directorio privado de la app)	Contenido
<code>captures/{sessionId}/{passId}/{captureId}.jpg</code>	Fotos definitivas (originales en la cámara; copias recibidas en el controlador).
<code>incoming/{captureId}.part</code>	Recepción en curso en el controlador; se renombra al verificar.
<code>test/{fecha}/{uuid}.jpg</code>	Fotos del modo prueba de cámara (PANT-32).
<code>short-test/{sessionId}/{captureId}.jpg</code>	Fotos de la prueba corta, en la cámara y en el controlador; se borran al cerrar la sesión o el contexto (RN-21).
<code>diagnostics/diagnostico_{deviceId}_{fecha}.json</code>	Exportaciones de diagnóstico.
<code>Paths.cache/quality/</code>	Copias reducidas y recortes temporales de calidad (se borran al terminar).

Reglas:

- La foto de `takePictureAsync` queda en caché: se **mueve** a `captures/...` con `new File(uri).move(destino)` antes de registrar la captura.
- El nombre de archivo es el `captureId`, nunca la hora.
- La foto original nunca se recomprime ni se reduce.
- El md5 se obtiene con `file.md5` de `expo-file-system` y viaja con la foto.
- Al arrancar, un archivo en `captures/` sin fila en `captures` se registra en `event_log` y se conserva (no se borra).

12.5 Reglas de escritura

- Toda escritura que toque varias tablas usa `withExclusiveTransactionAsync` (R-14).
- Toda fila con `updated_at` lo actualiza en cada cambio.
- Las filas de evidencia (sesiones, pasadas, secuencias, repeticiones, cambios de marcador, capturas, resultados de calidad, incidencias) nunca se eliminan; la liberación de espacio borra solo archivos y marca `file_deleted_at`. Únicas excepciones: un borrador DRAFT descartado (no tiene datos dependientes) y los archivos de prueba corta (sus filas se conservan con `file_deleted_at`).
- El cierre de una sesión (CLOSING → CLOSED) y la creación de sus elementos de `sync_queue` ocurren en la misma transacción.
- `event_log` se purga después de `logging.eventLogMaxDays`; `processed_messages`, después de `protocol.processedMessagesTtlHours`.
- Catálogos: el bootstrap reemplaza las cuatro tablas dentro de una transacción (borrar e insertar) y luego actualiza `app_meta.catalog_version` y `app_meta.bootstrap_at`.

13. Estados: eventos y condiciones

Transición	Evento	Condición (guarda)	Efecto
Sesión			
(nueva) → DRAFT	«Crear sesión» (PANT-12)	Catálogos descargados; batería y espacio del controlador sobre los mínimos de inicio; ninguna sesión abierta (7.12)	Un borrador anterior se descarta.
DRAFT → (eliminada)	«Descartar borrador»	—	Se borra la fila (no tiene datos dependientes).
DRAFT → PREPARING	Se abre PANT-13	—	Token nuevo (<code>pairing.token</code> y su hash); se inician el servidor de control y el receptor de fotos.
PREPARING → READY	Prueba corta aprobada (14.12)	Ambas cámaras CONECTADA	<code>short_test_passed_at</code> ; se habilita «Nueva pasada».

Transición	Evento	Condición (guarda)	Efecto
READY → PREPARING	«Liberar CÁMARA X» (PANT-13)	Sin pasada abierta; esa cámara PERDIDA o DESCONECTADA	released = 1; se borra short_test_passed_at; exige nueva prueba corta.
READY → ACTIVE	La pasada pasa a ACTIVE	RN-10 (inicio) y RN-12	—
ACTIVE → PAUSED	Pausa	Manual; enlace PERDIDA (incluye socket cerrado); batería < pauseBatteryPct o espacio < pauseFreeSpaceBytes en cualquier celular; controlador o cámara fuera de primer plano	PAUSE con su motivo; incidencia si es automática.
PAUSED → ACTIVE	«Reanudar»	RN-10 (reanudación): ambas cámaras CONECTADA, ningún RESYNC en curso, batería y espacio sobre los umbrales de pausa	RESUME.
ACTIVE / PAUSED → READY	Pasada cerrada	—	END_PASS.
PREPARING / READY / ACTIVE / PAUSED → CLOSING	«Cerrar sesión» (PANT-19)	—	No se emiten órdenes; una pasada abierta se cierra INCOMPLETE con incidencia.
CLOSING → CLOSED	Fin del cierre	Sin fotos por llegar, o el operador eligió «Cerrar igual» (incidencia TRANSFERENCIA)	Se crean los elementos de sync_queue (15.5); SESSION_CLOSED a las cámaras conectadas; los servidores se detienen al recibir sus ACK o agotar los reenvíos; se borran los archivos de prueba corta.
CLOSED → SYNCED	Sincronización	Todos sus elementos de sync_queue en HECHO	—
SYNCED → CLOSED	Llega una foto tardía (8.13)	—	Se encolan la captura, la pasada y el lote de secuencias afectados.
Pasada			
(nueva) → READY	«Iniciar pasada» o «Repetir lateral» (PANT-15)	RN-10 (inicio), RN-12	passOrder = máximo de la hilera en la sesión + 1; START_PASS a las cámaras.
READY → ACTIVE	ACK de START_PASS de ambas cámaras	—	Sesión ACTIVE; SESSION_CONTEXT a las cámaras.
ACTIVE ↔ PAUSED	Pausa / reanudar	Igual que la sesión	—
ACTIVE / PAUSED → COMPLETED / INCOMPLETE	«Cerrar pasada» (PANT-18)	Ninguna orden en curso; INCOMPLETE exige incidencia	Contadores finales; END_PASS.
READY → INCOMPLETE	Cancelar antes de empezar (p. ej., sin ACK de START_PASS)	Incidencia	—
Secuencia			
(nueva) → CREATED	Disparo (botón o temporizador)	RN-11; pasada ACTIVE; ambas cámaras CONECTADA	Número siguiente, dos captureId, contexto y GPS.
CREATED → COMMAND_SENT	Órdenes enviadas	—	Espera captureResponseTimeoutMs.
CREATED → CANCELLED	No se pudo enviar	—	Incidencia.
COMMAND_SENT / PARTIAL / INCOMPLETE → ...	Cambia un resultado por cámara	—	Estado = deriveSequenceStatus(slots).
Resultado por cámara			
PENDIENTE → OK_PENDIENTE_ARCHIVO	CAPTURE_OK	—	—
PENDIENTE → OK_RECIBIDA	La foto aceptada llega antes que su CAPTURE_OK	—	ACK FILE.
OK_PENDIENTE_ARCHIVO → OK_RECIBIDA	Foto recibida y verificada	—	ACK FILE.

Transición	Evento	Condición (guarda)	Efecto
PENDIENTE → RECHAZADA_CALIDAD / ERROR_CAMARA	QUALITY_ERROR, o ERROR INTERNAL en respuesta a la orden (14.4)	—	Opción «Repetir CÁMARA X».
PENDIENTE → SIN_RESPUESTA	Vence la espera, o ERROR COMMAND_EXPIRED, WRONG_PASS o BUSY	—	Incidencia; opción «Repetir CÁMARA X».
SIN_RESPUESTA → OK_PENDIENTE_ARCHIVO / OK_RECIBIDA / RECHAZADA_CALIDAD / ERROR_CAMARA	Respuesta tardía, foto recibida o RESYNC	—	Se recalcula la secuencia.
RECHAZADA_CALIDAD / ERROR_CAMARA / SIN_RESPUESTA → PENDIENTE	«Repetir CÁMARA X»	RN-07; RN-11; pasada ACTIVE; esa cámara CONECTADA	Fila en retake_requests; nuevo captureId en ese rol (applyRetake).
Transferencia			
PENDIENTE_LOCAL → TRANSFIRIENDO_LOCAL	Inicio de envío	Enlace CONECTADA con el controlador de la sesión de la foto (RN-22); un envío a la vez por cámara	—
TRANSFIRIENDO_LOCAL → RECIBIDA_CONTROLADOR	classifyLocalTransfer = RECIBIDA (respuesta HTTP o ACK FILE)	—	—
TRANSFIRIENDO_LOCAL → PENDIENTE_LOCAL	classifyLocalTransfer = REINTENTAR: red, tiempo agotado, NO_SPACE, INTERNAL, o md5 o tamaño distinto hasta transfer.maxChecksumRetries	—	attempts + 1; espera transfer.retryDelaysMs.
TRANSFIRIENDO_LOCAL → ERROR_LOCAL	classifyLocalTransfer = ERROR_LOCAL (tabla de 14.8)	—	event_log TRANSFER/ERROR en la cámara; la foto se conserva.
ERROR_LOCAL → PENDIENTE_LOCAL	«Reintentar transferencias con error» (PANT-31 sin pasada activa, o PANT-09)	—	attempts = 0 y checksum_failures = 0; event_log MANUAL_RETRY.
Sincronización			
PENDIENTE_NUBE → SUBIENDO → SINCRONIZADO	Envío confirmado por el backend	—	—
SUBIENDO → PENDIENTE_NUBE	REINTENTAR o RENOVAR_TOKEN	—	Espera sync.retryDelaysMs.
SUBIENDO → ERROR_SINCRONIZACION	DEFINITIVO	—	Visible en PANT-20.
ERROR_SINCRONIZACION → PENDIENTE_NUBE	«Reintentar errores» (PANT-20)	—	—
SINCRONIZADO → PENDIENTE_NUBE	Foto tardía (8.13)	Solo la pasada y las secuencias de esa pasada	Se recalculan los contadores de la pasada; la sesión conserva su remote_sync_status.
Enlace de cámara			
CONECTADA → INESTABLE	Sin heartbeat > unstableAfterMs	—	Aviso en la tarjeta de la cámara.
INESTABLE → CONECTADA	Vuelve el heartbeat	—	—
CONECTADA / INESTABLE → PERDIDA	Sin heartbeat > lostAfterMs, socket cerrado, o sin ACK tras ackMaxRetries reenvíos	—	Con pasada ACTIVE: pausa; incidencia DESCONEXION.
DESCONECTADA / PERDIDA → EMPAREJANDO → CONECTADA	PAIR_REQUEST y PAIRED	Validaciones de 14.6	SESSION_CONTEXT; RESYNC_REQUEST si la sesión ya tuvo pasadas.
CONECTADA / INESTABLE / PERDIDA → DESCONECTADA	El controlador restaura la sesión al abrir (8.10)	—	Espera el PAIR_REQUEST de cada cámara.
Usuario			
SIN_SESION → AUTENTICADO / CAMBIO_CONTRASENA_REQUERIDO	Login correcto, con o sin internet	RN-02, RN-03, RN-04	Modo ONLINE u OFFLINE.

Transición	Evento	Condición (guarda)	Efecto
CAMBIO_CONTRASENA_REQUERIDO → AUTENTICADO	Contraseña cambiada (PANT-06)	Internet	Se crea el verificador.
AUTENTICADO (OFFLINE) → AUTENTICADO (ONLINE)	Contraseña validada con internet (7.5)	Mismo usuario	Tokens y verificador nuevos.
AUTENTICADO → SIN_SESION	«Cerrar sesión»	RN-19	Se borran los tokens; el verificador se conserva.
AUTENTICADO / CAMBIO_CONTRASENA_REQUERIDO → SIN_SESION	Revocación (7.8)	Ninguna (no aplica RN-19)	Se borran tokens y verificador; los datos de campo se conservan.
(al abrir la app) → SIN_SESION	Sesión OFFLINE que ya no cumple RN-03 (7.11)	—	PANT-02.

14. Protocolo local v1

14.1 Canales y puertos

Canal	Dirección	Uso
WebSocket ws://{host}:{controlPort}/control (8765)	Cámaras → servidor del controlador	Mensajes JSON del protocolo (texto, ≤ 64 KB).
HTTP http://{host}:{filePort}/local/captures (8766), opción A	Cámaras → receptor del controlador	Fotos (cuerpo binario + metadatos en cabecera).
HTTP en la cámara, opción B	Controlador → servidor de archivos de la cámara	Descarga de fotos.

Puertos en `pairing.controlPort` y `pairing.filePort`. Los tres celulares en la misma red Wi-Fi local (router portátil recomendado; hotspot del controlador como alternativa). Cámaras en modo avión con Wi-Fi activo para que Android no desvíe el tráfico a datos móviles.

14.2 QR de emparejamiento

```
{
  "type": "RIACHUELO_PAIR",
  "protocolVersion": 1,
  "host": "192.168.8.101",
  "controlPort": 8765,
  "filePort": 8766,
  "sessionId": "5b0f7a52-1d3c-4f0e-9a51-3f2b8c7d9e10",
  "pairingToken": "q3Jx9mVt0b2L8wZr4nKp1A",
  "controllerAppVersion": "0.2.0"
}
```

- `host` sale de `getIpAddressAsync()` de `expo-network`. Si devuelve vacío o `0.0.0.0` (por ejemplo, con el hotspot del controlador), la pantalla permite escribir la IP manualmente (pregunta Q-04).
- `pairingToken`: 16 bytes de `Crypto.getRandomBytes` en `base64url`. El controlador y la cámara lo guardan en `SecureStore` (`pairing.token`) para reconectar; en `SQLite` solo va su SHA-256 (`pairing_token_hash`). No vence por tiempo: vale hasta que la sesión se cierra o se regenera el QR. «Regenerar QR» invalida el anterior: las cámaras conectadas siguen conectadas, pero si se desconectan deben escanear el QR nuevo.

14.3 Sobre y reglas comunes

- Todo mensaje es un `Envelope JSON` en un frame de texto (tipos en 14.5) y se valida con `zod` antes de procesarse.
- `messageId` es único; un reenvío conserva el mismo `messageId`. El receptor guarda los procesados en `processed_messages`; ante un duplicado no vuelve a procesar y reenvía la respuesta guardada.
- `sessionId` debe ser la sesión abierta del receptor (si no: `ERROR_UNKNOWN_SESSION`), salvo en `PAIR_REQUEST`, que sigue 14.6. Salvo `PAIR_REQUEST`, el socket debe estar emparejado (si no: `ERROR_NOT_PAIED`). Las fotos de sesiones anteriores viajan por HTTP y siguen 8.13.

- **Confirmación:** PAIRED, SESSION_CONTEXT, START_PASS, END_PASS, PAUSE, RESUME y SESSION_CLOSED exigen un ACK MESSAGE con su messageId en refMessageId. Sin ACK en protocol.ackTimeoutMs, se reenvía con el mismo messageId hasta protocol.ackMaxRetries veces; luego el enlace pasa a PERDIDA. Los demás mensajes no llevan ACK: CAPTURE_COMMAND se responde con CAPTURE_OK, QUALITY_ERROR o ERROR, y si no hay respuesta el resultado queda en SIN_RESPUESTA.
- v debe ser 1 (si no: PAIR_REJECTED PROTOCOL_MISMATCH o ERROR_INVALID_MESSAGE).
- sentAt usa el reloj del emisor; la cámara compara vencimientos con el reloj del controlador estimado (serverTime de PAIRED y heartbeats).

14.4 Catálogo de mensajes

Tipo	Dirección	Cuándo	Respuesta esperada
PAIR_REQUEST	Cámara → controlador	Primer mensaje tras conectar (también al reconectar)	PAIRED o PAIR_REJECTED
PAIRED	Controlador → cámara	Emparejamiento aceptado; incluye configuración y sesiones ya sincronizadas	ACK
PAIR_REJECTED	Controlador → cámara	Token inválido, sesión cerrada, rol ocupado, versión distinta, usuario no permitido	La cámara muestra el motivo y cierra el socket (con SESSION_CLOSED cierra además su contexto, 8.12)
SESSION_CONTEXT	Controlador → cámaras	Tras PAIRED, al iniciar pasada y al cambiar marcador	ACK
START_PASS / END_PASS	Controlador → cámaras	Inicio y cierre de pasada	ACK
CAPTURE_COMMAND	Controlador → cada cámara	Nueva secuencia, repetición o prueba corta (purpose)	CAPTURE_OK o QUALITY_ERROR (o ERROR)
CAPTURE_OK	Cámara → controlador	Foto guardada y aceptada	— (el ACK FILE llega con la foto)
QUALITY_ERROR	Cámara → controlador	Foto rechazada o fallo de cámara	—
PAUSE / RESUME	Controlador → cámaras	Pausa o reanudación	ACK
HEARTBEAT	Ambos	Cada heartbeatIntervalMs; la cámara responde con echoSentAt	HEARTBEAT
ACK	Ambos	Confirma un mensaje (MESSAGE) o una foto (FILE)	—
RESYNC_REQUEST / RESYNC_STATE	Controlador → cámara / respuesta	Tras una reconexión durante la sesión	RESYNC_STATE en una o más páginas (14.9)
SESSION_CLOSED	Controlador → cámaras	Sesión cerrada	ACK
ERROR	Ambos	Mensaje inválido o no aplicable	—

Códigos de ERROR y efecto:

Código	Lo envía	Cuándo	Efecto en quien lo recibe
UNKNOWN_SESSION	Controlador o cámara	El sessionId no es la sesión abierta del receptor	Cámara: cierra el socket y pide escanear el QR vigente. Controlador: marca esa cámara PERDIDA e incidencia.
NOT_PAIRED	Controlador	Llega un mensaje distinto de PAIR_REQUEST antes de PAIRED en ese socket	La cámara envía PAIR_REQUEST.
UNKNOWN_DEVICE	Controlador	El deviceId del sobre no es el emparejado en ese socket	El controlador cierra el socket; event_log.
INVALID_MESSAGE	Ambos	Falla la validación zod o v ≠ 1	No se reenvía el mismo mensaje; event_log (es un error de programación).
COMMAND_EXPIRED	Cámara	CAPTURE_COMMAND recibido después de expiresAt (reloj estimado del controlador)	Resultado SIN_RESPUESTA; incidencia con el desfase medido.

Código	Lo envía	Cuándo	Efecto en quien lo recibe
WRONG_PASS	Cámara	Orden SECUENCIA con un passId distinto de su pasada activa, u orden PRUEBA_CORTA con una pasada activa	Resultado SIN_RESPUESTA; el controlador reenvía SESSION_CONTEXT a esa cámara; incidencia.
BUSY	Cámara	Llega una orden mientras procesa otra	Resultado SIN_RESPUESTA (no ocurre si se cumple RN-11).
INTERNAL	Ambos	Excepción no controlada	En respuesta a CAPTURE_COMMAND: resultado ERROR_CAMARA. En otro caso: event_log (y en el controlador, incidencia).

14.5 Tipos del protocolo — src/protocol/messages.ts

```
// src/protocol/messages.ts — Protocolo local v1 (WebSocket, JSON). Contrato: no modificar sin actualizar el archivo m
    ↪ aestro.
import type {
    CameraRole,
    CaptureMode,
    Direction,
    ISODateString,
    LateralCode,
    Platform,
    QualityMetrics,
    QualityReason,
    QualityStatus,
    UserRole,
    UUID,
} from '../domain/types';
export const PROTOCOL_VERSION = 1 as const;
/** Contenido del código QR que muestra el controlador. */
export interface PairingQrPayload {
    type: 'RIACHUELO_PAIR';
    protocolVersion: typeof PROTOCOL_VERSION;
    host: string; // IP del controlador en la red local
    controlPort: number;
    filePort: number;
    sessionId: UUID;
    pairingToken: string; // base64url de 16 bytes aleatorios
    controllerAppVersion: string;
}
export interface Envelope<T extends MessageType = MessageType> {
    v: typeof PROTOCOL_VERSION;
    type: T;
    messageId: UUID; // único por mensaje; un reenvío conserva el mismo messageId
    sessionId: UUID;
    deviceId: UUID; // emisor
    sentAt: ISODateString; // reloj del emisor
    payload: PayloadMap[T];
}
export type PairRejectReason =
    | 'INVALID_TOKEN'
    | 'SESSION_CLOSED'
    | 'ROLE_TAKEN'
    | 'PROTOCOL_MISMATCH'
    | 'APP_VERSION_MISMATCH'
    | 'USER_NOT_ALLOWED';
export type ProtocolErrorCode =
    | 'UNKNOWN_SESSION'
    | 'UNKNOWN_DEVICE'
    | 'NOT_PAired'
    | 'INVALID_MESSAGE'
    | 'COMMAND_EXPIRED'
    | 'WRONG_PASS'
    | 'BUSY'
    | 'INTERNAL';
export interface CaptureContextConfig {
    configVersion: string;
    qualityProfileVersion: string;
    jpegQuality: number;
    shutterSound: boolean;
    quality: {
        timeoutMs: number;
        exposure: {
            analysisWidth: number;
            darkLevel: number;
            brightLevel: number;
            minMean: number;
            maxMean: number;
        };
    };
}
```

```

        maxDarkRatio: number;
        maxBrightRatio: number;
    };
    sharpness: { regionSize: number; regionCount: 1 | 2 | 3; minLaplacianVariance: number };
};
heartbeatIntervalMs: number;
transferIncludeRejected: boolean;
}
export interface PayloadMap {
    /** Cámara → controlador, primer mensaje tras conectar el WebSocket. */
    PAIR_REQUEST: {
        pairingToken: string;
        requestedRole: CameraRole;
        userId: UUID;
        userName: string;
        /** Roles y modo de acceso del usuario con sesión en la cámara (datos de su último login; RN-02). */
        userRoles: UserRole[];
        authMode: 'ONLINE' | 'OFFLINE';
        platform: Platform;
        model: string;
        osVersion: string;
        appVersion: string;
        protocolVersion: typeof PROTOCOL_VERSION;
        batteryLevel: number | null;
        freeSpaceBytes: number | null;
        pendingTransfers: number;
    };
    /** Controlador → cámara. */
    PAIRED: {
        role: CameraRole;
        controllerDeviceId: UUID;
        serverTime: ISODateString;
        config: CaptureContextConfig;
        /** Sesiones ya SINCRONIZADAS en el backend: la cámara puede liberar esas fotos (retención). */
        syncedSessionIds: UUID[];
    };
    PAIR_REJECTED: { reason: PairRejectReason; detail: string | null };
    /** Controlador → cámaras: contexto vigente (se reenvía al cambiar pasada o marcador). */
    SESSION_CONTEXT: {
        passId: UUID | null;
        lateralCode: LateralCode | null;
        lotId: string | null;
        rowId: string | null;
        segmentId: string | null;
        markerId: string | null;
        direction: Direction | null;
        mode: CaptureMode;
        intervalMs: number;
    };
    START_PASS: { passId: UUID; lateralCode: LateralCode; passOrder: number };
    END_PASS: { passId: UUID; status: 'COMPLETED' | 'INCOMPLETE' };
    CAPTURE_COMMAND: {
        /** PRUEBA_CORTA: captura de prueba sin pasada (passId y lateralCode nulos); no es evidencia ni se sincroniza. */
        purpose: 'SECUENCIA' | 'PRUEBA_CORTA';
        sequenceId: UUID; // en PRUEBA_CORTA es un identificador de la prueba (no existe en capture_sequences)
        sequenceNumber: number; // 0 en PRUEBA_CORTA
        captureId: UUID; // captureId esperado para ESTA cámara
        passId: UUID | null;
        lateralCode: LateralCode | null;
        mode: CaptureMode;
        issuedAt: ISODateString;
        expiresAt: ISODateString;
        replacesCaptureId: UUID | null; // presente en repeticiones
    };
    CAPTURE_OK: {
        sequenceId: UUID;
        captureId: UUID;
        capturedAt: ISODateString;
        width: number;
        height: number;
        sizeBytes: number;
        md5: string;
        qualityStatus: Extract<QualityStatus, 'UTILIZABLE' | 'PENDIENTE_REVISION_TECNICA'>;
        reasons: QualityReason[]; // vacío si UTILIZABLE; ['TIEMPO_AGOTADO'] si PENDIENTE_REVISION_TECNICA
        metrics: QualityMetrics | null;
        profileVersion: string;
        captureDurationMs: number;
    };
    QUALITY_ERROR: {
        sequenceId: UUID;
        captureId: UUID;
        capturedAt: ISODateString | null; // null solo si fileAvailable = false
        qualityStatus: Extract<QualityStatus, 'REPETIR_NITIDEZ' | 'REPETIR_EXPOSICION' | 'ERROR_CAMARA'>;
    };
};

```

```

        reasons: QualityReason[];
        metrics: QualityMetrics | null;
        profileVersion: string;
        fileAvailable: boolean; // false en ERROR_CAMARA sin archivo
    };
    PAUSE: { reason: 'OPERADOR' | 'ENLACE_PERDIDO' | 'BATERIA' | 'ESPACIO' | 'SEGUNDO_PLANO' | 'OTRO' };
    RESUME: Record<string, never>;
    /** Ambos sentidos. El controlador inicia; la cámara responde con echoSentAt. */
    HEARTBEAT: {
        echoSentAt: ISODateString | null;
        role: CameraRole | 'CONTROLADOR';
        batteryLevel: number | null;
        freeSpaceBytes: number | null;
        pendingTransfers: number;
        lastSequenceId: UUID | null;
        appState: 'active' | 'background' | 'inactive';
    };
    ACK: {
        ackType: 'MESSAGE' | 'FILE';
        refMessageId: UUID | null;
        captureId: UUID | null;
        result: 'RECEIVED' | 'ALREADY_RECEIVED' | 'REJECTED';
        reason: string | null;
    };
    RESYNC_REQUEST: { passId: UUID | null; lastSequenceIdKnown: UUID | null };
    /** Solo capturas de evidencia de la sesión (sin las de prueba corta) que aún no están RECIBIDA_CONTROLADOR, en pági
        ↪ nas de hasta protocol.resyncPageSize. */
    RESYNC_STATE: {
        lastSequenceIdSeen: UUID | null;
        page: number; // 1..totalPages
        totalPages: number; // 1 aunque no haya capturas pendientes
        captures: Array<{
            captureId: UUID;
            sequenceId: UUID;
            qualityStatus: QualityStatus;
            localTransferStatus: 'PENDIENTE_LOCAL' | 'TRANSFIRIENDO_LOCAL' | 'RECIBIDA_CONTROLADOR' | 'ERROR_LOCAL';
        }>;
    };
    SESSION_CLOSED: { sessionId: UUID };
    ERROR: { code: ProtocolErrorCode; detail: string | null; refMessageId: UUID | null };
}

export type MessageType = keyof PayloadMap;
export const MESSAGE_TYPES: readonly MessageType[] = [
    'PAIR_REQUEST',
    'PAIRED',
    'PAIR_REJECTED',
    'SESSION_CONTEXT',
    'START_PASS',
    'END_PASS',
    'CAPTURE_COMMAND',
    'CAPTURE_OK',
    'QUALITY_ERROR',
    'PAUSE',
    'RESUME',
    'HEARTBEAT',
    'ACK',
    'RESYNC_REQUEST',
    'RESYNC_STATE',
    'SESSION_CLOSED',
    'ERROR',
];

/** Validación mínima de forma del sobre (la validación completa por tipo se hace con esquemas zod). */
export function isEnvelope(value: unknown): value is Envelope {
    if (typeof value !== 'object' || value === null) return false;
    const v = value as Record<string, unknown>;
    return (
        v.v === PROTOCOL_VERSION &&
        typeof v.type === 'string' &&
        (MESSAGE_TYPES as readonly string[]).includes(v.type) &&
        typeof v.messageId === 'string' &&
        typeof v.sessionId === 'string' &&
        typeof v.deviceId === 'string' &&
        typeof v.sentAt === 'string' &&
        typeof v.payload === 'object' &&
        v.payload !== null
    );
}

/** Metadatos que acompañan a cada foto en la transferencia local (cabecera X-Capture-Meta, JSON en base64url). */
export interface LocalCaptureMeta {
    captureId: UUID;
    sequenceId: UUID;
    sessionId: UUID;
}
    
```

```

passId: UUID | null; // null solo si isTest
lateralCode: LateralCode | null; // null solo si isTest
isTest: boolean;
deviceId: UUID;
cameraRole: CameraRole;
userId: UUID;
capturedAt: ISODateString;
width: number;
height: number;
sizeBytes: number;
md5: string;
qualityStatus: QualityStatus;
qualityReasons: QualityReason[];
qualityMetrics: QualityMetrics | null;
profileVersion: string;
replacesCaptureId: UUID | null;
}
export interface LocalCaptureResponse {
  result: 'RECEIVED' | 'ALREADY_RECEIVED' | 'REJECTED';
  reason:
    | null
    | 'MD5_MISMATCH'
    | 'SIZE_MISMATCH'
    | 'UNKNOWN_SESSION'
    | 'UNKNOWN_SEQUENCE'
    | 'WRONG_DEVICE'
    | 'CAPTURE_CONFLICT'
    | 'NO_SPACE'
    | 'INVALID_META'
    | 'LENGTH_REQUIRED'
    | 'PAYLOAD_TOO_LARGE'
    | 'NOT_FOUND'
    | 'INTERNAL';
}

```

14.6 Emparejamiento paso a paso

1. El controlador genera el token, lo guarda en `pairing.token` y su hash en la sesión, inicia `ControlServer` y `FileReceiver`, obtiene la IP y muestra el QR (sesión DRAFT → PREPARING).
2. La cámara escanea, valida `type` y `protocolVersion`, conecta el `WebSocket` y envía `PAIR_REQUEST` con su función como `requestedRole`, su usuario (`userId`, `userName`, `userRoles`, `authMode`), versión, batería, espacio y pendientes.
3. El controlador valida en este orden y responde `PAIR_REJECTED` con el primer motivo que falle:
 - `protocolVersion` igual al suyo; si no, `PROTOCOL_MISMATCH`.
 - La sesión del sobre existe y el SHA-256 del token es igual a su `pairing_token_hash` (comparación en tiempo constante); si no, `INVALID_TOKEN`.
 - Sesión en `PREPARING`, `READY`, `ACTIVE`, `PAUSED` o `CLOSING` (en `CLOSING` solo para terminar transferencias: no se envían órdenes ni `SESSION_CONTEXT`); si está `CLOSED` o `SYNCED`, `SESSION_CLOSED`.
 - Misma versión de app si `pairing.requireSameAppVersion` (RN-17); si no, `APP_VERSION_MISMATCH`.
 - Al menos uno de `userRoles` en `MOBILE_ALLOWED_ROLES` (RN-02); si no, `USER_NOT_ALLOWED`.
 - Rol (RN-16): se acepta si ese `deviceId` tiene el rol pedido con `released = 0` (reconexión), o si ningún celular tiene ese rol con `released = 0` y ese `deviceId` no tiene el otro rol con `released = 0`; si no, `ROLE_TAKEN`.
4. Si es válido: guarda o actualiza `session_devices` (enlace CONECTADA, `released = 0`) y responde `PAIRED` con la configuración de captura y calidad (D-21) y `syncedSessionIds`; luego envía `SESSION_CONTEXT` (salvo en `CLOSING`). Si la sesión ya tuvo pasadas, envía además `RESYNC_REQUEST` (14.9).
5. La cámara cierra primero el contexto de otra sesión si lo tenía (8.12); después guarda o actualiza `camera_context` (con `controller_host`, `control_port` y `file_port`) y `pairing.token`, aplica la configuración, libera las fotos de sesiones sincronizadas (RN-09) y responde `ACK`.

14.7 Orden de captura paso a paso

1. **Controlador:** comprueba RN-11, pasada ACTIVE y ambas cámaras CONECTADA (la prueba corta sigue 14.12).
2. Crea la secuencia (CREATED): número siguiente, dos captureId, contexto vigente, GPS (`gpsService.latest()` con antigüedad $\leq$ `gps.maxAgeMs`), `issuedAt` y `expiresAt = issuedAt + commandValidityMs`; la guarda en una transacción.
3. Envía un CAPTURE_COMMAND (`purpose = SECUENCIA`) a cada cámara con su captureId → COMMAND_SENT; inicia la espera `captureResponseTimeoutMs`.
4. **Cámara:** valida sesión (UNKNOWN_SESSION), pasada (WRONG_PASS), vencimiento con el reloj estimado del controlador (COMMAND_EXPIRED) y que no esté procesando otra orden (BUSY); ante un messageId repetido reenvía la respuesta guardada.
5. Toma la foto (`takePictureAsync({ quality: jpegQuality, exif: false, shutterSound })`), la mueve a `captures/...`, obtiene md5 y tamaño, inserta la captura (columnas de captures según RN-05; CAPTURED, PENDIENTE_LOCAL) y ejecuta la calidad (sección 16).
6. Actualiza la captura y `quality_results`, responde CAPTURE_OK (UTILIZABLE o PENDIENTE_REVISION_TECNICA, con reasons) o QUALITY_ERROR, y encola la transferencia (las rechazadas con prioridad 1 si `includeRejected`).
7. **Controlador:** actualiza el resultado de esa cámara y recalcula el estado de la secuencia. Crea o actualiza la fila de la captura con los metadatos recibidos (`local_transfer_status = PENDIENTE_LOCAL`) solo si espera la foto: CAPTURE_OK, o QUALITY_ERROR con `fileAvailable = true` y `transfer.includeRejected = true`. Un QUALITY_ERROR sin archivo solo actualiza el resultado.
8. Al recibir y verificar la foto: captura RECIBIDA_CONTROLADOR (y `remote_sync_status = PENDIENTE_NUBE` si no es de prueba), resultado OK_RECIBIDA si era una foto aceptada, ACK FILE a la cámara.
9. **Repetición:** igual que los pasos 3 a 8, pero con un solo CAPTURE_COMMAND a la cámara afectada, el mismo sequenceId y sequenceNumber, un captureId nuevo y `replacesCaptureId`. La fila de `retake_requests` se guarda antes de enviar la orden.

14.8 Transferencia de fotos

Opción A (candidata): la cámara sube al controlador.

- Cámara: `createUploadTask(url, fileUri, { httpMethod: 'POST', uploadType: FileSystemUploadType.BINARY_CONTENT, headers })` de `expo-file-system/legacy`; `uploadAsync()` en la tarea y `cancelAsync()` si se supera `transfer.requestTimeoutMs`. El archivo se envía de forma nativa (no pasa por la memoria de JavaScript).
- Cabeceras: `Content-Type: image/jpeg`, `X-Capture-Meta: <LocalCaptureMeta en JSON, base64url>`, `X-Device-Id`.
- Controlador: receptor `HttpUploadConnection` (Anexo A) sobre `react-native-tcp-socket`. El sink abre `incoming/{captureId}.part` con `file.open()` y escribe cada bloque con `FileHandle.writeBytes`; al terminar verifica tamaño y md5, mueve el archivo a `captures/...`, registra la captura en una transacción y responde `LocalCaptureResponse`.
- Controles: el deviceId de los metadatos debe estar registrado en `session_devices` de la sesión de la foto (aunque el rol se haya liberado o la sesión esté cerrada, 8.13) y la IP de origen debe coincidir con la del WebSocket abierto de ese deviceId; el captureId debe haber sido emitido por el controlador para esa cámara (en una secuencia, en `retake_requests` o en la prueba pendiente de `app_meta.short_test_pending`; una foto de prueba de otro intento se responde RECEIVED y se descarta). Un captureId ya recibido con el mismo md5 se responde ALREADY_RECEIVED **después de descartar el cuerpo** (seguro para cualquier cliente HTTP); con otro md5, CAPTURE_CONFLICT.
- Respuestas (`LocalCaptureResponse`) y reacción de la cámara (`classifyLocalTransfer`, Anexo C):

HTTP	result / reason	Cuándo	La cámara
200	RECEIVED	Foto guardada y verificada	RECIBIDA_CONTROLADOR
200	ALREADY_RECEIVED	Ya estaba (mismo md5)	RECIBIDA_CONTROLADOR
400	REJECTED / INVALID_META	Cabecera ausente o inválida	ERROR_LOCAL
403	REJECTED / WRONG_DEVICE	Dispositivo no registrado en la sesión o IP distinta	ERROR_LOCAL
404	REJECTED / NOT_FOUND	Ruta distinta de /local/captures	ERROR_LOCAL
404	REJECTED / UNKNOWN_SESSION o UNKNOWN_SEQUENCE	Sesión desconocida, o captureId no emitido para esa cámara	ERROR_LOCAL
409	REJECTED / CAPTURE_CONFLICT	Mismo captureId con otro md5	ERROR_LOCAL
411	REJECTED / LENGTH_REQUIRED	Sin Content-Length o con chunked	ERROR_LOCAL
413	REJECTED / PAYLOAD_TOO_LARGE	Cuerpo mayor que el límite	ERROR_LOCAL
422	REJECTED / MD5_MISMATCH o SIZE_MISMATCH	El archivo llegó distinto	REINTENTAR hasta transfer.maxChecksumRetries; luego ERROR_LOCAL
500	REJECTED / INTERNAL	Error del receptor	REINTENTAR
507	REJECTED / NO_SPACE	Sin espacio en el controlador	REINTENTAR (con aviso de espacio)
—	(sin respuesta)	Red cortada o tiempo agotado	REINTENTAR

- Concurrencia: una subida a la vez por cámara (transfer.maxConcurrentPerCamera).

Opción B (alternativa): el controlador descarga de la cámara. Cada cámara sirve su carpeta captures/ en solo lectura con @dr.pogodin/react-native-static-server; tras CAPTURE_OK, el controlador descarga con File.downloadFileAsync(url, destino) usando la IP de la conexión WebSocket de esa cámara, verifica md5 y tamaño, registra y envía ACK FILE.

Decisión en la Fase 2 (ADR): se elige la opción que pase corte y reconexión sin duplicados, con menor complejidad y funcionamiento en ambos sistemas. Mientras tanto, FileSender/FileReceiver aíslan la decisión.

14.9 Heartbeat, desconexión, reconexión y RESYNC

- El controlador envía HEARTBEAT a cada cámara cada heartbeatIntervalMs (2 s); la cámara responde con echoSentAt y su estado (batería, espacio, pendientes, última secuencia, estado de la app).
- Con cada respuesta el controlador calcula el RTT y el desfase de reloj de esa cámara ($\text{desfase} = \text{sentAt}_{\text{cámara}} - (\text{echoSentAt} + \text{RTT}/2)$) y guarda la mediana de las últimas protocol.clockOffsetSamples (5) mediciones en session_devices.clock_offset_ms.
- Sin respuesta > unstableAfterMs (3 s) → INESTABLE; > lostAfterMs (6 s) → PERDIDA. El cierre o error del socket de una cámara la marca PERDIDA de inmediato. Con una pasada ACTIVE, PERDIDA pausa la pasada (PAUSE con motivo ENLACE_PERDIDO a la otra cámara) y registra una incidencia DESCONEXION.
- La cámara, si no recibe heartbeats por lostAfterMs o se cierra el socket, reintentará con las esperas protocol.reconnectDelaysMs (1, 2, 4 y 8 s, luego cada 8 s), enviando de nuevo PAIR_REQUEST con el token de pairing.token. Tras un PAIR_REJECTED deja de reintentar, muestra el motivo (PAIR_REJECT_MESSAGES) y ofrece «Reescanear QR».
- **RESYNC:** tras PAIRED de una cámara que vuelve, el controlador envía RESYNC_REQUEST. La cámara responde con una o más páginas RESYNC_STATE (page de totalPages, hasta protocol.resyncPageSize capturas por página) que contienen solo sus capturas de evidencia de esa sesión (sin las de prueba corta) que aún no están RECIBIDA_CONTROLADOR. Si el resultado de esa cámara para una captura emitida por el controlador está en PENDIENTE o SIN_RESPUESTA, pasa a OK_PENDIENTE_ARCHIVO (foto aceptada) o a RECHAZADA_CALIDAD / ERROR_CAMARA (rechazada). Un captureId que el controlador no emitió se registra como incidencia. El RESYNC termina al procesar la última página; recién entonces se puede reanudar. Las fotos llegan luego por la cola de transferencia.
- **Primer plano:** si la app del controlador sale de primer plano, pausa la pasada (PAUSE con motivo SEGUNDO_PLANO). Si una cámara informa appState ≠ active en su heartbeat, el controlador también pausa

con ese motivo e incidencia (mensaje PAUSA_SEGUNDO_PLANO). Al volver, se concilia antes de reanudar. En iPhone, el servidor deja de aceptar conexiones en segundo plano.

14.10 Temporizadores (valores en src/config, sección 17)

Parámetro	Valor inicial	Uso
protocol.heartbeatIntervalMs	2 000 ms	Frecuencia del heartbeat.
protocol.unstableAfterMs / lostAfterMs	3 000 / 6 000 ms	Estados INESTABLE y PERDIDA.
protocol.ackTimeoutMs / ackMaxRetries	2 000 ms / 3	Espera de ACK y reenvíos con el mismo messageId.
protocol.commandValidityMs	3 000 ms	Vigencia de CAPTURE_COMMAND.
capture.captureBudgetMs	2 000 ms	Tiempo previsto para tomar, mover y calcular el md5 de la foto.
quality.timeoutMs	4 000 ms	Límite de la evaluación de calidad (después: PENDIENTE_REVISION_TECNICA).
protocol.captureResponseTimeoutMs	7 000 ms	Espera de CAPTURE_OK/QUALITY_ERROR. Regla: $\geq$ captureBudgetMs + quality.timeoutMs + 500 ms (probada en el Anexo E).
shortTest.timeoutMs	15 000 ms	Límite de la prueba corta por cámara (14.12).
protocol.resyncPageSize	200 capturas	Tamaño de página de RESYNC_STATE (cabe en 64 KB; probado).
protocol.clockOffsetSamples	5	Mediciones para la mediana del desfase.
protocol.reconnectDelaysMs	1, 2, 4, 8 s	Reconexión de la cámara.
transfer.retryDelaysMs / retryMaxDelayMs	2, 4, 8, 16 s / 30 s	Reintentos de transferencia local.
transfer.maxChecksumRetries	3	md5 o tamaño distinto antes de ERROR_LOCAL.
sync.retryDelaysMs	1, 5, 15, 60 min	Reintentos de sincronización.

14.11 Implementación del servidor de control

- `TcpSocket.createServer((socket) => ...).listen({ port, host: '0.0.0.0', reuseAddress: true })` de `react-native-tcp-socket` (API verificada en 6.4.3).
- Por cada conexión se crea un `WsServerConnection` (Anexo A) con: `write = (bytes) => socket.write(bytes), end = () => socket.destroy()` y `sha1Base64 = (s) => Crypto.digestStringAsync(Crypto.CryptoDigestAlgorithm.SHA1, s, { encoding: Crypto.CryptoEncoding.BASE64 })`.
- `socket.on('data', (chunk) => conn.onData(chunk))`: el Buffer de la librería es un `Uint8Array` y se pasa directo. `socket.on('close' | 'error', ...)` → par desconectado.
- Las cámaras usan el `WebSocket` global de React Native (`new WebSocket(url)`, `onmessage` con texto).
- El servidor se detiene al cerrar la sesión de monitoreo. Funciona solo con la app en primer plano.

14.12 Prueba corta (contrato)

1. Requisitos: sesión PREPARING y ambas cámaras CONECTADA.
2. El controlador genera un `testId` (UUID, que viaja como `sequenceId`) y un `captureId` por cámara, guarda `{sessionId, testId, captureIds, issuedAt}` en `app_meta.short_test_pending` y envía a ambas un `CAPTURE_COMMAND` con `purpose = PRUEBA_CORTA`, `sequenceNumber = 0`, `passId = null`, `lateralCode = null` y `replacesCaptureId = null`. No crea fila en `capture_sequences`.
3. Cada cámara captura y evalúa igual que en 14.7, guarda la foto en `short-test/{sessionId}/{captureId}.jpg` y la captura con `isTest = true`, responde `CAPTURE_OK` o `QUALITY_ERROR` y la transfiere siempre (aunque sea rechazada) con prioridad -1 (primero en la cola) e `isTest = true` en `LocalCaptureMeta`.

4. **Aprobación por cámara:** respondió `CAPTURE_OK`, o `QUALITY_ERROR` con `REPETIR_NITIDEZ` o `REPETIR_EXPOSICION`, dentro de `captureResponseTimeoutMs`, y su foto llegó con md5 correcto (`RECEIVED`) dentro de `shortTest.timeoutMs` desde la orden. `ERROR_CAMARA` (con o sin archivo), un `ERROR` o un tiempo vencido la reprobaban (`PRUEBA_CORTA_FALLIDA`). Un `QUALITY_ERROR` con `REPETIR_*` aprueba la comunicación, pero `PANT-14` muestra `PRUEBA_CORTA_CALIDAD`.
5. Con ambas aprobadas: `short_test_passed_at` = ahora, sesión `READY`, se borra `app_meta.short_test_pending` y se registra el evento `SESSION/SHORT_TEST` con resultados, tiempos y desfase entre los `capturedAt` corregidos. Si alguna falla: evento con el motivo y opción «Repetir» (un intento nuevo reemplaza `short_test_pending`).
6. Las capturas de prueba no cambian secuencias ni contadores, no aparecen en `RESYNC_STATE` y nunca entran en `sync_queue` (RN-21). Una foto de prueba que no es de la prueba pendiente (de un intento anterior o de una sesión cerrada) se responde `RECEIVED` y se descarta.
7. Al liberar un rol y vincular otro celular, se borra `short_test_passed_at` y se repite la prueba con ambas cámaras.

15. Contrato con el backend (/api/v1)

15.1 Reglas generales

- URL base: `${EXP0_PUBLIC_API_URL}/api/v1`. JSON en camelCase, fechas ISO-8601 UTC.
- Autenticación: `Authorization: Bearer <accessToken>` en todo salvo `register`, `login`, `refresh`, `password-reset-requests` y `health`. Todas las llamadas llevan `X-Device-Id`.
- Errores con cuerpo `ApiErrorBody` (`code`, `message`, `fieldErrors`, `traceId`).
- Idempotencia por ID en sesiones, pasadas, secuencias, capturas e incidencias: repetir no duplica y responde lo mismo.
- HTTPS en entornos remotos; HTTP solo en desarrollo dentro de la red local.
- Endpoints marcados **Nuevo** no estaban en la Especificación v2.0: se acuerdan con el responsable del backend y se documentan en OpenAPI.

15.2 Endpoints

Método y ruta	Auth	Uso	Éxito	Errores principales	Origen
GET /health	No	Comprobar que el backend responde	200 HealthResponse	—	Nuevo
POST /auth/register	No	Registro	201 RegisterResponse	400 VALIDATION_ERROR; 409 EMAIL_ALREADY_REGISTERED	Nuevo
POST /auth/login	No	Login con datos del dispositivo	200 LoginResponse	401 INVALID_CREDENTIALS; 403 ACCOUNT_PENDING, ACCOUNT_REJECTED, ACCOUNT_BLOCKED, ROLE_NOT_ALLOWED, DEVICE_REVOKED; 429 TOO_MANY_ATTEMPTS	Esp. v2.0 §22.1
POST /auth/refresh	No	Renovar tokens (con rotación)	200 RefreshResponse	401 REFRESH_INVALID; 403 ACCOUNT_BLOCKED, DEVICE_REVOKED	Nuevo
POST /auth/logout	No (usa el refreshToken del cuerpo)	Revocar el refresh del dispositivo	204	—	Nuevo
GET /auth/me	Sí	Perfil actual	200 UserProfile	401	Nuevo
POST /auth/change-password	Sí	Cambiar contraseña	204	400 PASSWORD_POLICY, 401 INVALID_CREDENTIALS	Nuevo
POST /auth/password-reset-requests	No	Pedir restablecimiento al administrador	202 (siempre)	—	Nuevo
GET /mobile/bootstrap	Sí	Catálogos y parámetros	200 BootstrapResponse	401	Esp. v2.0 §22.1

Método y ruta	Auth	Uso	Éxito	Errores principales	Origen
POST /sessions	Sí	Crear o actualizar sesión (idempotente por sessionId)	200/201	400, 401	Esp. v2.0 \$22.1
POST /sessions/{sessionId}/passes	Sí	Crear o actualizar pasada (idempotente por passId)	200/201	404 SESSION_NOT_FOUND	Esp. v2.0 \$22.1
POST /sessions/{sessionId}/sequences/batch	Sí	Secuencias (incluidas incompletas), hasta sync.batchSize (200) por petición	200 BatchResponse	404 SESSION_NOT_FOUND, PASS_NOT_FOUND	Nuevo
POST /captures/upload	Sí	Multipart: file (JPEG) + metadata (JSON); idempotente por captureId	201 o 200 CaptureUploadResponse (duplicate: true)	404 SEQUENCE_NOT_FOUND; 409 CAPTURE_CONFLICT; 413 PAYLOAD_TOO_LARGE	Esp. v2.0 \$22.1
POST /sessions/{sessionId}/incidents/batch	Sí	Incidencias, hasta sync.batchSize (200) por petición	200 BatchResponse	404 SESSION_NOT_FOUND	Nuevo
GET /captures/{captureId}	Sí	Consultar estado de una captura (diagnóstico)	200	404	Esp. v2.0 \$22.1

Requisitos para el backend que salen de esta app: permitir sesiones simultáneas del mismo usuario en varios dispositivos; aceptar que una pasada o una secuencia ya enviada se actualice (fotos tardías, 8.13); refreshToken de al menos 14 días con rotación; JWT ≤ 2 KB; endpoint para que el administrador apruebe cuentas, asigne roles, asigne contraseñas temporales (marcando mustChangePassword y revocando refresh) y revoque dispositivos.

15.3 DTO — src/api/dto.ts

```
// src/api/dto.ts – Contrato esperado con Spring Boot /api/v1 (versión 1, a acordar con backend).
import type {
  AccountStatus,
  CameraRole,
  CaptureMode,
  Direction,
  FieldRow,
  ISODateString,
  LateralCode,
  Lot,
  Marker,
  PassStatus,
  Platform,
  QualityMetrics,
  QualityReason,
  QualityStatus,
  Segment,
  SequenceStatus,
  SessionStatus,
  SlotOutcome,
  IncidentType,
  UserProfile,
  UUID,
} from '../domain/types';
// ----- errores
export type ApiErrorCode =
  | 'VALIDATION_ERROR'
  | 'EMAIL_ALREADY_REGISTERED'
  | 'INVALID_CREDENTIALS'
  | 'ACCOUNT_PENDING'
  | 'ACCOUNT_REJECTED'
  | 'ACCOUNT_BLOCKED'
  | 'ROLE_NOT_ALLOWED'
  | 'DEVICE_REVOKED'
  | 'PASSWORD_POLICY'
  | 'TOO_MANY_ATTEMPTS'
  | 'TOKEN_EXPIRED'
  | 'REFRESH_INVALID'
  | 'SESSION_NOT_FOUND'
  | 'PASS_NOT_FOUND'
  | 'SEQUENCE_NOT_FOUND'
  | 'CAPTURE_CONFLICT'
  | 'PAYLOAD_TOO_LARGE'
  | 'INTERNAL_ERROR';
export interface ApiErrorBody {
```

```

    code: ApiErrorCode;
    message: string;
    fieldErrors?: Array<{ field: string; message: string }>;
    traceId?: string;
}
// ----- autenticación
export interface DeviceInfoDto {
    deviceId: UUID;
    platform: Platform;
    model: string;
    osVersion: string;
    appVersion: string;
}
export interface RegisterRequest {
    fullName: string;
    email: string;
    phone: string | null;
    employeeCode: string | null;
    password: string;
    acceptedPrivacyNotice: true;
}
export interface RegisterResponse { userId: UUID; status: Extract<AccountStatus, 'PENDIENTE_APROBACION'>; }
export interface LoginRequest { email: string; password: string; device: DeviceInfoDto; }
export interface TokenBundle {
    accessToken: string;
    accessTokenExpiresAt: ISODateString;
    refreshToken: string;
    refreshTokenExpiresAt: ISODateString;
}
export interface LoginResponse extends TokenBundle { user: UserProfile; serverTime: ISODateString; }
export interface RefreshRequest { refreshToken: string; deviceId: UUID; }
export type RefreshResponse = LoginResponse;
export interface LogoutRequest { refreshToken: string; deviceId: UUID; }
export interface ChangePasswordRequest { currentPassword: string; newPassword: string; }
export interface PasswordResetRequest { email: string; }
export interface HealthResponse { status: 'UP'; serverTime: ISODateString; }
// ----- bootstrap
export interface BootstrapResponse {
    catalogVersion: string;
    lots: Lot[];
    rows: FieldRow[];
    segments: Segment[];
    markers: Marker[];
    lateralCodes: LateralCode[];
    /** Perfil de calidad y parámetros publicados por el backend; si faltan, la app usa sus valores por defecto. */
    qualityProfile: { version: string; params: Record<string, number> } | null;
    serverTime: ISODateString;
}
// ----- sincronización (idempotente por ID)
export interface SessionUpsertRequest {
    sessionId: UUID;
    operatorUserId: UUID;
    controllerDeviceId: UUID;
    /** SYNCED es solo local: nunca se envía. El cierre se envía con CLOSED y endedAt. */
    status: Exclude<SessionStatus, 'SYNCED'>;
    mode: CaptureMode;
    intervalMs: number;
    startedAt: ISODateString | null;
    endedAt: ISODateString | null;
    appVersion: string;
    configVersion: string;
    qualityProfileVersion: string;
    shortTestPassedAt: ISODateString | null;
    /** Todos los celulares que ocuparon un rol en la sesión; released = true si el rol se liberó para reemplazarlo. */
    cameras: Array<{
        role: CameraRole;
        deviceId: UUID;
        userId: UUID;
        platform: Platform;
        model: string;
        osVersion: string;
        appVersion: string;
        pairedAt: ISODateString;
        released: boolean;
    }>;
}
export interface PassUpsertRequest {
    passId: UUID;
    lotId: string;
    rowId: string;
    lateralCode: LateralCode;
    passOrder: number;
}

```

```

direction: Direction;
startMarkerId: string | null;
endMarkerId: string | null;
status: PassStatus;
startedAt: ISODateString | null;
endedAt: ISODateString | null;
sequencesTotal: number;
sequencesComplete: number;
sequencesIncomplete: number;
markerChanges: Array<{
  markerChangeId: UUID;
  markerId: string | null;
  segmentId: string | null;
  changedAt: ISODateString;
  lat: number | null;
  lon: number | null;
  gpsAccuracyM: number | null;
  gpsTimestamp: ISODateString | null;
}>;
}
export interface SequenceDto {
  sequenceId: UUID;
  passId: UUID;
  sequenceNumber: number;
  mode: CaptureMode;
  status: SequenceStatus;
  segmentId: string | null;
  markerId: string | null;
  lat: number | null;
  lon: number | null;
  gpsAccuracyM: number | null;
  gpsTimestamp: ISODateString | null;
  issuedAt: ISODateString;
  completedAt: ISODateString | null;
  expectedCaptureIds: Record<CameraRole, UUID>;
  /** Resultado por cámara: permite saber qué fotos faltan aunque no se suban. */
  slotOutcomes: Record<CameraRole, SlotOutcome>;
}
export interface SequenceBatchRequest { sessionId: UUID; sequences: SequenceDto[]; }
/** Parte "metadata" (JSON en texto) del multipart de POST /api/v1/captures/upload; la parte "file" es el JPEG. */
export interface CaptureUploadMetadata {
  captureId: UUID;
  sequenceId: UUID;
  sessionId: UUID;
  passId: UUID;
  lateralCode: LateralCode;
  deviceId: UUID;
  cameraRole: CameraRole;
  cameraUserId: UUID;
  operatorUserId: UUID;
  capturedAt: ISODateString;
  width: number;
  height: number;
  sizeBytes: number;
  md5: string;
  quality: { status: QualityStatus; reasons: QualityReason[]; metrics: QualityMetrics | null; profileVersion: string }
    ;
  replacesCaptureId: UUID | null;
  /** Solo en repeticiones: contexto vigente cuando se pidió repetir (tabla retake_requests). */
  retakeContext: {
    requestedAt: ISODateString;
    segmentId: string | null;
    markerId: string | null;
    lat: number | null;
    lon: number | null;
    gpsAccuracyM: number | null;
    gpsTimestamp: ISODateString | null;
  } | null;
  appVersion: string;
}
export interface CaptureUploadResponse { captureId: UUID; status: 'SINCRONIZADO'; duplicate: boolean; }
export interface IncidentDto {
  incidentId: UUID;
  passId: UUID | null;
  sequenceId: UUID | null;
  captureId: UUID | null;
  deviceId: UUID | null;
  type: IncidentType;
  severity: 'INFO' | 'AVISO' | 'ERROR';
  detail: string;
  occurredAt: ISODateString;
  createdBy: 'SISTEMA' | 'OPERADOR';
}

```

```
}
export interface IncidentBatchRequest { sessionId: UUID; incidents: IncidentDto[]; }
export interface BatchResponse { accepted: number; duplicates: number; }
```

15.4 Ejemplos

Login (petición y respuesta):

```
{
  "email": "operador@riachuelo.pe",
  "password": "*****",
  "device": { "deviceId": "0c7e2f7c-8a40-4f43-9a7c-5f0d6f2a1b33", "platform": "android", "model": "Galaxy A35", "osVer
    sion": "14", "appVersion": "0.4.0" }
}
```

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9...",
  "accessTokenExpiresAt": "2026-11-09T15:30:00.000Z",
  "refreshToken": "rt_7f3c...",
  "refreshTokenExpiresAt": "2026-11-23T14:30:00.000Z",
  "user": { "id": "9dle...", "fullName": "Operador de Campo", "email": "operador@riachuelo.pe", "roles": ["OPERADOR_CAMP
    o 0"], "status": "ACTIVO", "mustChangePassword": false },
  "serverTime": "2026-11-09T14:30:00.000Z"
}
```

Bootstrap (extracto; los códigos de segmentos y marcadores son ficticios hasta que el equipo los defina con la empresa):

```
{
  "catalogVersion": "2026-11-08T22:00:00.000Z",
  "lots": [{ "id": "SWG1", "code": "SWG 1", "name": "Lote 1 (Piscina)", "active": true }],
  "rows": [{ "id": "SWG1-H05", "lotId": "SWG1", "number": 5, "plantCount": 385, "active": true }],
  "segments": [{ "id": "SEG-01", "rowId": "SWG1-H05", "code": "SEG-01", "startPlant": 101, "endPlant": 125, "isPilot":
    true }],
  "markers": [
    { "id": "MK-01-I", "rowId": "SWG1-H05", "segmentId": "SEG-01", "code": "MK-01-I", "description": "Inicio del segme
      nto 1", "position": "INICIO", "lat": null, "lon": null },
    { "id": "MK-01-F", "rowId": "SWG1-H05", "segmentId": "SEG-01", "code": "MK-01-F", "description": "Fin del segmento
      1", "position": "FIN", "lat": null, "lon": null }
  ],
  "lateralCodes": ["LATERAL_A", "LATERAL_B"],
  "qualityProfile": null,
  "serverTime": "2026-11-09T14:30:00.000Z"
}
```

Metadatos de una captura (parte metadata del multipart):

```
{
  "captureId": "a1f0c3e2-6b7d-4c1a-9f2e-0d3b5a7c9e11",
  "sequenceId": "e2b4d6f8-1a3c-4e5f-8a9b-0c1d2e3f4a5b",
  "sessionId": "5b0f7a52-1d3c-4f0e-9a51-3f2b8c7d9e10",
  "passId": "7c9e1a3b-5d7f-4a1b-8c2d-3e4f5a6b7c8d",
  "lateralCode": "LATERAL_A",
  "deviceId": "3e5a7c9e-1b3d-4f5a-9c7e-2d4f6a8c0e1b",
  "cameraRole": "CAMERA_1",
  "cameraUserId": "9dle0f2a-3b4c-4d5e-8f6a-7b8c9d0e1f2a",
  "operatorUserId": "0dle0f2a-3b4c-4d5e-8f6a-7b8c9d0e1f2a",
  "capturedAt": "2026-11-09T14:32:05.412Z",
  "width": 4000,
  "height": 3000,
  "sizeBytes": 3912044,
  "md5": "9e107d9d372bb6826bd81d3542a419d6",
  "quality": {
    "status": "UTILIZABLE",
    "reasons": [],
    "metrics": { "luminanceMean": 121.4, "darkRatio": 0.041, "brightRatio": 0.012, "laplacianVariance": 412.7, "analyz
      edRegions": 3, "durationMs": 940 },
    "profileVersion": "Q0"
  },
  "replacesCaptureId": null,
  "retakeContext": null,
  "appVersion": "0.4.0"
}
```

15.5 Orden de sincronización y errores

Al cerrar una sesión (CLOSING → CLOSED), en la misma transacción se crean sus elementos de sync_queue:

order_key	entity_type	entity_id	Petición
10	SESSION	sessionId	POST /sessions con status = CLOSING, endedAt = null, sus cámaras y shortTestPassedAt
20	PASS (uno por pasada)	passId	POST /sessions/{id}/passes con markerChanges
30	SEQUENCE_BATCH (uno por pasada)	passId	POST /sessions/{id}/sequences/batch, en peticiones de hasta sync.batchSize secuencias
40	CAPTURE (uno por foto)	captureId	POST /captures/upload (multipart)
50	INCIDENT_BATCH	sessionId	POST /sessions/{id}/incidents/batch, en peticiones de hasta sync.batchSize incidencias
60	SESSION_CLOSE	sessionId	POST /sessions con status = CLOSED y endedAt

Solo se encolan capturas con `is_test = 0` y `local_transfer_status = RECIBIDA_CONTROLADOR` (también las rechazadas recibidas, que sirven para auditar y calibrar). El backend considera completa la sesión al recibir el cierre (60). SYNCED es un estado solo local: nunca se envía al backend.

1. Comprobar GET /health; si falla, no se intenta nada (la cola sigue igual).
2. Exigir sesión de usuario ONLINE: si es OFFLINE, PANT-20 pide validar la contraseña con internet (7.5). Renovar el token si hace falta (7.4).
3. Procesar los elementos PENDIENTE cuyo next_attempt_at ya pasó, por sesión y por order_key. Un elemento que se envía en varias peticiones se marca HECHO solo cuando todas responden bien; si una falla, se reintenta completo (el backend es idempotente).
4. Capturas: `uploadAsync(url, fileUri, { httpMethod: 'POST', uploadType: FileSystemUploadType.MULTIPART, fieldName: 'file', mimeType: 'image/jpeg', parameters: { metadata: JSON.stringify(meta) }, headers: { Authorization, 'X-Device-Id': deviceId } })` de expo-file-system/legacy, una a la vez (sync.maxConcurrent). En una repetición, `meta.retakeContext` sale de `retake_requests`.
5. Clasificar cada fallo con `classifySyncError` (Anexo C): REINTENTAR → espera `sync.retryDelaysMs`; RENOVAR_TOKEN → renovar y reintentar una vez; REQUIERE_LOGIN → pausar la cola y pedir login con internet; SINCRONIZAR_PADRE → reencolar el padre con prioridad; DEFINITIVO → ERROR_DEFINITIVO / ERROR_SINCRONIZACION con el código visible en PANT-20.
6. Una sesión pasa a SYNCED cuando todos sus elementos están en HECHO. La falta de red no cuenta como intento fallido.
7. **Fotos tardías (8.13):** al recibir una foto de una sesión cerrada se agrega su CAPTURE y se vuelven a dejar en PENDIENTE (con `attempts = 0`) el PASS y el SEQUENCE_BATCH de su pasada, con un `upsert` por (entity_type, entity_id); la pasada y sus secuencias vuelven a PENDIENTE_NUBE. Si la sesión estaba SYNCED, vuelve a CLOSED.

15.6 Backend simulado (Fase 1)

- `src/api/mock/` implementa las mismas funciones que el cliente real (authApi, bootstrapApi, syncApi) con latencia y tasa de fallos configurables, y se activa con `EXP0_PUBLIC_USE MOCK_API=1`.
- Cuentas de prueba: `operador@demo.pe / Demo2026` (ACTIVO, OPERADOR_CAMPO); `admin@demo.pe / Demo2026` (ACTIVO, ADMINISTRADOR); `pendiente@demo.pe` (PENDIENTE_APROBACION); `bloqueado@demo.pe` (BLOQUEADO); `temporal@demo.pe / Temp2026` (mustChangePassword); `supervisor@demo.pe / Demo2026` (ROLE_NOT_ALLOWED).
- Catálogos semilla con los lotes SWG 1, SWG 2 y SWG 5 y sus hileras reales (Anexo D); segmentos y marcadores de ejemplo hasta tener los definitivos.
- Las subidas simuladas guardan solo el `captureId` y el `md5` para poder probar duplicados y conflictos.
- Acepta actualizar pasadas y secuencias ya recibidas (fotos tardías) igual que el backend real.

16. Calidad de imagen

16.1 Qué decide

La validación local solo decide si la foto es **técnicamente utilizable**. No decide presencia ni ausencia de plaga (RN-14). Cada resultado se guarda con sus métricas y la versión del perfil para explicar por qué se aceptó o se pidió repetir.

16.2 Procedimiento

1. **Exposición** sobre una copia reducida de toda la foto (`quality.exposure.analysisWidth = 256 px`): luminancia media y proporción de píxeles casi negros ($\leq \text{darkLevel}$) y saturados ($\geq \text{brightLevel}$). Si falla, el resultado es REPETIR_EXPOSICION y **no** se mide la nitidez.
2. **Nitidez** en 1 a 3 recortes cuadrados de `sharpness.regionSize` (512 px) **a resolución original**, sobre la línea media horizontal (25 %, 50 % y 75 % del ancho). Se calcula la varianza del Laplaciano de 4 vecinos en escala de grises y se usa el **máximo** (basta con que una zona tenga detalle para descartar desenfoque general; el movimiento afecta a todas). Medir a resolución original evita que la reducción oculte un desenfoque leve.
3. Decisión con `decideQuality` (Anexo B): REPETIR_EXPOSICION (EXPOSICION_OSCURA o EXPOSICION_SATURADA), REPETIR_NITIDEZ (NITIDEZ_BAJA) o UTILIZABLE.
4. Si todo tarda más de `quality.timeoutMs` (4 s): PENDIENTE_REVISION_TECNICA con TIEMPO_AGOTADO (la foto se acepta y queda marcada para revisión).
5. Si la foto no se puede leer: ERROR_CAMARA con ARCHIVO_INVALIDO.
6. Se borran las copias temporales de caché y se guarda `quality_results` (métricas, motivos, `duration_ms`, versión).

Herramientas (verificadas en el SDK 54): `ImageManipulator.manipulate(uri).resize({ width })` o `.crop({ originX, originY, width, height })` → `renderAsync()` → `saveAsync({ format: SaveFormat.JPEG, compress })`; lectura con `new File(uri).bytes()`; decodificación con `jpeg.decode(bytes, { useTArray: true, formatAsRGBA: false })`. El boceto `qualityPipeline.ts` del Anexo B compila contra esas firmas; se prueba en el celular en la tarea T-11.

16.3 Perfil Q0 (valores iniciales, no validados)

Parámetro	Q0	Significado
<code>analysisWidth</code>	256 px	Ancho de la copia para exposición.
<code>darkLevel / brightLevel</code>	20 / 245	Luminancia (0–255) de píxel casi negro / saturado.
<code>minMean / maxMean</code>	50 / 205	Rango aceptable de luminancia media.
<code>maxDarkRatio / maxBrightRatio</code>	0,40 / 0,20	Proporción máxima de píxeles oscuros / saturados.
<code>regionSize / regionCount</code>	512 px / 3	Recortes para nitidez.
<code>minLaplacianVariance</code>	100	Varianza mínima del Laplaciano.
<code>timeoutMs</code>	4 000 ms	Límite total; sumado a <code>capture.captureBudgetMs</code> debe caber en <code>protocol.captureResponseTimeoutMs</code> (14.10).

Referencia con imágenes sintéticas (pruebas del Anexo E): una textura nítida dio una varianza $\approx 16\,800$; con desenfoque de radio 1, $\approx 1\,000$; radio 2, ≈ 270 ; radio 4, ≈ 18 . Una foto oscura dio media 16 y 63 % de píxeles oscuros, pero varianza ≈ 244 : por eso la exposición se evalúa primero. Estos números solo prueban que el código funciona; los umbrales reales se calibran con fotos de vid (16.5).

16.4 Rendimiento

- Meta: ≤ 2 s por foto (RNF-05), medida con `duration_ms` en los celulares del piloto.
- Si se supera: bajar `regionCount` a 2 o 1, o `regionSize` a 384; la copia de exposición puede bajar a 192 px.
- Si aún no alcanza: evaluar un módulo nativo compatible con el SDK 54 (decisión registrada). La versión 1 de `react-native-fast-opencv` solo está probada con React Native 0.85 o superior; con el SDK 54 habría que evaluar su serie 0.4.x en un Development Build.

16.5 Calibración del perfil Q1 (Fase 6)

1. Tomar fotos con el soporte y los celulares del piloto en sol directo, sombra, contraluz y caminata normal y lenta.
2. El especialista y el equipo etiquetan cada foto como útil o no útil.
3. Exportar las métricas (`quality_results`) con la etiqueta.
4. Elegir los umbrales que mejor separan útiles de no útiles; registrar el perfil **Q1** y la evidencia en `docs/evidencias/`.
5. Probar Q1 en una hilera y congelarlo con la versión del piloto.

17. Configuración — `src/config/defaults.ts` (CFG-1)

```
// src/config/defaults.ts — Parámetros configurables (versión CFG-1).
// Los valores marcados "calibrar" son iniciales: se ajustan con mediciones y NO son resultados validados.
export const CONFIG_VERSION = 'CFG-1';
export interface AppConfig {
  auth: {
    offlineLoginMaxDays: number; // ventana para entrar sin internet desde la última validación online
    offlineMaxFailedAttempts: number; // intentos fallidos antes del bloqueo temporal (solo login sin internet)
    offlineLockoutMinutes: number;
    pbkdf2Iterations: number; // calibrar: objetivo  $\leq 1,5$  s en el celular más lento del piloto
    passwordMinLength: number;
    refreshMarginSeconds: number; // renovar el access token si vence en menos de este margen
  };
  catalog: {
    bootstrapWarnAgeHours: number; // advertir si los catálogos son más antiguos
  };
  pairing: {
    controlPort: number;
    filePort: number;
    /** RN-17: src/config/index.ts lo fija en true cuando ENV.appEnv === 'piloto'. */
    requireSameAppVersion: boolean;
    pairingTokenBytes: number;
  };
  protocol: {
    version: 1;
    heartbeatIntervalMs: number;
    unstableAfterMs: number; // sin heartbeat → INESTABLE
    lostAfterMs: number; // sin heartbeat → PERDIDA y pausa
    reconnectDelaysMs: number[]; // esperas de la cámara entre intentos de reconexión
    commandValidityMs: number; // vigencia de CAPTURE_COMMAND
    captureResponseTimeoutMs: number; // espera de CAPTURE_OK / QUALITY_ERROR ( $\geq$  captureBudgetMs + quality.timeoutMs + 500)
    ackTimeoutMs: number; // espera de ACK de los mensajes que lo requieren
    ackMaxRetries: number; // reenvíos con el mismo messageId antes de considerar el enlace PERDIDA
    resyncPageSize: number; // capturas por mensaje RESYNC_STATE
    clockOffsetSamples: number; // mediciones usadas para la mediana del desfase de reloj
    processedMessagesTtlHours: number;
    maxMessageBytes: number;
  };
  capture: {
    defaultMode: 'MANUAL' | 'AUTOMATICO' | 'MIXTO';
    intervalMs: number; // calibrar
    minIntervalMs: number;
    maxIntervalMs: number;
    jpegQuality: number; // 0..1
    shutterSound: boolean;
    captureBudgetMs: number; // tiempo previsto para tomar, mover y calcular el md5 de la foto
  };
  shortTest: {
    timeoutMs: number; // límite de la prueba corta por cámara (captura + calidad + transferencia + ACK)
  };
  quality: {
```

```

profileVersion: string;
timeoutMs: number; // si se supera: PENDIENTE_REVISION_TECNICA
exposure: {
  analysisWidth: number;
  darkLevel: number; // luminancia 0..255 considerada "casi negra"
  brightLevel: number; // luminancia 0..255 considerada "saturada"
  minMean: number; // calibrar
  maxMean: number; // calibrar
  maxDarkRatio: number; // calibrar
  maxBrightRatio: number; // calibrar
};
sharpness: {
  regionSize: number; // lado del recorte en píxeles de la foto original
  regionCount: 1 | 2 | 3; // recortes sobre la línea media horizontal
  minLaplacianVariance: number; // calibrar
};
};
transfer: {
  maxConcurrentPerCamera: number;
  retryDelaysMs: number[];
  retryMaxDelayMs: number;
  requestTimeoutMs: number;
  includeRejected: boolean; // transferir también fotos rechazadas (después de las útiles)
  maxChecksumRetries: number; // MD5_MISMATCH / SIZE_MISMATCH antes de ERROR_LOCAL
};
sync: {
  maxConcurrent: number;
  retryDelaysMs: number[];
  requestTimeoutMs: number;
  healthPath: string; // relativo a la base `${EXPO_PUBLIC_API_URL}/api/v1`
  batchSize: number; // secuencias o incidencias por petición
};
gps: {
  timeIntervalMs: number;
  distanceIntervalM: number;
  maxAgeMs: number; // lectura más antigua no se asocia a la secuencia
  warnAccuracyM: number; // calibrar
};
device: {
  minBatteryToStartPct: number;
  warnBatteryPct: number;
  pauseBatteryPct: number;
  warnFreeSpaceBytes: number;
  minFreeSpaceToStartBytes: number;
  pauseFreeSpaceBytes: number;
};
retention: {
  /** TRAS_SINCRONIZACION: la cámara borra sus copias con ACK de sesiones que el controlador informa SINCRONIZADAS.
   *  */
  cameraPolicy: 'CONSERVAR' | 'TRAS_SINCRONIZACION';
  /** CONSERVAR: el controlador solo libera espacio por acción manual y solo de sesiones SINCRONIZADAS. */
  controllerPolicy: 'CONSERVAR' | 'TRAS_SINCRONIZACION';
};
logging: {
  eventLogMaxDays: number;
};
};
const GB = 1024 * 1024 * 1024;
export const DEFAULT_CONFIG: AppConfig = {
  auth: {
    offlineLoginMaxDays: 7,
    offlineMaxFailedAttempts: 5,
    offlineLockoutMinutes: 15,
    pbkdf2Iterations: 40_000,
    passwordMinLength: 8,
    refreshMarginSeconds: 60,
  },
  catalog: { bootstrapWarnAgeHours: 72 },
  pairing: { controlPort: 8765, filePort: 8766, requireSameAppVersion: false, pairingTokenBytes: 16 },
  protocol: {
    version: 1,
    heartbeatIntervalMs: 2_000,
    unstableAfterMs: 3_000,
    lostAfterMs: 6_000,
    reconnectDelaysMs: [1_000, 2_000, 4_000, 8_000],
    commandValidityMs: 3_000,
    captureResponseTimeoutMs: 7_000,
    ackTimeoutMs: 2_000,
    ackMaxRetries: 3,
    resyncPageSize: 200,
    clockOffsetSamples: 5,
    processedMessagesTtlHours: 24,
  },
};

```

```

    maxMessageBytes: 64 * 1024,
  },
  capture: {
    defaultMode: 'MANUAL',
    intervalMs: 2_000,
    minIntervalMs: 1_000,
    maxIntervalMs: 10_000,
    jpegQuality: 1,
    shutterSound: false,
    captureBudgetMs: 2_000,
  },
  shortTest: { timeoutMs: 15_000 },
  quality: {
    profileVersion: '00',
    timeoutMs: 4_000,
    exposure: {
      analysisWidth: 256,
      darkLevel: 20,
      brightLevel: 245,
      minMean: 50,
      maxMean: 205,
      maxDarkRatio: 0.4,
      maxBrightRatio: 0.2,
    },
  },
  sharpness: { regionSize: 512, regionCount: 3, minLaplacianVariance: 100 },
},
transfer: {
  maxConcurrentPerCamera: 1,
  retryDelaysMs: [2_000, 4_000, 8_000, 16_000],
  retryMaxDelayMs: 30_000,
  requestTimeoutMs: 30_000,
  includeRejected: true,
  maxChecksumRetries: 3,
},
sync: {
  maxConcurrent: 1,
  retryDelaysMs: [60_000, 300_000, 900_000, 3_600_000],
  requestTimeoutMs: 60_000,
  healthPath: '/health',
  batchSize: 200,
},
gps: { timeIntervalMs: 1_000, distanceIntervalM: 0, maxAgeMs: 10_000, warnAccuracyM: 15 },
device: {
  minBatteryToStartPct: 30,
  warnBatteryPct: 25,
  pauseBatteryPct: 15,
  warnFreeSpaceBytes: 3 * GB,
  minFreeSpaceToStartBytes: 2 * GB,
  pauseFreeSpaceBytes: 1 * GB,
},
retention: { cameraPolicy: 'TRAS_SINCRONIZACION', controllerPolicy: 'CONSERVAR' },
logging: { eventLogMaxDays: 30 },
};

```

Reglas:

- `src/config/index.ts` exporta `CONFIG`: los valores por defecto, más (en el controlador) el `qualityProfile` del bootstrap si el backend lo envía, y `pairing.requireSameAppVersion = (ENV.appEnv === 'piloto')` (RN-17).
- Reglas de coherencia, probadas en el Anexo E: `captureResponseTimeoutMs ≥ captureBudgetMs + quality.timeoutMs + 500`; `commandValidityMs < captureResponseTimeoutMs`; `shortTest.timeoutMs > captureResponseTimeoutMs`; `ackTimeoutMs < lostAfterMs`; batería: pausa < aviso < inicio; espacio: pausa < inicio < aviso; una página de `RESYNC_STATE` cabe en `maxMessageBytes`. Un cambio que rompa una regla no se acepta.
- Las cámaras no usan su propia configuración de captura y calidad durante una sesión: usan la que reciben en `PAIRED` (D-21).
- El intervalo de la sesión se elige en `PANT-12` dentro de `capture.minIntervalMs` y `capture.maxIntervalMs`.
- Cambiar un valor por defecto sube `CONFIG_VERSION` (CFG-2, CFG-3...) y se registra en la sección 25.
- `src/config/env.ts` lee las variables con su nombre completo (Expo las reemplaza al compilar):

```
export const ENV = {
```

```
apiUrl: process.env.EXPO_PUBLIC_API_URL ?? '',
useMockApi: process.env.EXPO_PUBLIC_USE MOCK_API === '1',
appEnv: process.env.EXPO_PUBLIC_APP_ENV === 'piloto' ? 'piloto' : 'dev',
} as const;
```

18. Seguridad y privacidad

- **Contraseñas:** nunca se guardan ni se registran. Política mínima: 8 caracteres con letras y números (el backend aplica la misma). El login sin internet usa solo el verificador PBKDF2 (7.5).
- **Tokens:** solo en SecureStore; nunca en SQLite, logs ni diagnósticos. Renovación con rotación; revocación según 7.8.
- **Dispositivo perdido:** el administrador revoca el dispositivo; en el siguiente contacto con internet la app borra tokens y verificador. Recomendar bloqueo de pantalla en los celulares del proyecto.
- **Red local:** token de emparejamiento aleatorio (16 bytes) que en SQLite se guarda solo como hash (el valor va en SecureStore) y se compara en tiempo constante; se rechazan mensajes y fotos de sesiones o dispositivos no emparejados; la IP de una subida debe coincidir con la del WebSocket de esa cámara. El HTTP local no está cifrado: la red de campo debe tener contraseña WPA2 y no conectarse a redes públicas. Esta limitación queda documentada (Esp. v2.0 §23.2).
- **Datos en el celular:** fotos y base de datos en el almacenamiento privado de la app. La base de datos no está cifrada (limitación documentada); los secretos van en SecureStore.
- **Privacidad:** aviso en el registro; datos personales mínimos (nombre, correo; celular y código opcionales); cámaras orientadas a la vegetación; el diagnóstico exportado no contiene contraseñas, tokens ni el verificador.
- **Dependencias:** versiones fijadas; licencias permisivas (MIT, BSD, Apache-2.0). No copiar código de repositorios sin licencia o con licencia GPL.
- **Backend:** HTTPS en entornos remotos; la app nunca contiene credenciales de PostgreSQL, S3, RabbitMQ ni del servicio de IA (R-18).

19. Errores y mensajes al usuario — src/ui/messages.ts

Reglas: un código, un texto; nunca mostrar el texto de una excepción; el detalle técnico va a event_log; los mensajes no culpan al usuario y dicen qué hacer.

```
// src/ui/messages.ts - Mensajes al usuario por código (español, Perú). Un código = un texto.
import type { ApiErrorCode } from '../api/dto';
import type { OfflineDenyReason } from '../auth/offlineAuth';
import type { PairRejectReason } from '../protocol/messages';
export type AppErrorCode =
  | 'SIN_INTERNET'
  | 'BACKEND_NO_DISPONIBLE'
  | 'PERMISO_CAMARA'
  | 'PERMISO_UBICACION'
  | 'PERMISO_RED_LOCAL'
  | 'GPS_NO_DISPONIBLE'
  | 'GPS_IMPRECISO'
  | 'CATALOGOS_FALTANTES'
  | 'CATALOGOS_ANTIGUOS'
  | 'BATERIA_BAJA'
  | 'BATERIA_CRITICA'
  | 'ESPACIO_BAJO'
  | 'ESPACIO_CRITICO'
  | 'CAMARAS_NO_LISTAS'
  | 'PRUEBA_CORTA_PENDIENTE'
  | 'CAMARA_DESCONECTADA'
  | 'ORDEN_VENCIDA'
  | 'SESION_ABIERTA_IMPIDE_SALIR'
  | 'CAMBIO_FUNCION_BLOQUEADO'
  | 'PENDIENTES_DE_TRANSFERENCIA'
  | 'CIERRE_CON_PENDIENTES'
  | 'FOTOS_DE_OTRO_CONTROLADOR'
  | 'LIBERAR_CAMARA_NO_PERMITIDO'
  | 'PRUEBA_CORTA_FALLIDA'
  | 'PRUEBA_CORTA_CALIDAD'
  | 'PAUSA_SEGUNDO_PLANO'
  | 'REAUTENTICACION_REQUERIDA'
```

```

| 'QR_INVALIDO'
| 'VERSION_DISTINTA'
| 'ERROR_INESPERADO';
export const API_ERROR_MESSAGES: Record<ApiErrorCode, string> = {
  VALIDATION_ERROR: 'Revisa los datos marcados e inténtalo de nuevo.',
  EMAIL_ALREADY_REGISTERED: 'Ese correo ya tiene una cuenta. Inicia sesión o solicita restablecer tu contraseña.',
  INVALID_CREDENTIALS: 'Correo o contraseña incorrectos.',
  ACCOUNT_PENDING: 'Tu cuenta está pendiente de aprobación por el administrador.',
  ACCOUNT_REJECTED: 'Tu solicitud de cuenta fue rechazada. Comunícate con el administrador.',
  ACCOUNT_BLOCKED: 'Tu cuenta está bloqueada. Comunícate con el administrador.',
  ROLE_NOT_ALLOWED: 'Tu usuario no tiene permiso para usar la app móvil.',
  DEVICE_REVOKED: 'Este celular fue desactivado por el administrador.',
  PASSWORD_POLICY: 'La contraseña no cumple los requisitos.',
  TOO_MANY_ATTEMPTS: 'Demasiados intentos. Espera unos minutos e inténtalo de nuevo.',
  TOKEN_EXPIRED: 'Tu sesión venció. Vuelve a iniciar sesión.',
  REFRESH_INVALID: 'Tu sesión ya no es válida. Vuelve a iniciar sesión con internet.',
  SESSION_NOT_FOUND: 'El servidor aún no tiene la sesión; se enviará primero.',
  PASS_NOT_FOUND: 'El servidor aún no tiene la pasada; se enviará primero.',
  SEQUENCE_NOT_FOUND: 'El servidor aún no tiene la secuencia; se enviará primero.',
  CAPTURE_CONFLICT: 'El servidor ya tiene una foto con ese identificador pero con otro contenido. Requiere revisión.',
  PAYLOAD_TOO_LARGE: 'La foto supera el tamaño permitido por el servidor.',
  INTERNAL_ERROR: 'El servidor tuvo un problema. Se volverá a intentar.',
};
export const OFFLINE_LOGIN_MESSAGES: Record<OfflineDenyReason, string> = {
  SIN_VERIFICADOR: 'Sin internet solo puede entrar quien ya inició sesión con internet en este celular.',
  OTRO_USUARIO: 'Sin internet solo puede entrar el último usuario que inició sesión con internet en este celular.',
  VENCIDO: 'Pasaron demasiados días sin conectarte. Inicia sesión con internet.',
  BLOQUEADO_TEMPORAL: 'Demasiados intentos fallidos. Espera unos minutos.',
  CUENTA_NO_ACTIVADA: 'Tu cuenta no está activa.',
  ROL_NO_PERMITIDO: 'Tu usuario no tiene permiso para usar la app móvil.',
  CAMBIO_CONTRASEÑA_PENDIENTE: 'Debes cambiar tu contraseña con internet antes de entrar sin conexión.',
};
export const PAIR_REJECT_MESSAGES: Record<PairRejectReason, string> = {
  INVALID_TOKEN: 'El código QR no es válido o ya fue reemplazado. Escanea el QR actual del controlador.',
  SESSION_CLOSED: 'La sesión del controlador ya está cerrada.',
  ROLE_TAKEN: 'Esa función de cámara ya está ocupada por otro celular.',
  PROTOCOL_MISMATCH: 'La versión del protocolo no coincide. Actualiza la app en ambos celulares.',
  APP_VERSION_MISMATCH: 'La versión de la app no coincide con la del controlador.',
  USER_NOT_ALLOWED: 'El usuario de este celular no puede unirse a la sesión.',
};
export const APP_ERROR_MESSAGES: Record<AppErrorCode, string> = {
  SIN_INTERNET: 'No hay internet. Esta acción necesita conexión.',
  BACKEND_NO_DISPONIBLE: 'No se pudo contactar al servidor. Se reintentará más tarde.',
  PERMISO_CAMARA: 'Activa el permiso de cámara en Ajustes para continuar.',
  PERMISO_UBICACION: 'Activa el permiso de ubicación en Ajustes para registrar el GPS.',
  PERMISO_RED_LOCAL: 'Activa "Red local" para esta app en Ajustes del iPhone.',
  GPS_NO_DISPONIBLE: 'Sin señal GPS: las secuencias se guardarán sin coordenadas.',
  GPS_IMPRECISO: 'Precisión GPS baja: se usará lote, hilera y marcador para ubicar.',
  CATALOGOS_FALTANTES: 'Descarga los catálogos con internet antes de crear una sesión.',
  CATALOGOS_ANTIGUOS: 'Los catálogos tienen varios días. Actualízalos si tienes internet.',
  BATERIA_BAJA: 'Batería baja: conecta el power bank.',
  BATERIA_CRITICA: 'Batería crítica: la pasada se pausó.',
  ESPACIO_BAJO: 'Queda poco espacio libre. Sincroniza y libera espacio al volver.',
  ESPACIO_CRITICO: 'Espacio insuficiente: la pasada se pausó.',
  CAMARAS_NO_LISTAS: 'Las dos cámaras deben estar conectadas para iniciar la pasada.',
  PRUEBA_CORTA_PENDIENTE: 'Haz la prueba corta antes de iniciar la primera pasada.',
  CAMARA_DESCONECTADA: 'Se perdió la conexión con una cámara. La pasada se pausó.',
  ORDEN_VENCIDA: 'La orden llegó tarde y se descartó.',
  SESION_ABIERTA_IMPIDE_SALIR: 'Cierra la sesión de monitoreo antes de cerrar tu sesión de usuario.',
  CAMBIO_FUNCION_BLOQUEADO: 'No puedes cambiar la función con una sesión abierta o con pendientes en este celular.',
  PENDIENTES_DE_TRANSFERENCIA: 'Hay fotos pendientes de enviar al controlador.',
  CIERRE_CON_PENDIENTES: 'Aún faltan fotos por llegar al controlador. Puedes esperar o cerrar igual: llegarán en el próximo emparejamiento.',
  FOTOS_DE_OTRO_CONTROLADOR: 'Hay fotos de otra sesión: se enviarán cuando este celular se vincule con su controlador.',
  LIBERAR_CAMARA_NO_PERMITIDO: 'Solo puedes liberar una cámara desconectada y sin una pasada abierta.',
  PRUEBA_CORTA_FALLIDA: 'La prueba corta falló en una cámara. Revisa la conexión y repítela.',
  PRUEBA_CORTA_CALIDAD: 'La comunicación funciona, pero la foto de prueba salió con mala calidad. Revisa la posición y la luz.',
  PAUSA_SEGUNDO_PLANO: 'Un celular salió de la app: la pasada se pausó. Vuelve a la app para continuar.',
  REAUTENTICACION_REQUERIDA: 'Hay internet. Ingresar tu contraseña para validar tu acceso y poder sincronizar.',
  QR_INVALIDO: 'Este código QR no es de la app de monitoreo.',
  VERSION_DISTINTA: 'Las versiones de la app no coinciden entre los celulares.',
  ERROR_INESPERADO: 'Ocurrió un error inesperado. Quedó registrado en el diagnóstico.',
};

```

20. Registro de eventos, diagnóstico e indicadores

20.1 event_log

Categoría	Eventos
AUTH	LOGIN_ONLINE_OK, LOGIN_OFFLINE_OK, REAUTH_OK, LOGIN_FAIL (código), REFRESH_OK, REVOKED, LOGOUT, PBKDF2_MS (tiempo medido)
DEVICE	ROLE_SET, PERMISSION, BATTERY_LOW, STORAGE_LOW, APP_STATE
CATALOG	BOOTSTRAP_OK, BOOTSTRAP_FAIL
SESSION	STATE (de → a), PASS_STATE, MARKER_CHANGE, SHORT_TEST (resultado y tiempos), ROLE_RELEASED, CONTEXT_CLOSED (cámara, con el motivo de 8.12)
NET	SERVER_START, PAIRED, PAIR_REJECTED, LINK_STATE, RESYNC, CLOCK_OFFSET
CAPTURE	SEQUENCE_CREATED, COMMAND_SENT, CAPTURE_OK, QUALITY_ERROR, RETAKE, LATENCY_MS
TRANSFER	START, OK, RETRY, ERROR (motivo, intentos, tiempo), MANUAL_RETRY, LATE (foto de una sesión cerrada)
SYNC	START, ITEM_OK, ITEM_RETRY, ITEM_ERROR, SESSION_SYNCED
GPS	FIX_LOST, FIX_OK, LOW_ACCURACY
ERROR	Errores no controlados (sin datos sensibles)

data_j son lleva solo identificadores, códigos, tiempos y conteos.

20.2 Exportar diagnóstico

PANT-09 genera diagnostics/diagnostico_{deviceId}_{fecha}.json con: versión de app, protocolo, configuración y perfil; datos del equipo; función; resumen de sesiones; últimos logging.eventLogMaxDays días de eventos; conteos de colas. Se comparte con expo-sharing (shareAsync).

20.3 Indicadores que deben poder calcularse con los datos de la app

Duración de cada pasada y de cada segmento (por cambio de marcador); secuencias completas, parciales e incompletas; porcentaje de fotos UTILIZABLE; repeticiones por cámara; desconexiones; latencia de orden; desfase entre cámaras; tiempos de transferencia y de sincronización. Son evidencia complementaria para el postest (Curso §3.17) y no reemplazan los instrumentos de la investigación.

21. Pruebas

21.1 Pruebas unitarias obligatorias

Módulo	Qué se prueba	Referencia
Dominio	Transiciones válidas e inválidas; metas conocidas; estado de secuencia por resultados; repetición; todo cambio permitido de un resultado produce una transición permitida ; vuelta de SYNCED a CLOSED	Anexo E (domain.test.ts)
Configuración	Coherencia de tiempos (captura + calidad caben en la espera de respuesta), umbrales, límites, ruta de salud y tamaño de página de RESYNC	Anexo E (domain.test.ts)
Protocolo	Esquemas zod: aceptan ejemplos válidos y rechazan campos faltantes, tipos erróneos y $v \neq 1$; duplicados por messageId	Tarea T-01
Códec WebSocket	Clave de la RFC 6455; frames partidos; 16 bits; eco con cliente real; fragmentación; ping/pong; cierre; handshake inválido; límite 64 KB	Anexo E (wsCodec.test.ts)
Receptor HTTP	Foto de 3 MB con md5; duplicado ALREADY_RECEIVED; md5 erróneo 422; Expect: 100-continue; base64url	Anexo E (httpUpload.test.ts)
Calidad	Nítida → UTILIZABLE; borrosa → REPETIR_NITIDEZ; oscura y saturada → REPETIR_EXPOSICION; JPEG ida y vuelta; recortes dentro de la imagen	Anexo E (quality.test.ts)

Módulo	Qué se prueba	Referencia
Autenticación	Base64; PBKDF2 igual al de Node; reglas sin internet (otro usuario, vencido, rol, contraseña temporal, bloqueo y desbloqueo)	Anexo E (offlineAuth.test.ts)
Validaciones	Política de contraseña; registro válido e inválido; mensajes con texto	Anexo E (validation.test.ts)
Sincronización y transferencia	Esperas con tope; clasificación de errores de sincronización; decisión tras cada envío local (classifyLocalTransfer); orden de la cola; idempotencia de reencolar	Anexo E (retry.test.ts) + T-17 y T-20
Repositorios	Conversión fila ↔ dominio; (opcional) SQL con sql.js en Jest	T-02

Las pruebas del Anexo E usan node:test. Al pasarlas a Jest: cambiar test por el global de Jest y assert por expect (o mantener node:assert/strict), y marcar wsCodec.test.ts y httpUpload.test.ts con `/** @jest-environment node */` (usan net, http y el paquete ws como dependencia de desarrollo).

21.2 Casos de prueba funcionales (manuales)

ID	Caso	Resultado esperado	Fase
CP-01	Login con credenciales correctas e internet	Entra; tokens guardados; pide función si no tiene	F1
CP-02	Login con contraseña incorrecta	«Correo o contraseña incorrectos»; no entra	F1
CP-03	Login de cuenta pendiente, rechazada, bloqueada o con rol no permitido	Mensaje específico de cada caso	F1
CP-04	Login sin internet del último usuario (≤ 7 días)	Entra en modo sin internet con fecha límite	F1
CP-05	Login sin internet de otro usuario, o con el verificador vencido	No entra; mensaje correcto	F1
CP-06	5 contraseñas incorrectas sin internet	Bloqueo de 15 min con hora de fin	F1
CP-07	Login con contraseña temporal	Solo permite cambiarla; luego entra	F1
CP-08	Registro válido / correo repetido / sin internet	PANT-04 / error en el campo / botón deshabilitado	F1
CP-09	Recuperar contraseña	Mensaje neutro	F1
CP-10	Elegir función y reiniciar	La función se mantiene	F1
CP-11	Cambiar función con sesión abierta o pendientes	Bloqueado con mensaje	F1
CP-12	Negar permiso de cámara o ubicación	Pantalla guía; no continúa	F1
CP-13	Descargar catálogos y elegir valores en modo avión	Valores disponibles sin internet	F1
CP-14	Pasada completa con el simulador (manual y automático)	Secuencias completas; contadores correctos	F1
CP-15	Error de calidad simulado y repetición	Nuevo captureId con replacesCaptureId; la otra foto se conserva	F1
CP-16	Cerrar la app a mitad de pasada y abrirla	Pasada PAUSED con todos sus datos; se reanuda	F1
CP-17	Emparejar dos cámaras reales con QR	Ambas CONECTADA; prueba corta aprobada	F2/F3
CP-18	QR anterior tras regenerarlo; rol ocupado; versión de app distinta con EXPO_PUBLIC_APP_ENV=piloto	INVALID_TOKEN; ROLE_TAKEN; APP_VERSION_MISMATCH	F2
CP-19	Iniciar pasada sin cámaras o sin prueba corta	Bloqueado con mensaje	F2
CP-20	Apagar el router 10 s durante una pasada	Pausa automática; reconexión; RESYNC; se reanuda sin duplicados	F2
CP-21	Cortar la red durante una transferencia	La foto llega una sola vez con el mismo captureId	F2
CP-22	Una cámara se apaga durante la pasada	Secuencias PARTIAL/INCOMPLETE visibles; incidencia; la otra cámara conserva sus fotos	F3
CP-23	Cambiar marcador	Las secuencias siguientes llevan el nuevo marcador	F1
CP-24	Cerrar LATERAL_A y crear LATERAL_B; intentar B sin cerrar A	B permitido / B bloqueado	F1
CP-25	Cerrar sesión con transferencias pendientes	Ofrece «Esperar» o «Cerrar igual»; con «Cerrar igual» registra la incidencia y llega a CLOSED	F2

ID	Caso	Resultado esperado	Fase
CP-26	Sincronizar con internet	Todo SINCRONIZADO; sesión SYNCED	F4
CP-27	Repetir la sincronización	Sin duplicados en el backend	F4
CP-28	Token vencido durante la sincronización	Se renueva y continúa	F4
CP-29	Error definitivo (409) en una captura	ERROR_SINCRONIZACION visible; el resto continúa	F4
CP-30	Batería o espacio bajo el umbral crítico	Pausa automática con mensaje	F2
CP-31	Minimizar la app durante una pasada	Pausa con motivo SEGUNDO_PLANO; concilia al volver	F2/F5
CP-32	Cerrar sesión de usuario con una sesión de monitoreo abierta	Bloqueado (RN-19)	F1
CP-33	Dispositivo revocado por el administrador	Al conectarse: tokens y verificador borrados; datos de campo intactos	F4
CP-34	Liberar una cámara desconectada y vincular otro celular en su rol	El nuevo celular entra; se exige nueva prueba corta; el anterior queda liberado en la sesión	F3
CP-35	Cerrar la sesión con una foto pendiente y vincular la cámara a una sesión nueva del mismo controlador	La foto llega una vez, la secuencia se completa y se sincroniza (una sesión SYNCED vuelve a CLOSED y luego a SYNCED)	F2 / F4
CP-36	Entrar sin internet y luego recuperar la conexión	Aparece el aviso; tras escribir la contraseña el modo pasa a ONLINE sin salir de la pantalla	F1
CP-37	Prueba corta con una cámara tapada y con una cámara apagada	Tapada: aprueba con aviso de calidad; apagada: reprueba con el motivo	F1 (simulador) / F2

21.3 Prueba técnica obligatoria de comunicación local

Criterio	Qué se demuestra
Conexión local	Controlador y cámara se comunican por Wi-Fi sin internet.
Orden	El controlador envía CAPTURE_COMMAND y recibe la respuesta.
Captura	La cámara toma una foto real y la guarda.
Calidad	La cámara evalúa y reporta el resultado.
Transferencia	La foto llega al controlador (opción A o B) y se verifica el md5.
Persistencia	El controlador guarda archivo y registro en SQLite.
Corte	Se corta la red y la foto se conserva en la cámara.
Reconexión	Se reintenta con el mismo captureId sin duplicados.
Tres teléfonos	Misma sequenceId con dos captureId; la falla de una cámara no rompe la pasada.
Multiplataforma	Todo lo anterior en Android (F2, F3) y en iOS (F5).

21.4 Fallos inducidos

Apagar el router en plena transferencia; cerrar la app del controlador y de una cámara durante una pasada; cerrar la sesión con fotos pendientes; liberar un rol y reemplazar la cámara; repetir a propósito un captureId y un messageId; simular tiempo agotado y caída del backend; llenar el almacenamiento hasta el umbral y bajar la batería hasta el aviso; minimizar la app; negar el permiso de red local en iPhone; usar un QR antiguo; adelantar la hora de una cámara 5 s (desfase).

21.5 Combinaciones de dispositivos

Combinación	Objetivo	Fase
Controlador Android + 2 cámaras Android	Flujo principal y configuración del piloto	F2, F3, F4, F6
Controlador iPhone + 2 cámaras Android	Servidor local en iOS	F5
Controlador Android + cámara iPhone + cámara Android	Rol de cámara en iOS e interoperabilidad	F5
Tres iPhone (si el equipo los tiene)	Validación completa en iOS	F5 (opcional)

21.6 Evidencia de prueba

Cada prueba registra: versión de app, backend y configuración; celulares; configuración del soporte; fecha; lote, hilera y lateral; pasos; resultado; diagnóstico exportado y capturas de pantalla. Se guarda en docs/evidencias/ con el ID del caso.

22. Plan de construcción por tareas

Cada tarea es un pedido a la IA (plantilla 23.1) y termina con el reporte 23.2. El orden respeta las dependencias. Fechas de referencia: F0 1–2 oct; F1 3–9 oct; F2 12–16 oct; F3 19–23 oct; F4 26–29 oct; F5 30 oct–2 nov; F6 3–6 nov; piloto 9–18 nov. Si la Fase 1 se atrasa, T-06 (registro, recuperación y cambio de contraseña desde Ajustes) es la única tarea que puede pasar a la Fase 2: las demás son base de las fases siguientes.

Definición de terminado (todas las tareas): criterios de aceptación cumplidos; npm run typecheck, npm run lint y npm test sin errores; probado en el celular de la fase; sin TODO en rutas críticas; reporte 23.2 entregado; sección 24 actualizada.

Fase 0 — Preparación

T-00 · Proyecto base y herramientas

- Objetivo: crear riachuelo-mobile con el SDK 54, TypeScript estricto, Expo Router, Jest, ESLint y Prettier, alias @/, estructura de carpetas (6.2), app.json y eas.json de referencia (5.4, 5.5), .env.example, AGENTS.md y este archivo en docs/.
- Secciones: 1, 5, 6.
- Aceptación: npx expo start abre la app en el iPhone (Expo Go SDK 54) y en Android; los tres scripts pasan con una prueba de humo.
- No incluye: lógica de negocio.

T-01 · Contratos, configuración y código de referencia

- Objetivo: copiar exactamente los archivos de las secciones 11, 14.5, 15.3, 17 y 19 y de los Anexos A, B (solo quality.ts) y C; crear domain/ids.ts (newId() con Crypto.randomUUID()), domain/time.ts (nowIso()), protocol/schemas.ts (un esquema zod por mensaje) y protocol/envelope.ts (createEnvelope, parseEnvelope); pasar las pruebas del Anexo E a Jest.
- Aceptación: todas las pruebas de referencia pasan en Jest; los esquemas zod aceptan los ejemplos válidos y rechazan los inválidos.
- Depende de: T-00.

Fase 1 — Base en iPhone con Expo Go

T-02 · Base de datos y almacenamiento

- Objetivo: storage/db.ts (12.2), migrations/001_initial.ts (12.1), repositorios con conversión snake_case ↔ camelCase, files.ts (rutas 12.4, mover a captures/, md5, Paths.availableDiskSpace) y secureStore.ts (claves 12.3).
- Aceptación: migra en un celular nuevo y en uno con datos; PRAGMA user_version = 1; una pantalla de desarrollo (solo __DEV__) muestra conteos por tabla y crea datos de prueba (una sesión abierta y una captura pendiente de transferir); reglas 12.5 cumplidas.
- Depende de: T-01.

T-03 · Identidad del equipo, permisos y diagnóstico base

- Objetivo: deviceIdentity.ts (RF-11) con expo-device y expo-application; permissions.ts con funciones genéricas para consultar y pedir los permisos de cámara y ubicación (la pantalla PANT-07 va en T-07); eventLog.ts; exportDiagnostics.ts con expo-sharing (RF-45).
- Aceptación: el deviceId persiste; las funciones de permisos devuelven el estado real; el diagnóstico se comparte sin datos sensibles.
- Depende de: T-02.

T-04 · Cliente API y backend simulado

- Objetivo: api/httpClient.ts (tiempo de espera con AbortController, cabeceras, errores ApiErrorBody → ApiErrorCode), authApi.ts, bootstrapApi.ts, syncApi.ts y api/mock/ (15.6) con los datos del Anexo D.
- Aceptación: con EXPO_PUBLIC_USE MOCK_API=1 todas las funciones de 15.2 responden como dice el contrato, incluidos los errores de las cuentas de prueba.
- Depende de: T-01.

T-05 · Autenticación (con y sin internet)

- Objetivo: auth/authService.ts (7.4, 7.5, 7.8, 7.11), tokenStore.ts, integración de offlineAuth.ts (medir y registrar PBKDF2_MS), authStore.ts (zustand), aviso y ventana de revalidación al volver internet (7.5), PANT-01 (arranque y redirección de 6.4), rutas protegidas con Stack.Protected (8.11), PANT-02, PANT-06 y cabecera (usuario, modo, función).
- Requerimientos: RF-02 a RF-06, RF-09, RF-10; RN-01 a RN-04, RN-19, RN-20.
- Aceptación: CP-01 a CP-07 y CP-36 con el backend simulado; la guarda de RN-19 con pruebas unitarias (CP-32 se ejecuta en T-13, cuando ya existen sesiones de monitoreo).
- Depende de: T-02, T-03, T-04.

T-06 · Registro, recuperación y cambio de contraseña

- Objetivo: PANT-03, PANT-04, PANT-05 y el cambio desde Ajustes (7.3, 7.6, 7.7).
- Requerimientos: RF-01, RF-07, RF-08.
- Aceptación: CP-08 y CP-09; validaciones de 7.3 por campo.
- Depende de: T-05, T-07.

T-07 · Función del dispositivo, permisos y Ajustes

- Objetivo: PANT-08, PANT-07 (permisos según la función, en ese orden) y PANT-09 completos (7.9, 7.12).
- Requerimientos: RF-12, RF-13, RF-14; RN-15.
- Aceptación: CP-10 y CP-12; CP-11 con datos de prueba creados en la pantalla de desarrollo de T-02 (se repite en T-13).
- Depende de: T-03, T-05.

T-08 · Catálogos

- Objetivo: descarga y reemplazo atómico (12.5), PANT-11 y selectores encadenados lote → hilera → segmento → marcador.
- Requerimientos: RF-15, RF-16.
- Aceptación: CP-13; aviso de catálogos antiguos.
- Depende de: T-04, T-05.

T-09 · Dominio del recorrido (sin interfaz)

- Objetivo: `controller/sessionService.ts`, `passService.ts`, `sequenceEngine.ts` y `domain/rules.ts` (RN-06, RN-07, RN-10 a RN-13 y RN-21 como funciones puras), con reloj, IDs y transporte inyectados; repeticiones con `retake_requests`, cambios de marcador con `marker_changes`, «Repetir lateral» y la definición de sesión abierta (7.12).
- Requerimientos: RF-17, RF-20 a RF-22, RF-24 a RF-27 (lógica).
- Aceptación: pruebas de todas las transiciones de la sección 13, de RN-10 (inicio y reanudación), de RN-11 (nunca dos órdenes superpuestas), de la repetición y del cierre con fotos pendientes.
- Depende de: T-01, T-02.

T-10 · GPS, batería, espacio y pantalla encendida (controlador)

- Objetivo: `gpsService.ts` (`watchPositionAsync` con `Accuracy.Highest`, `timeInterval` y última lectura con antigüedad), `batteryService.ts`, `storageInfo.ts`, `keepAwake.ts`.
- Requerimientos: RF-38, RF-46; RN-18.
- Aceptación: la secuencia recibe la lectura con antigüedad $\leq \text{gps.maxAgeMs}$ o ninguna; avisos de batería y espacio con los umbrales de configuración.
- Depende de: T-02.

T-11 · Calidad de imagen

- Objetivo: `device/qualityPipeline.ts` (16.2, Anexo B) con fotos de muestra de `assets/sample/` (útil, oscura, borrosa).
- Requerimientos: RF-33.
- Aceptación: las tres muestras dan `UTILIZABLE`, `REPETIR_EXPOSICION` y `REPETIR_NITIDEZ`; se registra `duration_ms` en el iPhone (meta ≤ 2 s).
- Depende de: T-01, T-02.

T-12 · Captura en la cámara y modo prueba

- Objetivo: `device/cameraService.ts` (`CameraView` y `takePictureAsync` (`{ quality, exif: false, shutterSound }`), mover a `captures/`, md5 y tamaño) y PANT-32.
- Requerimientos: RF-29, RF-32, RF-33; RN-05.
- Aceptación: en el iPhone se toma una foto real, se guarda y se evalúa; el modo prueba no crea datos de sesión; la pantalla no se apaga.
- Depende de: T-11.

T-13 · Simulador de cámaras y pantallas del controlador

- Objetivo: `local-network/simulator/` (6.3), `factory.ts` y PANT-10, 12, 13 (QR simulado), 14 (prueba corta con las cámaras virtuales, 14.12), 15, 16, 17, 18, 19 y 21; recorrido completo en Expo Go.
- Requerimientos: RF-18, RF-19, RF-23 y RF-42 (simulados), RF-21, RF-22, RF-24 a RF-27, RF-43, RF-46, RF-47, RF-48.
- Aceptación: CP-11, CP-14, CP-15, CP-23, CP-24, CP-32 y CP-37 (simulado) en un iPhone con Expo Go.
- Depende de: T-07, T-08, T-09, T-10, T-11.

T-14 · Recuperación tras cierre o reinicio

- Objetivo: 8.10 en el controlador con el simulador (sesión, pasada, secuencias, colas, servidores simulados y cámaras en DESCONECTADA hasta reconectar). La recuperación de la cámara real va en T-16 (camera_context, pairing.token, reconexión) y T-17 (capturas en CAPTURED, cola de transferencias).
- Requerimientos: RF-44.
- Aceptación: CP-16; ningún dato perdido.
- Depende de: T-12, T-13.
- Cierre de fase: etiqueta v0.1.

Fase 2 — Red local con dos teléfonos (Android, Development Build)

T-15 · Development Build

- Objetivo: instalar expo-dev-client y react-native-tcp-socket 6.4.3; compilar el APK de desarrollo (EAS o Android Studio); factory.ts usa la red real fuera de Expo Go.
- Aceptación: la app abre en Android con Development Build y sigue funcionando en Expo Go con el simulador.
- Depende de: T-14.

T-16 · Control local real y emparejamiento

- Objetivo: tcp/tcpControlServer.ts (14.11, Anexo A), client/wsControlClient.ts, camera/cameraAgent.ts (lado cámara del protocolo y recuperación de camera_context y pairing.token), emparejamiento real (14.6), PANT-30 y PANT-31, heartbeat, estados de enlace, ACK y reenvíos, códigos de ERROR (14.4), reconexión y RESYNC paginado (14.9), cierre del contexto de la cámara (8.12), «Liberar CÁMARA X», reglas 14.3, y recuperación de 8.10 con la red real.
- Requerimientos: RF-28, RF-31, RF-34, RF-36, RF-44; RN-16, RN-17.
- Aceptación: CP-17, CP-18 y CP-20 con un controlador y una cámara Android; cerrar y abrir la app de cada uno durante una pasada sin perder datos.
- Depende de: T-15.

T-17 · Transferencia de fotos

- Objetivo: camera/captureService.ts (14.7, pasos 4 a 6, y reevaluación de capturas en CAPTURED de 8.10); opción A (14.8): client/uploadFileSender.ts con createUploadTask, tcp/tcpFileReceiver.ts con HttpUploadConnection y sink sobre FileHandle, camera/transferQueue.ts (prioridades, classifyLocalTransfer, reintentos y estados de la sección 13, envío solo al controlador de la sesión de cada foto, reintento manual), recepción de fotos tardías (8.13), ACK FILE. Si la opción A falla en los criterios, prototipar la B.
- Requerimientos: RF-35, RF-37; RN-22.
- Aceptación: CP-21; registrar la decisión A/B como ADR (23.3).
- Depende de: T-16.

T-18 · Integración de dos teléfonos y prueba corta

- Objetivo: orden de captura real de punta a punta (14.7), prueba corta real (14.12) en PANT-14, pausa automática (incluida la de segundo plano) y medición de latencia.
- Requerimientos: RF-19, RF-23, RF-30.
- Aceptación: criterios 21.3 en Android (salvo «tres teléfonos»); CP-19, CP-25, CP-30, CP-31, CP-35 (sin la parte de sincronización) y CP-37.
- Depende de: T-17.
- Cierre de fase: etiqueta v0.2.

Fase 3 — Tres teléfonos

T-19 · Dos cámaras reales y monitor

- Objetivo: `sequenceId` compartida con dos `captureId`, concurrencia, falla de una cámara, modos manual, automático y mixto, desfase de reloj (14.9), retención en la cámara con `syncedSessionIds`, monitor completo.
- Requerimientos: RF-22, RF-42; RN-09.
- Aceptación: criterio «tres teléfonos» de 21.3; CP-22 y CP-34; desfase medido por secuencia.
- Depende de: T-18.
- Cierre de fase: etiqueta v0.3.

Fase 4 — Sincronización

T-20 · Cola de sincronización con Spring Boot

- Objetivo: `sync/syncQueue.ts`, `sync/syncService.ts` y PANT-20 según 15.5 (elementos creados al cerrar, lotes de `sync.batchSize`, fotos tardías), con subida multipart por `uploadAsync`.
- Requerimientos: RF-39 a RF-41.
- Aceptación: CP-26 a CP-29, CP-33 y CP-35 (con sincronización) contra el backend real (o el simulado si aún no está listo).
- Depende de: T-19 y la disponibilidad de los endpoints.
- Cierre de fase: etiqueta v0.4.

Fase 5 — Validación en iOS

T-21 · Las tres funciones en iPhone

- Objetivo: compilar Release con Xcode (Apple ID gratuito), validar el permiso de red local, ATS (Q-07), pausa en segundo plano y las combinaciones de 21.5; corregir problemas.
- Aceptación: criterio «multiplataforma» de 21.3; problemas resueltos o documentados con evidencia.
- Depende de: T-20.
- Cierre de fase: etiqueta v0.5.

Fase 6 — Campo y versión del piloto

T-22 · Calibración en campo

- Objetivo: APK preview; calibrar el perfil Q1 (16.5), el intervalo, `gps.warnAccuracyM` y las iteraciones de PBKDF2; medir batería y espacio por jornada.
- Aceptación: perfil Q1 y configuración CFG-2 documentados con evidencia.
- Depende de: T-21.

T-23 · Versión congelada del piloto

- Objetivo: etiqueta v1.0.0-piloto; regresión completa (21.2); este archivo actualizado; decisiones cerradas.
- Aceptación: todos los CP aplicables aprobados en los tres Android del piloto; durante el piloto no se cambia el código salvo una corrección crítica registrada.
- Depende de: T-22.

23. Plantillas

23.1 Pedido a la IA

Tarea: T-XX – <nombre> (sección 22 de docs/MAESTRO_APP_MOVIL.md).
Lee primero las secciones 1 a 7 y las que cita la tarea. Cumple las reglas R-01 a R-20.
Estado actual del repositorio: <pegar la fila de la sección 24 y los cambios recientes>.
Entrega: código completo de los archivos de la tarea, sus pruebas y el reporte de la sección 23.2.
No modifiques los contratos (secciones 11, 12, 14, 15 y 17). Si algo no está definido, detente y pregunta.

23.2 Reporte de avance (lo entrega la IA)

Tarea: T-XX
Hecho: <resumen>
Archivos creados o modificados: <lista>
Cómo probar: <comandos y pasos en el celular>
Resultado de typecheck / lint / test: <resultado>
Criterios de aceptación: <uno por uno: cumplido / pendiente y por qué>
Supuestos tomados: <lista o "ninguno">
Preguntas para el equipo: <lista o "ninguna">
Siguiente tarea sugerida: <T-YY>

23.3 Registro de decisión (ADR) — docs/adr/ADR-NNN-titulo.md

ADR-NNN – <título>
Fecha: <dd/mm/aaaa> · Estado: propuesta | aceptada | sustituida por ADR-MMM
Contexto: <problema y restricciones>
Decisión: <qué se decidió>
Alternativas consideradas: <opciones y por qué no>
Consecuencias: <qué cambia en el código y en este archivo>
Evidencia: <pruebas, mediciones, enlaces a docs/evidencias/>

24. Registro de avance

Tarea	Fase	Estado	Fecha	Responsable	Evidencia o commit	Notas
T-00	F0	Pendiente				
T-01	F0	Pendiente				
T-02	F1	Pendiente				
T-03	F1	Pendiente				
T-04	F1	Pendiente				
T-05	F1	Pendiente				
T-06	F1	Pendiente				
T-07	F1	Pendiente				
T-08	F1	Pendiente				
T-09	F1	Pendiente				
T-10	F1	Pendiente				
T-11	F1	Pendiente				
T-12	F1	Pendiente				
T-13	F1	Pendiente				
T-14	F1	Pendiente				
T-15	F2	Pendiente				
T-16	F2	Pendiente				
T-17	F2	Pendiente				
T-18	F2	Pendiente				
T-19	F3	Pendiente				
T-20	F4	Pendiente				
T-21	F5	Pendiente				

Tarea	Fase	Estado	Fecha	Responsable	Evidencia o commit	Notas
T-22	F6	Pendiente				
T-23	F6	Pendiente				

Estados: Pendiente · En curso · Hecho · Bloqueado (indicar por qué).

25. Decisiones registradas y preguntas abiertas

25.1 Versiones de este archivo

Versión	Fecha	Cambio
1.0	01/10/2026	Primera versión: login obligatorio en los tres celulares, registro con aprobación, login sin internet, recuperación vía administrador, contratos validados y plan de tareas. Incluye las 25 correcciones de la revisión independiente (0.5).

25.2 Decisiones técnicas (ADR)

ADR	Decisión	Estado
ADR-001	Expo SDK 54 fijo (D-03).	Aceptada
ADR-002	Cuentas: login obligatorio, registro con aprobación, login sin internet de 7 días, recuperación vía administrador (D-04 a D-07).	Aceptada
ADR-003	expo-network en lugar de NetInfo (D-19).	Aceptada (este archivo)
ADR-004	Subidas nativas con uploadAsync de expo-file-system/legacy (D-11, D-22).	Aceptada (este archivo)
ADR-005	PBKDF2 con @noble/hashes 1.8.0 (D-20).	Aceptada (este archivo)
ADR-006	Transferencia de fotos: opción A o B (D-11).	Pendiente (Fase 2)
ADR-007	Servidor de control con react-native-tcp-socket y códec propio (D-10).	Pendiente (Fase 2)

25.3 Preguntas abiertas

ID	Pregunta	Quién decide	Antes de
Q-01	Códigos definitivos de lotes, hileras, segmentos y marcadores.	Equipo + empresa	F6 (idealmente F1)
Q-02	¿El ADMINISTRADOR puede operar en campo o solo dar soporte? (Propuesto: puede).	Equipo	F1
Q-03	Texto final del aviso de privacidad y datos pedidos en el registro (celular, código de trabajador).	Equipo + empresa	F1
Q-04	¿getIpAddressAsync() devuelve la IP correcta con el hotspot del controlador en los modelos del piloto? (si no, entrada manual o router).	Prueba F2	F2
Q-05	Política de retención final y espacio mínimo.	Equipo	F6
Q-06	Tamaño del JWT (≤ 2 KB) y duración del refresh token (≥ 14 días).	Backend	F1
Q-07	¿NSAllowsLocalNetworking basta para HTTP a direcciones IP en iOS?	Prueba F5	F5
Q-08	Modelos de los celulares del piloto y si las dos cámaras serán del mismo modelo.	Equipo	F2
Q-09	Umbral Q1 e intervalo automático.	Calibración F6	F6
Q-10	Iteraciones de PBKDF2 según el tiempo medido.	Medición F1	F1
Q-11	¿El backend publicará qualityProfile en el bootstrap?	Backend	F4
Q-12	Identificadores definitivos de la app (bundleIdentifier, package).	Equipo	F2
Q-13	Confirmar que el backend permite al mismo usuario en varios dispositivos a la vez.	Backend	F1
Q-14	Confirmar que el backend acepta actualizar pasadas y secuencias ya sincronizadas (fotos tardías, 8.13) y sesiones cerradas sin pasadas.	Backend	F4

26. Glosario

Término	Definición
ACK	Confirmación de que un mensaje o una foto se recibió y guardó.
APK preview	Instalador de Android con el código incluido; funciona sin laptop ni internet.
Bootstrap	Descarga de catálogos y parámetros para trabajar sin internet.
captureId / sequenceId / passId / sessionId	Identificadores de una foto, de una orden coordinada, de una pasada por un lateral y de la sesión de monitoreo.
Desfase	Diferencia de reloj o de momento de captura entre las dos cámaras.
Development Build	Versión de la app que incluye librerías nativas propias; necesita Metro para cargar el código.
Expo Go	App de Expo para probar sin compilar; solo trae los módulos de Expo.
Foto tardía	Foto que llega al controlador después de cerrar su sesión; se acepta y se sincroniza (8.13).
Función del dispositivo	Papel del celular en la sesión: CONTROLADOR, CAMERA_1 o CAMERA_2. No es el rol del usuario.
Heartbeat	Mensaje periódico que confirma que un celular sigue conectado.
Idempotencia	Repetir una operación no crea duplicados.
LATERAL_A / LATERAL_B	Códigos de las dos caras de una hilera.
Liberar un rol	Acción del controlador que deja libre el rol de una cámara desconectada para reemplazarla por otro celular.
PBKDF2	Función que deriva un verificador a partir de la contraseña de forma lenta, para no guardar la contraseña.
Perfil de calidad	Conjunto versionado de umbrales de exposición y nitidez (Q0, Q1...).
Prueba corta	Captura de prueba en ambas cámaras antes de la primera pasada para comprobar orden, calidad, transferencia y ACK; no es evidencia (14.12).
Refresh token	Token de larga duración para renovar el token de acceso sin volver a escribir la contraseña.
RESYNC	Conciliación entre controlador y cámara después de una reconexión.
Resultado por cámara	Estado de cada cámara dentro de una secuencia (pendiente, aceptada, recibida, rechazada, error, sin respuesta).
Verificador sin internet	Dato derivado de la contraseña que permite comprobarla en el celular sin guardarla.
Ventana sin internet	Días durante los cuales se puede entrar sin internet desde la última validación con el backend.

27. Fuentes y trazabilidad

Documentos del proyecto: Especificación técnico-funcional v2.0; Stack Tecnológico v2.1; documento académico «Curso Integrador II»; archivo «MAPEO CHANCHITO MOLINOS actualizado»; Especificación técnica, stack y ruta de construcción de la aplicación móvil v1.0; decisiones del equipo del 01/10/2026 (D-02, D-04 a D-08, D-15, D-16).

Documentación y paquetes consultados el 01/10/2026:

1. Expo SDK 54 (versiones, Xcode, Node): <https://expo.dev/changelog/sdk-54>
2. Referencia del SDK 54: <https://docs.expo.dev/versions/v54.0.0/>
3. Expo Go y versiones del SDK: <https://docs.expo.dev/troubleshooting/expo-go-version-mismatch/>
4. Instalación de Expo Router: <https://docs.expo.dev/router/installation/>
5. Pruebas con Jest: <https://docs.expo.dev/develop/unit-testing/>
6. ESLint y Prettier: <https://docs.expo.dev/guides/using-eslint/>

7. Development Build: <https://docs.expo.dev/develop/development-builds/create-a-build/>
8. Distribución interna con EAS: <https://docs.expo.dev/build/internal-distribution/>
9. Registro npm (versiones del SDK 54 en expo@54.0.37/bundledNativeModules.json y declaraciones de cada paquete): <https://registry.npmjs.org/>
10. react-native-tcp-socket (servidor TCP; issue #207 con un servidor WebSocket): <https://github.com/Rapssito/react-native-tcp-socket>
11. @noble/hashes: <https://github.com/paulmillr/noble-hashes>
12. jpeg-js: <https://github.com/eugeneware/jpeg-js>
13. react-native-static-server (opción B): <https://github.com/birdofpreyru/react-native-static-server>
14. RFC 6455, protocolo WebSocket: <https://www.rfc-editor.org/rfc/rfc6455>
15. Cuentas de Apple (gratuita y de pago): <https://developer.apple.com/support/compare-memberships/>

Anexo A. Código de referencia probado: red local

A.1 Servidor WebSocket mínimo — src/local-network/wsCodec.ts

Probado con un cliente WebSocket real (paquete ws): clave de la RFC 6455, frames partidos byte a byte, longitudes de 7 y 16 bits, mensajes de hasta 65 000 bytes, mensajes fragmentados, ping/pong, cierre ordenado, handshake inválido (400) y mensaje sobre el límite (cierre 1009).

```
// src/local-network/wsCodec.ts — Servidor WebSocket mínimo (RFC 6455) sobre un socket TCP crudo.
// Sin dependencias de plataforma: recibe bytes (Uint8Array) y entrega bytes. En la app, el socket es
// react-native-tcp-socket y sha1Base64 usa expo-crypto (digestStringAsync SHA-1, codificación BASE64).
export const WS_GUID = '258EAF45-E914-47DA-95CA-C5AB0DC85B11';
export const OPCODE = { CONTINUATION: 0x0, TEXT: 0x1, BINARY: 0x2, CLOSE: 0x8, PING: 0x9, PONG: 0xa } as const;
export type Sha1Base64 = (input: string) => Promise<string>;
export class WsProtocolError extends Error {
  constructor(message: string, public readonly closeCode: number = 1002) {
    super(message);
    this.name = 'WsProtocolError';
  }
}
export interface HttpRequestHead {
  method: string;
  path: string;
  headers: Record<string, string>; // claves en minúsculas
  headerBytes: number; // bytes ocupados por la cabecera, incluido \r\n\r\n
}
function asciiDecode(bytes: Uint8Array): string {
  let s = '';
  for (let i = 0; i < bytes.length; i++) s += String.fromCharCode(bytes[i]);
  return s;
}
function asciiEncode(text: string): Uint8Array {
  const out = new Uint8Array(text.length);
  for (let i = 0; i < text.length; i++) out[i] = text.charCodeAt(i) & 0xff;
  return out;
}
export function concatBytes(a: Uint8Array, b: Uint8Array): Uint8Array {
  if (a.length === 0) return b;
  if (b.length === 0) return a;
  const out = new Uint8Array(a.length + b.length);
  out.set(a, 0);
  out.set(b, a.length);
  return out;
}
/** Devuelve la cabecera HTTP si ya llegó completa (termina en \r\n\r\n); si no, null. */
export function parseHttpHead(bytes: Uint8Array, maxHeaderBytes = 8192): HttpRequestHead | null {
  const limit = Math.min(bytes.length, maxHeaderBytes);
  for (let i = 3; i < limit; i++) {
    if (bytes[i - 3] === 13 && bytes[i - 2] === 10 && bytes[i - 1] === 13 && bytes[i] === 10) {
      const lines = asciiDecode(bytes.subarray(0, i - 3)).split('\r\n');
      const [method = '', path = ''] = (lines[0] ?? '').split(' ');
      const headers: Record<string, string> = {};
      for (const line of lines.slice(1)) {
        const idx = line.indexOf(':');
        if (idx > 0) headers[line.slice(0, idx).trim().toLowerCase()] = line.slice(idx + 1).trim();
      }
      return { method, path, headers, headerBytes: i + 1 };
    }
  }
}
```

```

    }
  }
  if (bytes.length >= maxHeaderBytes) throw new WsProtocolError('Cabecera HTTP demasiado grande', 1009);
  return null;
}
export async function computeAcceptKey(secWebSocketKey: string, sha1Base64: Sha1Base64): Promise<string> {
  return sha1Base64(secWebSocketKey + WS_GUID);
}
export function encodeFrame(opcode: number, payload: Uint8Array, fin = true): Uint8Array {
  const len = payload.length;
  const headerLen = len < 126 ? 2 : len <= 0xffff ? 4 : 10;
  const out = new Uint8Array(headerLen + len);
  out[0] = (fin ? 0x80 : 0) | (opcode & 0x0f);
  if (len < 126) {
    out[1] = len;
  } else if (len <= 0xffff) {
    out[1] = 126;
    out[2] = (len >> 8) & 0xff;
    out[3] = len & 0xff;
  } else {
    out[1] = 127;
    const hi = Math.floor(len / 0x100000000);
    const lo = len >> 0;
    out[2] = (hi >> 24) & 0xff; out[3] = (hi >> 16) & 0xff; out[4] = (hi >> 8) & 0xff; out[5] = hi & 0xff;
    out[6] = (lo >> 24) & 0xff; out[7] = (lo >> 16) & 0xff; out[8] = (lo >> 8) & 0xff; out[9] = lo & 0xff;
  }
  out.set(payload, headerLen);
  return out;
}
export interface WsFrame { fin: boolean; opcode: number; payload: Uint8Array; }
/** Decodifica frames del cliente (enmascarados) aunque lleguen partidos en varios bloques TCP. */
export class FrameDecoder {
  private buf: Uint8Array = new Uint8Array(0);
  constructor(private readonly maxPayload = 64 * 1024, private readonly requireMask = true) {}
  push(chunk: Uint8Array): WsFrame[] {
    this.buf = concatBytes(this.buf, chunk);
    const frames: WsFrame[] = [];
    for (;;) {
      const buf = this.buf;
      if (buf.length < 2) break;
      const b0 = buf[0];
      const b1 = buf[1];
      if ((b0 & 0x70) !== 0) throw new WsProtocolError('Bits RSV activos sin extensión negociada');
      const fin = (b0 & 0x80) !== 0;
      const opcode = b0 & 0x0f;
      const masked = (b1 & 0x80) !== 0;
      if (this.requireMask && !masked) throw new WsProtocolError('Los frames del cliente deben venir enmascarados');
      let len = b1 & 0x7f;
      let offset = 2;
      if (len === 126) {
        if (buf.length < 4) break;
        len = (buf[2] << 8) | buf[3];
        offset = 4;
      } else if (len === 127) {
        if (buf.length < 10) break;
        const hi = ((buf[2] << 24) | (buf[3] << 16) | (buf[4] << 8) | buf[5]) >> 0;
        const lo = ((buf[6] << 24) | (buf[7] << 16) | (buf[8] << 8) | buf[9]) >> 0;
        if (hi !== 0) throw new WsProtocolError('Frame demasiado grande', 1009);
        len = lo;
        offset = 10;
      }
      if (len > this.maxPayload) throw new WsProtocolError('Frame demasiado grande', 1009);
      const isControl = (opcode & 0x08) !== 0;
      if (isControl && (!fin || len > 125)) throw new WsProtocolError('Frame de control inválido');
      const maskLen = masked ? 4 : 0;
      if (buf.length < offset + maskLen + len) break;
      const start = offset + maskLen;
      const payload = buf.slice(start, start + len);
      if (masked) {
        const m0 = buf[offset], m1 = buf[offset + 1], m2 = buf[offset + 2], m3 = buf[offset + 3];
        for (let i = 0; i < payload.length; i++) {
          const k = i & 3;
          payload[i] ^= k === 0 ? m0 : k === 1 ? m1 : k === 2 ? m2 : m3;
        }
      }
      frames.push({ fin, opcode, payload });
      this.buf = buf.slice(start + len);
    }
    return frames;
  }
}
export type WsEvent =

```

```

    | { kind: 'text'; text: string }
    | { kind: 'binary'; data: Uint8Array }
    | { kind: 'ping'; data: Uint8Array }
    | { kind: 'pong'; data: Uint8Array }
    | { kind: 'close'; code: number; reason: string };
/** Une frames fragmentados en mensajes completos; los frames de control pasan directo. */
export class MessageAssembler {
    private parts: Uint8Array[] = [];
    private size = 0;
    private opcode: number | null = null;
    private readonly decoder = new TextDecoder();
    constructor(private readonly maxMessage = 64 * 1024) {}
    accept(frame: WsFrame): WsEvent | null {
        switch (frame.opcode) {
            case OPCODE.PING:
                return { kind: 'ping', data: frame.payload };
            case OPCODE.PONG:
                return { kind: 'pong', data: frame.payload };
            case OPCODE.CLOSE: {
                const p = frame.payload;
                const code = p.length >= 2 ? (p[0] << 8) | p[1] : 1005;
                const reason = p.length > 2 ? this.decoder.decode(p.subarray(2)) : '';
                return { kind: 'close', code, reason };
            }
            case OPCODE.TEXT:
            case OPCODE.BINARY:
                if (this.opcode !== null) throw new WsProtocolError('Nuevo mensaje antes de terminar el anterior');
                this.opcode = frame.opcode;
                break;
            case OPCODE.CONTINUATION:
                if (this.opcode === null) throw new WsProtocolError('Continuación sin mensaje iniciado');
                break;
            default:
                throw new WsProtocolError(`Opcode no soportado: ${frame.opcode}`);
        }
        this.size += frame.payload.length;
        if (this.size > this.maxMessage) throw new WsProtocolError('Mensaje demasiado grande', 1009);
        this.parts.push(frame.payload);
        if (!frame.fin) return null;
        const data = new Uint8Array(this.size);
        let off = 0;
        for (const part of this.parts) {
            data.set(part, off);
            off += part.length;
        }
        const opcode = this.opcode;
        this.parts = [];
        this.size = 0;
        this.opcode = null;
        return opcode === OPCODE.TEXT ? { kind: 'text', text: this.decoder.decode(data) } : { kind: 'binary', data };
    }
}
export interface WsConnectionHandlers {
    onOpen(head: HttpRequestHead): void;
    onText(text: string): void;
    onClose(code: number, reason: string): void;
    onError(error: Error): void;
}
/**
 * Conexión de servidor: valida el handshake, responde 101, decodifica mensajes, responde PING con PONG
 * y completa el cierre. `write` envía bytes por el socket; `end` cierra el socket.
 */
export class WsServerConnection {
    private state: 'HANDSHAKE' | 'OPEN' | 'CLOSING' | 'CLOSED' = 'HANDSHAKE';
    private pending: Uint8Array = new Uint8Array(0);
    private queue: Promise<void> = Promise.resolve();
    private readonly decoder: FrameDecoder;
    private readonly assembler: MessageAssembler;
    private readonly encoder = new TextEncoder();
    constructor(
        private readonly write: (bytes: Uint8Array) => void,
        private readonly end: () => void,
        private readonly sha1Base64: Sha1Base64,
        private readonly handlers: WsConnectionHandlers,
        maxMessageBytes = 64 * 1024,
    ) {
        this.decoder = new FrameDecoder(maxMessageBytes);
        this.assembler = new MessageAssembler(maxMessageBytes);
    }
    get isOpen(): boolean {
        return this.state === 'OPEN';
    }
}

```

```

/** Llamar con cada bloque recibido del socket. Procesa en orden aunque el handshake sea asíncrono. */
onData(chunk: Uint8Array): Promise<void> {
    this.queue = this.queue.then(() => this.process(chunk)).catch((err: unknown) => this.fail(err));
    return this.queue;
}

sendText(text: string): void {
    if (this.state !== 'OPEN') throw new Error('WebSocket no abierto');
    this.write(encodeFrame(OPCODE.TEXT, this.encoder.encode(text)));
}

close(code = 1000, reason = ''): void {
    if (this.state === 'CLOSED' || this.state === 'CLOSING') return;
    if (this.state === 'OPEN') this.write(encodeFrame(OPCODE.CLOSE, this.closePayload(code, reason)));
    this.state = 'CLOSING';
}

private closePayload(code: number, reason: string): Uint8Array {
    const r = this.encoder.encode(reason);
    const p = new Uint8Array(2 + r.length);
    p[0] = (code >>> 8) & 0xff;
    p[1] = code & 0xff;
    p.set(r, 2);
    return p;
}

private async process(chunk: Uint8Array): Promise<void> {
    if (this.state === 'CLOSED') return;
    if (this.state === 'HANDSHAKE') {
        this.pending = concatBytes(this.pending, chunk);
        const head = parseHttpHead(this.pending);
        if (!head) return;
        const h = head.headers;
        const key = h['sec-websocket-key'];
        const valid =
            head.method === 'GET' &&
            (h['upgrade'] ?? '').toLowerCase() === 'websocket' &&
            (h['connection'] ?? '').toLowerCase().includes('upgrade') &&
            h['sec-websocket-version'] === '13' &&
            typeof key === 'string' &&
            key.length > 0;
        if (!valid) {
            this.write(asciiEncode('HTTP/1.1 400 Bad Request\r\nConnection: close\r\nContent-Length: 0\r\n\r\n'));
            this.state = 'CLOSED';
            this.end();
            return;
        }
        const accept = await computeAcceptKey(key, this.shalBase64);
        this.write(
            asciiEncode(
                'HTTP/1.1 101 Switching Protocols\r\nUpgrade: websocket\r\nConnection: Upgrade\r\n' +
                'Sec-WebSocket-Accept: ${accept}\r\n\r\n',
            ),
        );
        this.state = 'OPEN';
        const rest = this.pending.slice(head.headerBytes);
        this.pending = new Uint8Array(0);
        this.handlers.onOpen(head);
        if (rest.length === 0) return;
        chunk = rest;
    }
    for (const frame of this.decoder.push(chunk)) {
        const ev = this.assembler.accept(frame);
        if (!ev) continue;
        if (ev.kind === 'text') this.handlers.onText(ev.text);
        else if (ev.kind === 'ping') this.write(encodeFrame(OPCODE.PONG, ev.data));
        else if (ev.kind === 'close') {
            if (this.state === 'OPEN') this.write(encodeFrame(OPCODE.CLOSE, this.closePayload(ev.code === 1005 ? 1000 : ev
                .code, '')));
            this.state = 'CLOSED';
            this.handlers.onClose(ev.code, ev.reason);
            this.end();
            return;
        }
    }
    // Mensajes binarios y PONG se ignoran en el protocolo v1.
}

private fail(err: unknown): void {
    const error = err instanceof Error ? err : new Error(String(err));
    const code = err instanceof WsProtocolError ? err.closeCode : 1011;
    if (this.state === 'OPEN') this.write(encodeFrame(OPCODE.CLOSE, this.closePayload(code, '')));
    this.state = 'CLOSED';
    this.handlers.onError(error);
    this.end();
}
}

```

A.2 Receptor HTTP de fotos (opción A) — src/local-network/httpUpload.ts

Probado con una foto de 3 MB: md5 verificado, duplicado respondido como ALREADY_RECEIVED tras descartar el cuerpo, md5 erróneo respondido con 422 y soporte de Expect: 100-continue.

```
// src/local-network/httpUpload.ts - Receptor HTTP mínimo para la opción A de transferencia
// (POST /local/captures con el JPEG como cuerpo binario y metadatos en la cabecera X-Capture-Meta).
// Sin dependencias de plataforma: en la app, el socket es react-native-tcp-socket y el sink escribe con FileHandle.
import { concatBytes, parseHttpHead, type HttpRequestHead } from './wsCodec';
export interface UploadResult { status: number; body: unknown; }
export interface UploadSink {
  /** Bloque del cuerpo en orden. */
  write(chunk: Uint8Array): void;
  /** Cuerpo completo: verificar tamaño/hash, registrar en SQLite y devolver la respuesta. */
  finish(): Promise<UploadResult>;
  /** Conexión cortada o error: descartar el archivo temporal. */
  abort(reason: string): void;
}
/** Decide si acepta la petición. Devuelve un sink o una respuesta inmediata de rechazo. */
export type UploadRouter = (head: HttpRequestHead, contentType: string) => Promise<UploadSink | UploadResult>;
const REASONS: Record<number, string> = {
  100: 'Continue', 200: 'OK', 400: 'Bad Request', 403: 'Forbidden', 404: 'Not Found', 409: 'Conflict', 411: 'Length Required',
  413: 'Payload Too Large', 422: 'Unprocessable Entity', 500: 'Internal Server Error', 507: 'Insufficient Storage',
};
function responseBytes(status: number, body: unknown): Uint8Array {
  const json = new TextEncoder().encode(JSON.stringify(body));
  const head =
    `HTTP/1.1 ${status} ${REASONS[status] ?? 'Status'}\r\nContent-Type: application/json\r\n` +
    `Content-Length: ${json.length}\r\nConnection: close\r\n\r\n`;
  const headBytes = new TextEncoder().encode(head);
  return concatBytes(headBytes, json);
}
function isSink(x: UploadSink | UploadResult): x is UploadSink {
  return typeof (x as UploadSink).write === 'function';
}
export class HttpUploadConnection {
  private state: 'HEAD' | 'BODY' | 'DONE' = 'HEAD';
  private pending: Uint8Array = new Uint8Array(0);
  private remaining = 0;
  private sink: UploadSink | null = null;
  private queue: Promise<void> = Promise.resolve();
  constructor(
    private readonly write: (bytes: Uint8Array) => void,
    private readonly end: () => void,
    private readonly route: UploadRouter,
    private readonly maxBodyBytes = 50 * 1024 * 1024,
  ) {}
  onData(chunk: Uint8Array): Promise<void> {
    this.queue = this.queue.then(() => this.process(chunk)).catch((err: unknown) => {
      this.sink?.abort(String(err));
      this.respond(500, { result: 'REJECTED', reason: 'INTERNAL' });
    });
    return this.queue;
  }
  /** Llamar si el socket se cierra antes de terminar. */
  onClose(): void {
    if (this.state === 'BODY') this.sink?.abort('Conexión cerrada antes de recibir todo el cuerpo');
    this.state = 'DONE';
  }
  private respond(status: number, body: unknown): void {
    if (this.state === 'DONE') return;
    this.state = 'DONE';
    this.write(responseBytes(status, body));
    this.end();
  }
  private async process(chunk: Uint8Array): Promise<void> {
    if (this.state === 'DONE') return;
    if (this.state === 'HEAD') {
      this.pending = concatBytes(this.pending, chunk);
      const head = parseHttpHead(this.pending);
      if (!head) return;
      const rest = this.pending.slice(head.headerBytes);
      this.pending = new Uint8Array(0);
      if ((head.headers['transfer-encoding'] ?? '').toLowerCase().includes('chunked')) {
        this.respond(411, { result: 'REJECTED', reason: 'LENGTH_REQUIRED' });
        return;
      }
    }
    const contentType = head.headers['content-type'];
    const contentLength = Number(head.headers['content-length']);
    const maxBodyBytes = this.maxBodyBytes;
    if (contentLength > maxBodyBytes) {
      this.respond(413, { result: 'REJECTED', reason: 'PAYLOAD_TOO_LARGE' });
      return;
    }
    if (contentType === 'application/json') {
      const body = JSON.parse(rest);
      const sink = this.route(head, contentType);
      if (sink instanceof Promise) {
        sink.then((sink) => {
          this.sink = sink;
          this.sink.write(rest);
          this.sink.finish().then((result) => {
            this.respond(result.status, result.body);
          });
        }).catch((err) => {
          this.sink?.abort(String(err));
          this.respond(500, { result: 'REJECTED', reason: 'INTERNAL' });
        });
      } else {
        this.sink = sink;
        this.sink.write(rest);
        this.sink.finish().then((result) => {
          this.respond(result.status, result.body);
        });
      }
    } else {
      this.sink = this.route(head, contentType);
      if (sink instanceof Promise) {
        sink.then((sink) => {
          this.sink = sink;
          this.sink.write(rest);
          this.sink.finish().then((result) => {
            this.respond(result.status, result.body);
          });
        }).catch((err) => {
          this.sink?.abort(String(err));
          this.respond(500, { result: 'REJECTED', reason: 'INTERNAL' });
        });
      } else {
        this.sink = sink;
        this.sink.write(rest);
        this.sink.finish().then((result) => {
          this.respond(result.status, result.body);
        });
      }
    }
  }
}
```

```

if (!Number.isInteger(contentLength) || contentLength <= 0) {
  this.respond(411, { result: 'REJECTED', reason: 'LENGTH_REQUIRED' });
  return;
}
if (contentLength > this.maxBodyBytes) {
  this.respond(413, { result: 'REJECTED', reason: 'PAYLOAD_TOO_LARGE' });
  return;
}
const routed = await this.route(head, contentLength);
if (!isSink(routed)) {
  this.respond(routed.status, routed.body);
  return;
}
if ((head.headers['expect'] ?? '').toLowerCase() === '100-continue') {
  this.write(new TextEncoder().encode('HTTP/1.1 100 Continue\r\n\r\n'));
}
this.sink = routed;
this.remaining = contentLength;
this.state = 'BODY';
if (rest.length === 0) return;
chunk = rest;
}
if (this.state === 'BODY' && this.sink) {
  const take = chunk.length <= this.remaining ? chunk : chunk.subarray(0, this.remaining);
  this.sink.write(take);
  this.remaining -= take.length;
  if (this.remaining === 0) {
    const result = await this.sink.finish();
    this.respond(result.status, result.body);
  }
}
}
}
}

/** base64url → texto UTF-8 (para la cabecera X-Capture-Meta). Sin depender de Buffer. */
export function decodeBase64UrlToText(value: string): string {
  const b64 = value.replace(/-/g, '+').replace(/_/g, '/');
  const alphabet = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/=';
  const clean = b64.replace(/=+$/, '');
  const bytes: number[] = [];
  let buffer = 0;
  let bits = 0;
  for (const ch of clean) {
    const v = alphabet.indexOf(ch);
    if (v < 0) throw new Error('base64url inválido');
    buffer = (buffer << 6) | v;
    bits += 6;
    if (bits >= 8) {
      bits -= 8;
      bytes.push((buffer >> bits) & 0xff);
    }
  }
  return new TextDecoder().decode(new Uint8Array(bytes));
}

/** Texto UTF-8 → base64url sin relleno (lado cámara). */
export function encodeTextToBase64Url(text: string): string {
  const bytes = new TextEncoder().encode(text);
  const alphabet = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789-_';
  let out = '';
  let i = 0;
  for (; i + 2 < bytes.length; i += 3) {
    const n = (bytes[i] << 16) | (bytes[i + 1] << 8) | bytes[i + 2];
    out += alphabet[(n >> 18) & 63] + alphabet[(n >> 12) & 63] + alphabet[(n >> 6) & 63] + alphabet[n & 63];
  }
  const rem = bytes.length - i;
  if (rem === 1) {
    const n = bytes[i] << 16;
    out += alphabet[(n >> 18) & 63] + alphabet[(n >> 12) & 63];
  } else if (rem === 2) {
    const n = (bytes[i] << 16) | (bytes[i + 1] << 8);
    out += alphabet[(n >> 18) & 63] + alphabet[(n >> 12) & 63] + alphabet[(n >> 6) & 63];
  }
  return out;
}
}

```

Anexo B. Código de referencia: calidad de imagen

B.1 Métricas y decisión (probado) — src/device/quality.ts

```
// src/device/quality.ts - Métricas y decisión de calidad (funciones puras, perfil Q0).
// Flujo en la app: (1) copia reducida a analysisWidth → jpeg-js → luminanceStats; si la exposición falla, termina.
// (2) recortes a resolución original (regionRects) → jpeg-js → laplacianVariance → se usa el máximo.
import type { QualityMetrics, QualityReason, QualityResult } from '../domain/types';
export interface RawImage {
  width: number;
  height: number;
  /** Píxeles intercalados RGB (3 canales) o RGBA (4 canales), como entrega jpeg-js con useTArray. */
  data: Uint8Array;
  channels: 3 | 4;
}
export interface ExposureStats { mean: number; darkRatio: number; brightRatio: number; }
export interface QualityProfile {
  version: string;
  exposure: {
    darkLevel: number;
    brightLevel: number;
    minMean: number;
    maxMean: number;
    maxDarkRatio: number;
    maxBrightRatio: number;
  };
  sharpness: { minLaplacianVariance: number };
}
/** Luminancia Rec. 601: Y = 0,299 R + 0,587 G + 0,114 B. */
export function toGray(img: RawImage): Uint8Array {
  const n = img.width * img.height;
  const out = new Uint8Array(n);
  const c = img.channels;
  const d = img.data;
  for (let i = 0, p = 0; i < n; i++, p += c) {
    out[i] = (299 * d[p] + 587 * d[p + 1] + 114 * d[p + 2] + 500) / 1000;
  }
  return out;
}
export function exposureStats(gray: Uint8Array, darkLevel: number, brightLevel: number): ExposureStats {
  let sum = 0;
  let dark = 0;
  let bright = 0;
  for (let i = 0; i < gray.length; i++) {
    const y = gray[i];
    sum += y;
    if (y <= darkLevel) dark++;
    else if (y >= brightLevel) bright++;
  }
  const n = gray.length || 1;
  return { mean: sum / n, darkRatio: dark / n, brightRatio: bright / n };
}
/** Varianza de la respuesta al Laplaciano de 4 vecinos [0 1 0; 1 -4 1; 0 1 0] sobre píxeles interiores. */
export function laplacianVariance(gray: Uint8Array, width: number, height: number): number {
  if (width < 3 || height < 3) return 0;
  let sum = 0;
  let sumSq = 0;
  let count = 0;
  for (let y = 1; y < height - 1; y++) {
    const row = y * width;
    for (let x = 1; x < width - 1; x++) {
      const i = row + x;
      const v = gray[i - width] + gray[i + width] + gray[i - 1] + gray[i + 1] - 4 * gray[i];
      sum += v;
      sumSq += v * v;
      count++;
    }
  }
  const mean = sum / count;
  return sumSq / count - mean * mean;
}
export interface Rect { originX: number; originY: number; width: number; height: number; }
/** Recortes cuadrados sobre la línea media horizontal (25 %, 50 % y 75 % del ancho), dentro de la imagen. */
export function regionRects(imgWidth: number, imgHeight: number, regionSize: number, count: 1 | 2 | 3): Rect[] {
  const size = Math.max(16, Math.min(regionSize, imgWidth, imgHeight));
  const centers = count === 1 ? [0.5] : count === 2 ? [0.33, 0.67] : [0.25, 0.5, 0.75];
  const originY = Math.round((imgHeight - size) / 2);
  return centers.map((c) => {
    const cx = Math.round(imgWidth * c);
    const originX = Math.min(Math.max(0, cx - Math.floor(size / 2)), imgWidth - size);
  });
}
```

```

    return { originX, originY, width: size, height: size };
  });
}
/** Decide el resultado. La exposición se evalúa primero porque una foto oscura también parece borrosa (sección 16). *
  ↵ /
export function decideQuality(
  exposure: ExposureStats,
  laplacianVar: number | null,
  profile: QualityProfile,
  extra: { analyzedRegions: number; durationMs: number },
): QualityResult {
  const e = profile.exposure;
  const reasons: QualityReason[] = [];
  if (exposure.mean < e.minMean || exposure.darkRatio > e.maxDarkRatio) reasons.push('EXPOSICION_OSCURA');
  else if (exposure.mean > e.maxMean || exposure.brightRatio > e.maxBrightRatio) reasons.push('EXPOSICION_SATURADA');
  const metrics: QualityMetrics = {
    luminanceMean: Math.round(exposure.mean * 10) / 10,
    darkRatio: Math.round(exposure.darkRatio * 1000) / 1000,
    brightRatio: Math.round(exposure.brightRatio * 1000) / 1000,
    laplacianVariance: laplacianVar === null ? -1 : Math.round(laplacianVar * 10) / 10,
    analyzedRegions: extra.analyzedRegions,
    durationMs: Math.round(extra.durationMs),
  };
  if (reasons.length > 0) return { status: 'REPETIR_EXPOSICION', reasons, metrics, profileVersion: profile.version };
  if (laplacianVar === null) throw new Error('Falta la nitidez cuando la exposición es correcta');
  if (laplacianVar < profile.sharpness.minLaplacianVariance) {
    return { status: 'REPETIR_NITIDEZ', reasons: ['NITIDEZ_BAJA'], metrics, profileVersion: profile.version };
  }
  return { status: 'UTILIZABLE', reasons: [], metrics, profileVersion: profile.version };
}

```

B.2 Integración con Expo (boceto que compila contra las firmas del SDK 54) — src/device/qualityPipeline.ts

Se prueba en el celular en la tarea T-11 y se ajusta si los tiempos superan la meta.

```

// src/device/qualityPipeline.ts – Boceto de integración (usa APIs verificadas del SDK 54; probar en la Fase 1).
import { ImageManipulator, SaveFormat } from 'expo-image-manipulator';
import { File } from 'expo-file-system';
import jpeg from 'jpeg-js';
import type { QualityResult } from '../domain/types';
import { decideQuality, exposureStats, laplacianVariance, regionRects, toGray, type QualityProfile, type RawImage } from './quality';

export interface QualityPipelineConfig {
  timeoutMs: number;
  exposure: QualityProfile['exposure'] & { analysisWidth: number };
  sharpness: { regionSize: number; regionCount: 1 | 2 | 3; minLaplacianVariance: number };
}

async function decode(uri: string): Promise<RawImage> {
  const file = new File(uri);
  const bytes = await file.bytes();
  const img = jpeg.decode(bytes, { useTArray: true, formatAsRGBA: false, maxMemoryUsageInMB: 64 });
  file.delete(); // copia temporal en caché
  return { width: img.width, height: img.height, data: img.data as Uint8Array, channels: 3 };
}

async function render(ctx: ReturnType<typeof ImageManipulator.manipulate>): Promise<RawImage> {
  const ref = await ctx.renderAsync();
  const saved = await ref.saveAsync({ format: SaveFormat.JPEG, compress: 0.92 });
  return decode(saved.uri);
}

function withTimeout<T>(promise: Promise<T>, ms: number): Promise<T | 'TIMEOUT'> {
  return Promise.race([promise, new Promise<'TIMEOUT'>((resolve) => setTimeout(() => resolve('TIMEOUT'), ms))]);
}

/** photoUri: foto original ya guardada en captures/...; width y height: los que devolvió takePictureAsync. */
export async function evaluatePhoto(
  photoUri: string,
  width: number,
  height: number,
  cfg: QualityPipelineConfig,
  profileVersion: string,
): Promise<QualityResult> {
  const started = Date.now();
  const profile: QualityProfile = {
    version: profileVersion,
    exposure: cfg.exposure,
    sharpness: { minLaplacianVariance: cfg.sharpness.minLaplacianVariance },
  };
  const work = async (): Promise<QualityResult> => {
    // 1) Exposición sobre una copia reducida de toda la foto.

```

```
const small = await render(ImageManipulator.manipulate(photoUri).resize({ width: cfg.exposure.analysisWidth }));
const exp = exposureStats(toGray(small), cfg.exposure.darkLevel, cfg.exposure.brightLevel);
const exposureFails =
  exp.mean < cfg.exposure.minMean ||
  exp.darkRatio > cfg.exposure.maxDarkRatio ||
  exp.mean > cfg.exposure.maxMean ||
  exp.brightRatio > cfg.exposure.maxBrightRatio;
if (exposureFails) return decideQuality(exp, null, profile, { analyzedRegions: 0, durationMs: Date.now() - started
  });
// 2) Nitidez en recortes a resolución original; se usa el máximo.
let maxVar = 0;
const rects = regionRects(width, height, cfg.sharpness.regionSize, cfg.sharpness.regionCount);
for (const rect of rects) {
  const region = await render(ImageManipulator.manipulate(photoUri).crop(rect));
  maxVar = Math.max(maxVar, laplacianVariance(toGray(region), region.width, region.height));
}
return decideQuality(exp, maxVar, profile, { analyzedRegions: rects.length, durationMs: Date.now() - started });
};
try {
  const result = await withTimeout(work(), cfg.timeoutMs);
  if (result === 'TIMEOUT') {
    return { status: 'PENDIENTE_REVISION_TECNICA', reasons: ['TIEMPO_AGOTADO'], metrics: null, profileVersion };
  }
  return result;
} catch {
  return { status: 'ERROR_CAMARA', reasons: ['ARCHIVO_INVALIDO'], metrics: null, profileVersion };
}
}
```

Anexo C. Código de referencia probado: autenticación, validaciones y reintentos

C.1 Login sin internet — src/auth/offlineAuth.ts

```
// src/auth/offlineAuth.ts – Verificador local para iniciar sesión sin internet (RN-03, RN-20).
// Se guarda en SecureStore (clave "auth.offlineVerifier"); nunca se guarda la contraseña.
import { pbkdf2Async } from '@noble/hashes/pbkdf2';
import { sha256 } from '@noble/hashes/sha2';
import type { AccountStatus, UserRole } from '../domain/types';
import { MOBILE_ALLOWED_ROLES } from '../domain/types';
export interface OfflineVerifier {
  v: 1;
  userId: string;
  email: string; // normalizado: trim + minúsculas
  saltB64: string;
  iterations: number;
  hashB64: string; // PBKDF2-HMAC-SHA256, 32 bytes
}
/** Datos no secretos del último login online (SQLite, tabla app_meta / users_cache). */
export interface OfflineAuthRecord {
  userId: string;
  email: string;
  roles: UserRole[];
  accountStatus: AccountStatus;
  mustChangePassword: boolean;
  lastOnlineAuthAt: string; // ISO
  failedAttempts: number;
  lockedUntil: string | null; // ISO
}
export type OfflineDenyReason =
  | 'SIN_VERIFICADOR'
  | 'OTRO_USUARIO'
  | 'VENCIDO'
  | 'BLOQUEADO_TEMPORAL'
  | 'CUENTA_NO_ACTIVADA'
  | 'ROL_NO_PERMITIDO'
  | 'CAMBIO_CONTRASEÑA_PENDIENTE';
export type OfflineDecision = { allowed: true } | { allowed: false; reason: OfflineDenyReason; until?: string };
const B64 = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/';
export function bytesToBase64(bytes: Uint8Array): string {
  let out = '';
  let i = 0;
  for (; i + 2 < bytes.length; i += 3) {
    const n = (bytes[i] << 16) | (bytes[i + 1] << 8) | bytes[i + 2];
    out += B64[(n >> 18) & 63] + B64[(n >> 12) & 63] + B64[(n >> 6) & 63] + B64[n & 63];
  }
  const rem = bytes.length - i;
```

```

    if (rem === 1) {
      const n = bytes[i] << 16;
      out += B64[(n >> 18) & 63] + B64[(n >> 12) & 63] + '==';
    } else if (rem === 2) {
      const n = (bytes[i] << 16) | (bytes[i + 1] << 8);
      out += B64[(n >> 18) & 63] + B64[(n >> 12) & 63] + B64[(n >> 6) & 63] + '=';
    }
    return out;
  }
}
export function base64ToBytes(b64: string): Uint8Array {
  const clean = b64.replace(/=+$/, '');
  const out: number[] = [];
  let buffer = 0;
  let bits = 0;
  for (const ch of clean) {
    const v = B64.indexOf(ch);
    if (v < 0) throw new Error('base64 inválido');
    buffer = (buffer << 6) | v;
    bits += 6;
    if (bits >= 8) {
      bits -= 8;
      out.push((buffer >> bits) & 0xff);
    }
  }
  return new Uint8Array(out);
}
export function normalizeEmail(email: string): string {
  return email.trim().toLowerCase();
}
}
async function derive(password: string, salt: Uint8Array, iterations: number): Promise<Uint8Array> {
  return pbkdf2Async(sha256, new TextEncoder().encode(password), salt, { c: iterations, dkLen: 32 });
}
}
function constantTimeEqual(a: Uint8Array, b: Uint8Array): boolean {
  if (a.length !== b.length) return false;
  let diff = 0;
  for (let i = 0; i < a.length; i++) diff |= a[i] ^ b[i];
  return diff === 0;
}
}
/** Crear SOLO tras un login online exitoso con mustChangePassword = false (RN-04). */
export async function createVerifier(
  password: string,
  user: { userId: string; email: string },
  iterations: number,
  randomBytes: (n: number) => Uint8Array, // en la app: Crypto.getRandomBytes
): Promise<OfflineVerifier> {
  const salt = randomBytes(16);
  const hash = await derive(password, salt, iterations);
  return {
    v: 1,
    userId: user.userId,
    email: normalizeEmail(user.email),
    saltB64: bytesToBase64(salt),
    iterations,
    hashB64: bytesToBase64(hash),
  };
}
}
export async function verifyPassword(password: string, verifier: OfflineVerifier): Promise<boolean> {
  const hash = await derive(password, base64ToBytes(verifier.saltB64), verifier.iterations);
  return constantTimeEqual(hash, base64ToBytes(verifier.hashB64));
}
}
/** Reglas previas a pedir la verificación (no revela si la contraseña es correcta). */
export function canAttemptOfflineLogin(
  record: OfflineAuthRecord | null,
  verifier: OfflineVerifier | null,
  email: string,
  now: Date,
  cfg: { offlineLoginMaxDays: number },
): OfflineDecision {
  if (!record || !verifier) return { allowed: false, reason: 'SIN_VERIFICADOR' };
  if (normalizeEmail(email) !== verifier.email || record.userId !== verifier.userId) {
    return { allowed: false, reason: 'OTRO_USUARIO' };
  }
  if (record.lockedUntil && new Date(record.lockedUntil).getTime() > now.getTime()) {
    return { allowed: false, reason: 'BLOQUEADO_TEMPORAL', until: record.lockedUntil };
  }
  if (record.accountStatus !== 'ACTIVO') return { allowed: false, reason: 'CUENTA_NO_ACTIVA' };
  if (!record.roles.some((r) => MOBILE_ALLOWED_ROLES.includes(r))) return { allowed: false, reason: 'ROL_NO_PERMITIDO' };
  if (record.mustChangePassword) return { allowed: false, reason: 'CAMBIO_CONTRASENA_PENDIENTE' };
  const until = new Date(new Date(record.lastOnlineAuthAt).getTime() + cfg.offlineLoginMaxDays * 86_400_000);
  if (now.getTime() > until.getTime()) return { allowed: false, reason: 'VENCIDO', until: until.toISOString() };
  return { allowed: true };
}

```

```

}
export function registerFailedOfflineAttempt(
  record: OfflineAuthRecord,
  now: Date,
  cfg: { offlineMaxFailedAttempts: number; offlineLockoutMinutes: number },
): OfflineAuthRecord {
  const failedAttempts = record.failedAttempts + 1;
  if (failedAttempts >= cfg.offlineMaxFailedAttempts) {
    const lockedUntil = new Date(now.getTime() + cfg.offlineLockoutMinutes * 60_000).toISOString();
    return { ...record, failedAttempts: 0, lockedUntil };
  }
  return { ...record, failedAttempts };
}
export function registerSuccessfulOfflineLogin(record: OfflineAuthRecord): OfflineAuthRecord {
  return { ...record, failedAttempts: 0, lockedUntil: null };
}
export function offlineValidUntil(lastOnlineAuthAt: string, offlineLoginMaxDays: number): string {
  return new Date(new Date(lastOnlineAuthAt).getTime() + offlineLoginMaxDays * 86_400_000).toISOString();
}

```

C.2 Validaciones de formularios — src/auth/validation.ts

```

// src/auth/validation.ts - Validaciones de formularios (la API vuelve a validar).
export interface FieldError { field: string; message: string; }
const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
export function validatePassword(password: string, minLength: number): string | null {
  if (password.length < minLength) return `Debe tener al menos ${minLength} caracteres.`;
  if (!/[A-Za-zÁÉÍÓÚÑáéíóúñ]/.test(password) || !/\d/.test(password)) return 'Debe incluir letras y números.';
  if (password.trim() !== password) return 'No debe empezar ni terminar con espacios.';
  return null;
}
export function validateRegistration(
  input: { fullName: string; email: string; phone: string; password: string; confirm: string; accepted: boolean },
  minLength: number,
): FieldError[] {
  const errors: FieldError[] = [];
  if (input.fullName.trim().length < 5) errors.push({ field: 'fullName', message: 'Escribe tu nombre completo.' });
  if (!EMAIL_RE.test(input.email.trim())) errors.push({ field: 'email', message: 'Escribe un correo válido.' });
  if (input.phone.trim() !== '' && !/^+?\d{9,15}$/.test(input.phone.replace(/\s/g, ''))) {
    errors.push({ field: 'phone', message: 'Escribe un celular válido (9 a 15 dígitos).' });
  }
  const pw = validatePassword(input.password, minLength);
  if (pw) errors.push({ field: 'password', message: pw });
  if (input.password !== input.confirm) errors.push({ field: 'confirm', message: 'Las contraseñas no coinciden.' });
  if (!input.accepted) errors.push({ field: 'accepted', message: 'Debes aceptar el aviso de privacidad.' });
  return errors;
}

```

C.3 Reintentos y clasificación de errores — src/sync/retry.ts

```

// src/sync/retry.ts - Espera entre reintentos y clasificación de errores de sincronización.
import type { ApiErrorCode } from '../api/dto';
import type { LocalCaptureResponse } from '../protocol/messages';
/** attempt empieza en 1. Usa la tabla de esperas y repite la última; nunca supera maxDelayMs si se indica. */
export function nextDelayMs(attempt: number, delaysMs: readonly number[], maxDelayMs?: number): number {
  if (delaysMs.length === 0) return 0;
  const idx = Math.min(Math.max(attempt, 1), delaysMs.length) - 1;
  const d = delaysMs[idx];
  return maxDelayMs === undefined ? d : Math.min(d, maxDelayMs);
}
export type SyncErrorClass =
  | 'REINTENTAR' // red, tiempo agotado, 408, 429, 5xx: sigue en cola con espera creciente
  | 'RENOVAR_TOKEN' // 401 por token vencido: renovar y reintentar una vez
  | 'REQUIERE_LOGIN' // refresh inválido, cuenta bloqueada, dispositivo revocado, rol no permitido: pausar la cola
  | 'SINCRONIZAR_PADRE' // 404 de sesión/pasada/secuencia: enviar primero el registro padre
  | 'DEFINITIVO'; // 400, 409, 413, 422 y otros 4xx: ERROR_SINCRONIZACION, revisión manual
export function classifySyncError(input: { networkError: boolean; status?: number; code?: ApiErrorCode }): SyncErrorClass {
  const {
    networkError,
    status,
    code,
  } = input;
  if (networkError || status === undefined) return 'REINTENTAR';
  const { status, code } = input;
  if (status === 401) return code === 'REFRESH_INVALID' ? 'REQUIERE_LOGIN' : 'RENOVAR_TOKEN';
  if (status === 403) {
    return code === 'ACCOUNT_BLOCKED' || code === 'DEVICE_REVOKED' || code === 'ROLE_NOT_ALLOWED' || code === undefined
      ? 'REQUIERE_LOGIN'
      : 'DEFINITIVO';
  }
  if (status === 404 && (code === 'SESSION_NOT_FOUND' || code === 'PASS_NOT_FOUND' || code === 'SEQUENCE_NOT_FOUND'))

```

```

    {
      return 'SINCRONIZAR_PADRE';
    }
    if (status === 408 || status === 429 || status >= 500) return 'REINTENTAR';
    return 'DEFINITIVO';
  }
  // ----- transferencia local (cámara → controlador)
  export type LocalTransferDecision =
    | 'RECIBIDA' // RECIBIDA_CONTROLADOR
    | 'REINTENTAR' // vuelve a PENDIENTE_LOCAL con espera transfer.retryDelaysMs
    | 'ERROR_LOCAL'; // ERROR_LOCAL: event_log TRANSFER/ERROR en la cámara; solo reintento manual
  /**
   * Decide el siguiente estado de una foto tras un intento de envío.
   * checksumFailures cuenta los MD5_MISMATCH / SIZE_MISMATCH previos de esa foto (sin el actual).
   */
  export function classifyLocalTransfer(
    outcome: { networkError: true } | { networkError: false; response: LocalCaptureResponse },
    checksumFailures: number,
    maxChecksumRetries: number,
  ): LocalTransferDecision {
    if (outcome.networkError) return 'REINTENTAR'; // red, tiempo agotado o conexión cortada
    const { result, reason } = outcome.response;
    if (result === 'RECEIVED' || result === 'ALREADY_RECEIVED') return 'RECIBIDA';
    switch (reason) {
      case 'MD5_MISMATCH':
      case 'SIZE_MISMATCH':
        return checksumFailures + 1 < maxChecksumRetries ? 'REINTENTAR' : 'ERROR_LOCAL';
      case 'NO_SPACE':
      case 'INTERNAL':
        return 'REINTENTAR';
      default:
        // UNKNOWN_SESSION, UNKNOWN_SEQUENCE, WRONG_DEVICE, CAPTURE_CONFLICT, INVALID_META,
        // LENGTH_REQUIRED, PAYLOAD_TOO_LARGE, NOT_FOUND o motivo nulo en un rechazo
        return 'ERROR_LOCAL';
    }
  }
}

```

Anexo D. Datos del archivo de mapeo para el backend simulado

Lote (id propuesto)	Hoja del archivo	Hileras	Plantas (suma de hileras)	Plantas (hoja RESUMEN)
SWG 1 (SWG1)	LOTE 1(PISCINA)	31	9 986	9 987
SWG 2 (SWG2)	LOTE 2(MAIZ)	16	4 841	4 843
SWG 5 (SWG5)	LOTE 5 (Triangulo)	36	4 832	4 836

Las diferencias de pocas plantas entre la suma por hilera y la hoja RESUMEN ya estaban señaladas en la Especificación v2.0 (§4.1) y deben validarse con la empresa. Los segmentos del piloto (12 de 25 plantas: 4 por lote) y sus marcadores todavía no tienen códigos definitivos (Q-01).

```

// src/api/mock/pilotRows.ts – Hileras y plantas por hilera de los lotes del piloto, tomadas del archivo
// «MAPEO CHANCHITO MOLINOS actualizado» (columna «N° ptas» de cada hoja). Datos para el backend simulado;
// los códigos y nombres deben validarse con la empresa (pregunta Q-01).
import type { FieldRow, Lot } from '../../domain/types';
/** Posición i del arreglo = hilera i + 1. */
export const PILOT_PLANTS_PER_ROW: Record<'SWG1' | 'SWG2' | 'SWG5', readonly number[]> = {
  SWG1: [383, 384, 381, 384, 385, 386, 384, 385, 386, 384, 388, 388, 391, 382, 275, 275, 274, 273, 273, 272, 270,
    ↵ 268, 270, 269, 269, 268, 268, 267, 266, 265],
  SWG2: [251, 260, 267, 273, 279, 286, 292, 299, 307, 313, 319, 326, 332, 339, 347, 351],
  SWG5: [11, 19, 25, 33, 40, 47, 54, 62, 68, 75, 84, 91, 97, 104, 111, 118, 126, 132, 138, 146, 154, 161, 168, 174, 18
    ↵ 1, 188, 194, 200, 207, 213, 220, 223, 233, 239, 246, 250],
};
export const PILOT_LOTS: Lot[] = [
  { id: 'SWG1', code: 'SWG 1', name: 'Lote 1 (Piscina)', active: true },
  { id: 'SWG2', code: 'SWG 2', name: 'Lote 2 (Maíz)', active: true },
  { id: 'SWG5', code: 'SWG 5', name: 'Lote 5 (Triángulo)', active: true },
];
export function buildPilotRows(): FieldRow[] {
  const rows: FieldRow[] = [];
  for (const lot of PILOT_LOTS) {
    const plants = PILOT_PLANTS_PER_ROW[lot.id as keyof typeof PILOT_PLANTS_PER_ROW];
    plants.forEach((plantCount, i) => {
      const number = i + 1;
      rows.push({ id: `${lot.id}-H${String(number).padStart(2, '0')}`, lotId: lot.id, number, plantCount, active: true
        ↵ });
    });
  }
}

```

```
    return rows;
  }
}
```

Anexo E. Pruebas de referencia

Se ejecutaron con TypeScript 5.9 (modo estricto) y el ejecutor de pruebas de Node 22: **32 pruebas, 32 aprobadas, 0 fallidas**. Para repetirlas fuera de la app: `npm install typescript@~5.9.2 @types/node@20 @noble/hashes@1.8.0 jpeg-js@0.4.4 ws@8 @types/ws`, compilar con `npx tsc` y ejecutar `node --test dist/tests/*.test.js`. En la app se pasan a Jest como indica 21.1. El esquema SQL de la sección 12.1 se comprobó aparte en SQLite 3.45 con 33 casos (inserciones válidas e inválidas).

E.1 tests/domain.test.ts

```
import { test } from 'node:test';
import assert from 'node:assert/strict';
import {
  SESSION_TRANSITIONS, PASS_TRANSITIONS, SEQUENCE_TRANSITIONS, LOCAL_TRANSFER_TRANSITIONS, REMOTE_SYNC_TRANSITIONS,
  SLOT_TRANSITIONS, canTransition, assertTransition, InvalidTransitionError,
} from '../src/domain/stateMachines';
import { deriveSequenceStatus, applyRetake } from '../src/domain/sequence';
import type { CameraRole, SequenceSlot } from '../src/domain/types';
import { DEFAULT_CONFIG } from '../src/config/defaults';
test('transiciones de sesión válidas e inválidas', () => {
  assert.ok(canTransition(SESSION_TRANSITIONS, 'DRAFT', 'PREPARING'));
  assert.ok(canTransition(SESSION_TRANSITIONS, 'ACTIVE', 'PAUSED'));
  assert.ok(!canTransition(SESSION_TRANSITIONS, 'DRAFT', 'ACTIVE'));
  assert.ok(!canTransition(SESSION_TRANSITIONS, 'SYNCED', 'ACTIVE'));
  assert.throws(() => assertTransition('sesión', SESSION_TRANSITIONS, 'CLOSED', 'ACTIVE'), InvalidTransitionError);
});
test('todas las metas de transición son estados conocidos', () => {
  for (const map of [SESSION_TRANSITIONS, PASS_TRANSITIONS, SEQUENCE_TRANSITIONS, SLOT_TRANSITIONS, LOCAL_TRANSFER_TRANSITIONS, REMOTE_SYNC_TRANSITIONS] as Array<Record<string, readonly string[]>>) {
    const states = Object.keys(map);
    for (const targets of Object.values(map)) for (const t of targets) assert.ok(states.includes(t), t);
  }
});
function slots(o1: SequenceSlot['outcome'], o2: SequenceSlot['outcome']): Record<CameraRole, SequenceSlot> {
  return { CAMERA_1: { role: 'CAMERA_1', captureId: 'a', outcome: o1 }, CAMERA_2: { role: 'CAMERA_2', captureId: 'b', outcome: o2 } };
}
test('estado de secuencia derivado de los resultados por cámara', () => {
  assert.equal(deriveSequenceStatus(slots('PENDIENTE', 'PENDIENTE')), 'COMMAND_SENT');
  assert.equal(deriveSequenceStatus(slots('OK_PENDIENTE_ARCHIVO', 'PENDIENTE')), 'PARTIAL');
  assert.equal(deriveSequenceStatus(slots('OK_RECIBIDA', 'OK_PENDIENTE_ARCHIVO')), 'PARTIAL');
  assert.equal(deriveSequenceStatus(slots('OK_RECIBIDA', 'OK_RECIBIDA')), 'COMPLETE');
  assert.equal(deriveSequenceStatus(slots('OK_RECIBIDA', 'RECHAZADA_CALIDAD')), 'INCOMPLETE');
  assert.equal(deriveSequenceStatus(slots('SIN_RESPUESTA', 'ERROR_CAMARA')), 'INCOMPLETE');
});
test('una repetición reabre solo la cámara afectada y puede completar la secuencia', () => {
  let s = slots('OK_RECIBIDA', 'RECHAZADA_CALIDAD');
  assert.equal(deriveSequenceStatus(s), 'INCOMPLETE');
  s = applyRetake(s, 'CAMERA_2', 'c');
  assert.equal(s.CAMERA_2.captureId, 'c');
  assert.equal(s.CAMERA_1.outcome, 'OK_RECIBIDA');
  assert.equal(deriveSequenceStatus(s), 'PARTIAL');
  assert.ok(canTransition(SEQUENCE_TRANSITIONS, 'INCOMPLETE', 'PARTIAL'));
  s = { ...s, CAMERA_2: { ...s.CAMERA_2, outcome: 'OK_RECIBIDA' } };
  assert.equal(deriveSequenceStatus(s), 'COMPLETE');
});
test('todo cambio permitido de un resultado por cámara produce una transición de secuencia permitida', () => {
  const outcomes = Object.keys(SLOT_TRANSITIONS) as SequenceSlot['outcome'][];
  let checked = 0;
  for (const o1 of outcomes) for (const o2 of outcomes) {
    const prev = deriveSequenceStatus(slots(o1, o2));
    for (const n1 of SLOT_TRANSITIONS[o1]) {
      const next = deriveSequenceStatus(slots(n1, o2));
      assert.ok(prev === next || canTransition(SEQUENCE_TRANSITIONS, prev, next), `${o1},${o2} → ${n1},${o2}: ${prev} → ${next}`);
      checked++;
    }
  }
  for (const n2 of SLOT_TRANSITIONS[o2]) {
    const next = deriveSequenceStatus(slots(o1, n2));
    assert.ok(prev === next || canTransition(SEQUENCE_TRANSITIONS, prev, next), `${o1},${o2} → ${o1},${n2}: ${prev} → ${next}`);
    checked++;
  }
}
```

```

    }
    assert.ok(checked > 100);
  });
  test('configuración por defecto coherente', () => {
    const c = DEFAULT_CONFIG;
    assert.ok(c.protocol.unstableAfterMs > c.protocol.heartbeatIntervalMs);
    assert.ok(c.protocol.lostAfterMs > c.protocol.unstableAfterMs);
    assert.ok(c.protocol.captureResponseTimeoutMs > c.protocol.commandValidityMs);
    assert.ok(c.capture.intervalMs >= c.capture.minIntervalMs && c.capture.intervalMs <= c.capture.maxIntervalMs);
    assert.ok(c.device.minFreeSpaceToStartBytes > c.device.pauseFreeSpaceBytes);
    assert.ok(c.device.minBatteryToStartPct > c.device.pauseBatteryPct);
    assert.ok(c.quality.exposure.minMean < c.quality.exposure.maxMean);
    // La cámara debe poder capturar y evaluar antes de que el controlador declare SIN_RESPUESTA.
    assert.ok(c.protocol.captureResponseTimeoutMs >= c.capture.captureBudgetMs + c.quality.timeoutMs + 500);
    assert.ok(c.protocol.commandValidityMs < c.protocol.captureResponseTimeoutMs);
    assert.ok(c.shortTest.timeoutMs > c.protocol.captureResponseTimeoutMs);
    assert.ok(c.protocol.ackTimeoutMs < c.protocol.lostAfterMs);
    assert.ok(c.device.warnBatteryPct > c.device.pauseBatteryPct && c.device.warnBatteryPct < c.device.minBatteryToStartPct);
    assert.ok(c.device.warnFreeSpaceBytes > c.device.minFreeSpaceToStartBytes);
    assert.ok(c.sync.healthPath.startsWith('/') && !c.sync.healthPath.startsWith('/api'));
    assert.ok(c.transfer.maxChecksumRetries >= 1 && c.protocol.clockOffsetSamples >= 1 && c.sync.batchSize >= 1);
  });
  test('una página de RESYNC_STATE cabe en un mensaje', () => {
    const c = DEFAULT_CONFIG;
    const uuid = 'ffffffff-ffff-4fff-bfff-ffffffffffff';
    const page = {
      v: 1, type: 'RESYNC_STATE', messageId: uuid, sessionId: uuid, deviceId: uuid, sentAt: new Date().toISOString(),
      payload: {
        lastSequenceIdSeen: uuid, page: 1, totalPages: 99,
        captures: Array.from({ length: c.protocol.resyncPageSize }, () => ({
          captureId: uuid, sequenceId: uuid, qualityStatus: 'PENDIENTE_REVISION_TECNICA', localTransferStatus: 'TRANSFIRIENDO_LOCAL',
        })),
      },
    };
    assert.ok(new TextEncoder().encode(JSON.stringify(page)).length < c.protocol.maxMessageBytes);
  });
  test('una foto tardía reabre la sesión, la pasada y sus secuencias para sincronizarlas', () => {
    assert.ok(canTransition(SESSION_TRANSITIONS, 'SYNCED', 'CLOSED'));
    assert.ok(!canTransition(SESSION_TRANSITIONS, 'SYNCED', 'ACTIVE'));
    assert.ok(canTransition(REMOTE_SYNC_TRANSITIONS, 'SINCRONIZADO', 'PENDIENTE_NUBE'));
    assert.ok(!canTransition(REMOTE_SYNC_TRANSITIONS, 'SINCRONIZADO', 'SUBIENDO'));
  });

```

E.2 tests/wsCodec.test.ts

```

import { test } from 'node:test';
import assert from 'node:assert/strict';
import net from 'node:net';
import crypto from 'node:crypto';
import WebSocket from 'ws';
import { WsServerConnection, FrameDecoder, encodeFrame, OPCODE, computeAcceptKey } from '../src/local-network/wsCodec';

const sha1Base64 = async (s: string) => crypto.createHash('sha1').update(s).digest('base64');
test('clave Sec-WebSocket-Accept del ejemplo de la RFC 6455', async () => {
  assert.equal(await computeAcceptKey('dGh1IHNhbXBsZSBub25jZQ==', sha1Base64), 's3pPLMBiTxaQ9kYGzshZrBk+x0o=');
});
test('decodifica frames partidos byte a byte y con longitud de 16 bits', () => {
  const payload = new Uint8Array(300).map((_, i) => i % 251);
  const mask = [1, 2, 3, 4];
  const frame = encodeFrame(OPCODE.BINARY, payload);
  // convertir a frame enmascarado de cliente
  const masked = new Uint8Array(frame.length + 4);
  masked[0] = frame[0]; masked[1] = frame[1] | 0x80; masked[2] = frame[2]; masked[3] = frame[3];
  masked.set(mask, 4);
  for (let i = 0; i < payload.length; i++) masked[8 + i] = payload[i] ^ mask[i & 3];
  const dec = new FrameDecoder();
  const out = [] as ReturnType<FrameDecoder['push']>;
  for (const b of masked) out.push(...dec.push(new Uint8Array([b])));
  assert.equal(out.length, 1);
  assert.deepEqual(Array.from(out[0].payload), Array.from(payload));
});
function startServer(onText: (conn: WsServerConnection, text: string) => void): Promise<{ port: number; close: () => void; events: string[] }> {
  const events: string[] = [];
  return new Promise((resolve) => {
    const server = net.createServer((socket) => {
      const conn = new WsServerConnection(
        (bytes) => { socket.write(Buffer.from(bytes)); },

```

```

        () => { socket.end(); },
        sha1Base64,
        {
            onOpen: (head) => events.push('open:' + head.path),
            onText: (t) => { events.push('text'); onText(conn, t); },
            onClose: (code) => events.push('close:' + code),
            onError: (e) => events.push('error:' + e.message),
        },
    );
    socket.on('data', (chunk) => { void conn.onData(new Uint8Array(chunk)); });
});
server.listen(0, '127.0.0.1', () => {
    const port = (server.address() as net.AddressInfo).port;
    resolve({ port, close: () => server.close(), events });
});
});
}
test('eco de mensajes de texto con un cliente WebSocket real (ws)', async () => {
    const srv = await startServer((conn, t) => conn.sendText(JSON.stringify({ echo: t })));
    const ws = new WebSocket(`ws://127.0.0.1:${srv.port}/control`);
    await new Promise<void>((r, j) => { ws.once('open', () => r()); ws.once('error', j); });
    const sizes = [5, 125, 126, 1000, 65_000];
    for (const size of sizes) {
        const msg = 'x'.repeat(size);
        const reply = await new Promise<string>((r) => { ws.once('message', (d) => r(d.toString())); ws.send(msg); });
        assert.equal(JSON.parse(reply).echo.length, size);
    }
    // mensaje fragmentado
    const reply2 = await new Promise<string>((r) => {
        ws.once('message', (d) => r(d.toString()));
        ws.send('hola ', { fin: false });
        ws.send('mundo', { fin: true });
    });
    assert.equal(JSON.parse(reply2).echo, 'hola mundo');
    // ping → pong
    const pong = await new Promise<string>((r) => { ws.once('pong', (d) => r(d.toString())); ws.ping('p1'); });
    assert.equal(pong, 'p1');
    // cierre ordenado
    const closed = await new Promise<number>((r) => { ws.once('close', (code) => r(code)); ws.close(1000, 'fin'); });
    assert.equal(closed, 1000);
    srv.close();
    assert.ok(srv.events.includes('open:/control'));
    assert.ok(srv.events.includes('close:1000'));
});
test('rechaza un handshake inválido con 400', async () => {
    const srv = await startServer(() => undefined);
    const res = await new Promise<string>((resolve) => {
        const s = net.connect(srv.port, '127.0.0.1', () => {
            s.write('GET / HTTP/1.1\r\nHost: x\r\n\r\n');
        });
        let data = '';
        s.on('data', (d) => (data += d.toString()));
        s.on('end', () => resolve(data));
        s.on('close', () => resolve(data));
    });
    assert.match(res, /^HTTP\/1.1 400/);
    srv.close();
});
test('mensaje que supera el límite cierra con 1009', async () => {
    const srv = await startServer(() => undefined);
    const ws = new WebSocket(`ws://127.0.0.1:${srv.port}/`);
    await new Promise<void>((r) => ws.once('open', () => r()));
    const code = await new Promise<number>((r) => { ws.once('close', (c) => r(c)); ws.send('y'.repeat(70_000)); });
    assert.equal(code, 1009);
    srv.close();
});
});

```

E.3 tests/httpUpload.test.ts

```

import { test } from 'node:test';
import assert from 'node:assert/strict';
import net from 'node:net';
import http from 'node:http';
import crypto from 'node:crypto';
import { HttpUploadConnection, decodeBase64UrlToText, encodeTextToBase64Url, type UploadSink } from '../src/local-netw
    ork/httpUpload';
test('base64url ida y vuelta con tildes', () => {
    const text = JSON.stringify({ a: 'cámara ñ', b: [1, 2, 3] });
    assert.equal(decodeBase64UrlToText(encodeTextToBase64Url(text)), text);
});

```

```
function startServer(store: Map<string, Buffer>) {
  return new Promise<{ port: number; close: () => void }>((resolve) => {
    const server = net.createServer((socket) => {
      const conn = new HttpUploadConnection(
        (b) => { socket.write(Buffer.from(b)); },
        () => { socket.end(); },
        async (head, contentLength) => {
          if (head.method !== 'POST' || head.path !== '/local/captures') return { status: 404, body: { result: 'REJECT
            ↳ ED', reason: 'NOT_FOUND' } };
          const meta = JSON.parse(decodeBase64UrlToText(head.headers['x-capture-meta'] ?? ''));
          if (store.has(meta.captureId)) {
            // Duplicado: se descarta el cuerpo y se responde al final (seguro para cualquier cliente HTTP).
            const discard: UploadSink = { write: () => undefined, finish: async () => ({ status: 200, body: { result:
              ↳ 'ALREADY_RECEIVED', reason: null } }), abort: () => undefined };
            return discard;
          }
          const chunks: Buffer[] = [];
          const sink: UploadSink = {
            write: (c) => { chunks.push(Buffer.from(c)); },
            finish: async () => {
              const data = Buffer.concat(chunks);
              const md5 = crypto.createHash('md5').update(data).digest('hex');
              if (data.length !== contentLength || md5 !== meta.md5) return { status: 422, body: { result: 'REJECTED',
                ↳ reason: 'MD5_MISMATCH' } };
              store.set(meta.captureId, data);
              return { status: 200, body: { result: 'RECEIVED', reason: null } };
            },
            abort: () => undefined,
          };
          return sink;
        },
      );
      socket.on('data', (c) => { void conn.onData(new Uint8Array(c)); });
      socket.on('close', () => conn.onClose());
    });
    server.listen(0, '127.0.0.1', () => resolve({ port: (server.address() as net.AddressInfo).port, close: () => serve
      ↳ r.close() }));
  });
}

function upload(port: number, body: Buffer, meta: object, expectContinue = false) {
  return new Promise<{ status: number; json: any }>((resolve, reject) => {
    const headers: Record<string, string | number> = {
      'Content-Type': 'image/jpeg',
      'Content-Length': body.length,
      'X-Capture-Meta': encodeTextToBase64Url(JSON.stringify(meta)),
    };
    if (expectContinue) headers['Expect'] = '100-continue';
    const req = http.request({ host: '127.0.0.1', port, method: 'POST', path: '/local/captures', headers }, (res) => {
      let data = '';
      res.on('data', (d) => (data += d));
      res.on('end', () => resolve({ status: res.statusCode ?? 0, json: JSON.parse(data) }));
    });
    req.on('error', reject);
    if (expectContinue) req.on('continue', () => req.end(body));
    else req.end(body);
  });
}

test('recibe una foto de 3 MB, verifica md5 y responde idempotente', async () => {
  const store = new Map<string, Buffer>();
  const srv = await startServer(store);
  const body = crypto.randomBytes(3 * 1024 * 1024);
  const meta = { captureId: 'cap-1', md5: crypto.createHash('md5').update(body).digest('hex') };
  const r1 = await upload(srv.port, body, meta);
  assert.equal(r1.status, 200);
  assert.equal(r1.json.result, 'RECEIVED');
  assert.ok(store.get('cap-1')?.equals(body));
  const r2 = await upload(srv.port, body, meta);
  assert.equal(r2.json.result, 'ALREADY_RECEIVED');
  const r3 = await upload(srv.port, Buffer.from('abc'), { captureId: 'cap-2', md5: 'mal' }, true);
  assert.equal(r3.status, 422);
  assert.equal(r3.json.reason, 'MD5_MISMATCH');
  srv.close();
});
```

E.4 tests/quality.test.ts

```
import { test } from 'node:test';
import assert from 'node:assert/strict';
import jpeg from 'jpeg-js';
import { toGray, exposureStats, laplacianVariance, regionRects, decideQuality, type RawImage, type QualityProfile } from './src/device/quality';
import { DEFAULT_CONFIG } from './src/config/defaults';
const q = DEFAULT_CONFIG.quality;
const profile: QualityProfile = { version: q.profileVersion, exposure: q.exposure, sharpness: { minLaplacianVariance: q.sharpness.minLaplacianVariance } };
// Textura tipo follaje: ruido + bordes, determinista.
function texture(w: number, h: number, scale = 1, offset = 0): RawImage {
  const data = new Uint8Array(w * h * 3);
  let seed = 12345;
  const rnd = () => ((seed = (seed * 1103515245 + 12345) & 0x7fffffff) / 0x7fffffff);
  for (let y = 0; y < h; y++) for (let x = 0; x < w; x++) {
    const base = ((Math.floor(x / 6) + Math.floor(y / 6)) % 2) * 120 + 60 + rnd() * 70;
    const v = Math.max(0, Math.min(255, base * scale + offset));
    const p = (y * w + x) * 3;
    data[p] = v * 0.8; data[p + 1] = v; data[p + 2] = v * 0.6;
  }
  return { width: w, height: h, data, channels: 3 };
}
function boxBlur(img: RawImage, r: number): RawImage {
  const { width: w, height: h, data } = img;
  const out = new Uint8Array(data.length);
  for (let y = 0; y < h; y++) for (let x = 0; x < w; x++) for (let c = 0; c < 3; c++) {
    let s = 0, n = 0;
    for (let dy = -r; dy <= r; dy++) for (let dx = -r; dx <= r; dx++) {
      const xx = Math.min(w - 1, Math.max(0, x + dx)), yy = Math.min(h - 1, Math.max(0, y + dy));
      s += data[(yy * w + xx) * 3 + c]; n++;
    }
    out[(y * w + x) * 3 + c] = s / n;
  }
  return { ...img, data: out };
}
function evaluate(img: RawImage) {
  const gray = toGray(img);
  const exp = exposureStats(gray, profile.exposure.darkLevel, profile.exposure.brightLevel);
  const lv = laplacianVariance(gray, img.width, img.height);
  return { result: decideQuality(exp, lv, profile, { analyzedRegions: 1, durationMs: 0 }), exp, lv };
}
test('foto nítida y bien expuesta → UTILIZABLE', () => {
  const r = evaluate(texture(256, 256));
  assert.equal(r.result.status, 'UTILIZABLE', JSON.stringify(r.result.metrics));
});
test('desenfoque → REPETIR_NITIDEZ', () => {
  const r = evaluate(boxBlur(texture(256, 256), 4));
  assert.equal(r.result.status, 'REPETIR_NITIDEZ', JSON.stringify(r.result.metrics));
  assert.deepEqual(r.result.reasons, ['NITIDEZ_BAJA']);
});
test('foto oscura → REPETIR_EXPOSICION (aunque también parezca borrosa)', () => {
  const r = evaluate(texture(256, 256, 0.12));
  assert.equal(r.result.status, 'REPETIR_EXPOSICION');
  assert.deepEqual(r.result.reasons, ['EXPOSICION_OSCURA']);
});
test('foto saturada → REPETIR_EXPOSICION', () => {
  const r = evaluate(texture(256, 256, 1, 200));
  assert.equal(r.result.status, 'REPETIR_EXPOSICION');
  assert.deepEqual(r.result.reasons, ['EXPOSICION_SATURADA']);
});
test('ida y vuelta JPEG con jpeg-js (useTArray) mantiene la decisión', () => {
  const img = texture(320, 240);
  const encoded = jpeg.encode({ width: img.width, height: img.height, data: rgbToRgba(img.data) }, 90);
  const decoded = jpeg.decode(new Uint8Array(encoded.data), { useTArray: true, formatAsRGBA: false });
  const raw: RawImage = { width: decoded.width, height: decoded.height, data: decoded.data as Uint8Array, channels: 3 };
  assert.equal(evaluate(raw).result.status, 'UTILIZABLE');
});
test('recortes dentro de la imagen sobre la línea media', () => {
  const rects = regionRects(4000, 3000, 512, 3);
  assert.equal(rects.length, 3);
  for (const r of rects) {
    assert.ok(r.originX >= 0 && r.originX + r.width <= 4000);
    assert.ok(r.originY >= 0 && r.originY + r.height <= 3000);
    assert.equal(r.width, 512);
  }
  const small = regionRects(300, 200, 512, 1);
  assert.equal(small[0].width, 200);
});
```

```
function rgbToRgba(rgb: Uint8Array): Uint8Array {
  const out = new Uint8Array((rgb.length / 3) * 4);
  for (let i = 0, j = 0; i < rgb.length; i += 3, j += 4) { out[j] = rgb[i]; out[j + 1] = rgb[i + 1]; out[j + 2] = rgb[i + 2]; out[j + 3] = 255; }
  return out;
}
```

E.5 tests/offlineAuth.test.ts

```
import { test } from 'node:test';
import assert from 'node:assert/strict';
import crypto from 'node:crypto';
import { createVerifier, verifyPassword, canAttemptOfflineLogin, registerFailedOfflineAttempt, registerSuccessfulOfflineLogin, bytesToBase64, base64ToBytes, type OfflineAuthRecord } from '../src/auth/offlineAuth';
import { DEFAULT_CONFIG } from '../src/config/defaults';
const rnd = (n: number) => new Uint8Array(crypto.randomBytes(n));
test('base64 ida y vuelta', () => {
  for (const n of [0, 1, 2, 3, 16, 31, 32]) {
    const b = rnd(n);
    assert.deepEqual(Array.from(base64ToBytes(bytesToBase64(b))), Array.from(b));
    assert.equal(bytesToBase64(b), Buffer.from(b).toString('base64'));
  }
});
test('PBKDF2 coincide con la implementación de Node y verifica la contraseña', async () => {
  const v = await createVerifier('Riachuelo2026', { userId: 'u1', email: 'Oscar@Ejemplo.com' }, 1000, rnd);
  const expected = crypto.pbkdf2Sync('Riachuelo2026', Buffer.from(v.saltB64, 'base64'), 1000, 32, 'sha256').toString('base64');
  assert.equal(v.hashB64, expected);
  assert.equal(v.email, 'oscar@ejemplo.com');
  assert.ok(await verifyPassword('Riachuelo2026', v));
  assert.ok(!(await verifyPassword('riachuelo2026', v)));
});
test('reglas de login sin internet', async () => {
  const cfg = DEFAULT_CONFIG.auth;
  const v = await createVerifier('x', { userId: 'u1', email: 'a@b.c' }, 1000, rnd);
  const now = new Date('2026-11-10T12:00:00.000Z');
  const rec: OfflineAuthRecord = { userId: 'u1', email: 'a@b.c', roles: ['OPERADOR_CAMPO'], accountStatus: 'ACTIVO', mustChangePassword: false, lastOnlineAuthAt: '2026-11-05T12:00:00.000Z', failedAttempts: 0, lockedUntil: null };
  assert.deepEqual(canAttemptOfflineLogin(rec, v, 'A@B.C', now, cfg), { allowed: true });
  assert.equal((canAttemptOfflineLogin(rec, v, 'otro@b.c', now, cfg) as any).reason, 'OTRO_USUARIO');
  assert.equal((canAttemptOfflineLogin({ ...rec, lastOnlineAuthAt: '2026-11-01T00:00:00.000Z' }, v, 'a@b.c', now, cfg) as any).reason, 'VENCIDO');
  assert.equal((canAttemptOfflineLogin({ ...rec, roles: ['SUPERVISOR'] }, v, 'a@b.c', now, cfg) as any).reason, 'ROL_NO_PERMITIDO');
  assert.equal((canAttemptOfflineLogin({ ...rec, mustChangePassword: true }, v, 'a@b.c', now, cfg) as any).reason, 'CAMBIO_CONTRASEÑA_PENDIENTE');
  assert.equal((canAttemptOfflineLogin(null, null, 'a@b.c', now, cfg) as any).reason, 'SIN_VERIFICADOR');
  let r = rec;
  for (let i = 0; i < cfg.offlineMaxFailedAttempts; i++) r = registerFailedOfflineAttempt(r, now, cfg);
  const d = canAttemptOfflineLogin(r, v, 'a@b.c', now, cfg) as any;
  assert.equal(d.reason, 'BLOQUEADO_TEMPORAL');
  const later = new Date(now.getTime() + (cfg.offlineLockoutMinutes + 1) * 60_000);
  assert.deepEqual(canAttemptOfflineLogin(r, v, 'a@b.c', later, cfg), { allowed: true });
  assert.equal(registerSuccessfulOfflineLogin(r).lockedUntil, null);
});
test('tiempo de PBKDF2 con las iteraciones por defecto en Node (referencia, no Hermes)', async () => {
  const t0 = Date.now();
  await createVerifier('x', { userId: 'u', email: 'e' }, DEFAULT_CONFIG.auth.pbkdf2Iterations, rnd);
  const ms = Date.now() - t0;
  console.log(`PBKDF2 ${DEFAULT_CONFIG.auth.pbkdf2Iterations} iteraciones en Node: ${ms} ms`);
  assert.ok(ms >= 0);
});
```

E.6 tests/validation.test.ts

```
import { test } from 'node:test';
import assert from 'node:assert/strict';
import { validatePassword, validateRegistration } from '../src/auth/validation';
import { API_ERROR_MESSAGES, APP_ERROR_MESSAGES } from '../src/ui/messages';
test('política de contraseña', () => {
  assert.equal(validatePassword('abc123', 8), 'Debe tener al menos 8 caracteres.');
```

```
  assert.equal(validatePassword('abcdefgh', 8), 'Debe incluir letras y números.');
```

```
  assert.equal(validatePassword('Riachuelo2026', 8), null);
});
test('registro válido e inválido', () => {
  const ok = validateRegistration({ fullName: 'Oscar Gutiérrez', email: 'oscar@ejemplo.com', phone: '+51 987654321', password: 'Vid2026abc', confirm: 'Vid2026abc', accepted: true }, 8);
```

```

    assert.deepEqual(ok, []);
    const bad = validateRegistration({ fullName: '0', email: 'x', phone: '12', password: 'a', confirm: 'b', accepted: false }, 8);
    assert.deepEqual(bad.map((e) => e.field).sort(), ['accepted', 'confirm', 'email', 'fullName', 'password', 'phone']);
  });
  test('todos los mensajes tienen texto', () => {
    for (const v of [...Object.values(API_ERROR_MESSAGES), ...Object.values(APP_ERROR_MESSAGES)]) assert.ok(v.length > 1);
  });
  import { buildPilotRows, PILOT_PLANTS_PER_ROW } from '../src/api/mock/pilotRows';
  test('datos del archivo de mapeo para el backend simulado', () => {
    const rows = buildPilotRows();
    assert.equal(rows.length, 31 + 16 + 36);
    const sum = (a: readonly number[]) => a.reduce((x, y) => x + y, 0);
    assert.equal(sum(PILOT_PLANTS_PER_ROW.SWG1), 9986);
    assert.equal(sum(PILOT_PLANTS_PER_ROW.SWG2), 4841);
    assert.equal(sum(PILOT_PLANTS_PER_ROW.SWG5), 4832);
    assert.equal(rows[0].id, 'SWG1-H01');
    assert.equal(new Set(rows.map((r) => r.id)).size, rows.length);
  });

```

E.7 tests/retry.test.ts

```

import { test } from 'node:test';
import assert from 'node:assert/strict';
import { classifyLocalTransfer, classifySyncError, nextDelayMs } from '../src/sync/retry';
import type { LocalCaptureResponse } from '../src/protocol/messages';
test('espera creciente con tope', () => {
  const d = [2000, 4000, 8000, 16000];
  assert.equal(nextDelayMs(1, d), 2000);
  assert.equal(nextDelayMs(4, d), 16000);
  assert.equal(nextDelayMs(9, d), 16000);
  assert.equal(nextDelayMs(9, d, 10000), 10000);
});
test('clasificación de errores de sincronización', () => {
  assert.equal(classifySyncError({ networkError: true }), 'REINTENTAR');
  assert.equal(classifySyncError({ networkError: false, status: 503 }), 'REINTENTAR');
  assert.equal(classifySyncError({ networkError: false, status: 429 }), 'REINTENTAR');
  assert.equal(classifySyncError({ networkError: false, status: 401, code: 'TOKEN_EXPIRED' }), 'RENOVAR_TOKEN');
  assert.equal(classifySyncError({ networkError: false, status: 401, code: 'REFRESH_INVALID' }), 'REQUIERE_LOGIN');
  assert.equal(classifySyncError({ networkError: false, status: 403, code: 'DEVICE_REVOKED' }), 'REQUIERE_LOGIN');
  assert.equal(classifySyncError({ networkError: false, status: 404, code: 'PASS_NOT_FOUND' }), 'SINCRONIZAR_PADRE');
  assert.equal(classifySyncError({ networkError: false, status: 409, code: 'CAPTURE_CONFLICT' }), 'DEFINITIVO');
  assert.equal(classifySyncError({ networkError: false, status: 400, code: 'VALIDATION_ERROR' }), 'DEFINITIVO');
});
test('decisión tras un intento de transferencia local', () => {
  const rejected = (reason: LocalCaptureResponse['reason']) => ({ networkError: false as const, response: { result: 'REJECTED' as const, reason } });
  assert.equal(classifyLocalTransfer({ networkError: true }, 0, 3), 'REINTENTAR');
  assert.equal(classifyLocalTransfer({ networkError: false, response: { result: 'RECEIVED', reason: null } }, 0, 3), 'RECIBIDA');
  assert.equal(classifyLocalTransfer({ networkError: false, response: { result: 'ALREADY_RECEIVED', reason: null } }, 2, 3), 'RECIBIDA');
  assert.equal(classifyLocalTransfer(rejected('MD5_MISMATCH'), 0, 3), 'REINTENTAR');
  assert.equal(classifyLocalTransfer(rejected('SIZE_MISMATCH'), 1, 3), 'REINTENTAR');
  assert.equal(classifyLocalTransfer(rejected('MD5_MISMATCH'), 2, 3), 'ERROR_LOCAL');
  assert.equal(classifyLocalTransfer(rejected('NO_SPACE'), 0, 3), 'REINTENTAR');
  assert.equal(classifyLocalTransfer(rejected('INTERNAL'), 0, 3), 'REINTENTAR');
  for (const r of ['UNKNOWN_SESSION', 'UNKNOWN_SEQUENCE', 'WRONG_DEVICE', 'CAPTURE_CONFLICT', 'INVALID_META', 'LENGTH_REQUIRED', 'PAYLOAD_TOO_LARGE', 'NOT_FOUND'] as const) {
    assert.equal(classifyLocalTransfer(rejected(r), 0, 3), 'ERROR_LOCAL', r);
  }
});

```